DOKUMENTASI TEKNIS · EDISI LENGKAP · 29 OKTOBER 2026

Kato Report

Pelaporan dan penanganan kebersihan kampus: keluhan dikelompokkan dan diprioritaskan oleh LLM, penyelesaiannya dibuktikan dengan foto yang diperiksa model vision, dan pekerjaan harian petugas dicatat dengan foto kamera, waktu server, dan GPS
UPI Kampus Tasikmalaya · `ai-complaint.yvrtz.workers.dev` · `github.com/rdnsa/AI-complaint`

Daftar isi

1. Ringkasan sistem
2. Tumpukan teknologi dan pembagian kerja dua model
3. Arsitektur dan struktur proyek
4. Model data: tabel, kolom, migrasi, seed
5. Lokasi dan kode QR
6. Alur keluhan dari ujung ke ujung
7. Tugas harian dan laporan kebersihan
8. Analisis LLM teks, ringkasan harian, dan Kato Bot
9. Verifikasi foto (LLM vision)
10. Akun, sesi, dan wewenang
11. Jejak audit dan filter waktu
12. Ringkasan harian, statistik, dan grafik dashboard
13. Papan publik dan papan peringkat
14. Penyimpanan foto
15. Referensi API lengkap
16. Antarmuka pengguna
17. Caching dan performa
18. Konfigurasi, variabel, dan rahasia
19. Pengembangan lokal
20. Penyebaran dan operasi
21. Pemantauan dan pemecahan masalah
22. Keputusan rancangan yang berbiaya
23. Batasan yang diketahui dan pekerjaan lanjutan
24. Lampiran

1. Ringkasan sistem

Kato Report adalah aplikasi web untuk melaporkan dan menangani kondisi kebersihan **lokasi mana pun di kampus**: lantai sebuah gedung, halaman gedung, gedung yang dibersihkan utuh, atau tempat di luar gedung seperti Lapang Voli. Siapa saja (*pelapor*) memindai QR di lokasi itu, menulis keluhan dengan bahasa sehari-hari, dan memotret keadaannya dengan kamera langsung. *Pretrained LLM* mengubah keluhan itu menjadi tiket kerja terstruktur: kategori, prioritas, ringkasan netral, dan rekomendasi tindakan. Petugas kebersihan memakai satu halaman **tanpa login** (nama dipilih dari daftar, ditanyakan saat bertindak) untuk dua jenis pekerjaan: **Keluhan** (menangani laporan pelapor sampai selesai, hanya dengan foto bukti yang **dinilai bersih dan benar lokasinya oleh model vision**) dan **Tugas harian** (enam tugas tetap per lokasi, tiap tugas difoto dari kamera langsung dan dicap nama, lokasi, waktu server, dan GPS). Supervisor (SPV) satu-satunya pihak yang masuk dengan akun; ia memantau lewat dashboard lima tab. **Kato Bot**, asisten tanya-data berbasis *tool calling*, mengambang di setiap halaman. Tidak ada pelatihan model dan tidak ada dataset yang perlu dilabeli; kendali mutu dilakukan lewat rekayasa prompt dan validasi keluaran.

```
QR per lokasi (/report/<id>)  →  tulis keluhan + foto kamera  →  simpan  →  LLM teks (DeepSeek),
setelah respons terkirim
                                                                          ↓
        tab Keluhan (petugas, tanpa login) + dashboard SPV  +  cron 17.00 WIB → ringkasan harian
                      ↓
 "Saya kerjakan sekarang" / "Ambil alih"  →  "Sudah beres": foto bukti  →  LLM vision (Gemini):
bersih dan benar lokasi?  →  selesai

 tab Tugas harian (petugas): pilih nama + lokasi → tiap tugas: foto kamera (cap
nama/lokasi/waktu/GPS) → cek umur foto,
                           pakai-ulang, radius kampus → laporan kebersihan → rekap harian per
petugas untuk SPV
```

Pihak	Peran di kode	Masuk?	Yang dilakukan
Pelapor (mahasiswa, dosen, staf, tamu)	`reporter`	Tidak. Akun opsional, hanya agar laporan masuk papan peringkat	Melapor dari `/report/<id>` (QR) atau Beranda → “Saya pelapor” (`/student`)
Petugas kebersihan	`staff`	**Tidak pernah**; memilih nama dari dropdown	Menangani keluhan sampai selesai dengan foto bukti; mengirim tugas harian per lokasi
SPV (supervisor)	`supervisor`	Ya (`/login`)	Memantau, menghapus spam, menganalisis ulang, mengelola nama petugas dan akun SPV, mencetak QR

Fitur inti (kontribusi penelitian)	Fitur pendukung
1. **Klasifikasi otomatis** ke enam kategori, boleh lebih dari satu 2. **Penentuan prioritas** rendah/sedang/tinggi berbasis risiko keselamatan dan keterpakaian lokasi 3. **Ringkasan harian** satu paragraf, otomatis pukul 17.00 WIB 4. **Papan peringkat pelapor** beserta jumlah laporan yang benar-benar selesai 5. **Verifikasi foto**: benar lokasi yang dilaporkan, dan sudah bersih? — syarat menutup keluhan 6. **Kato Bot**: pertanyaan bahasa sehari-hari dijawab model lewat *tool calling* ke query agregat 7. **Tugas harian** dengan bukti foto kamera, cap waktu server, dan pemeriksaan GPS terhadap radius kampus	• Papan laporan publik (keluhan dan laporan kebersihan), dapat dibaca siapa saja • Semua foto hanya dari kamera langsung di halaman, tanpa pilihan galeri • Klaim keluhan yang bukan kunci: petugas lain dapat *mengambil alih*; semuanya tercatat • Jejak audit yang tidak dapat dihapus dari aplikasi • Dashboard SPV: ringkasan AI, grafik, rekap petugas, filter waktu menit (WIB), stiker QR • Pelacakan status untuk pelapor tanpa akun • Peta gedung; antarmuka dwibahasa ID/EN; tema gelap (bawaan) dan terang

Angka yang dapat diperiksa di repositori

Hal	Nilai	Sumber
Penangan HTTP	37 (36 titik akhir API + `/api/health`) dalam 11 modul route	`src/index.ts`, `src/routes/*.ts`
Migrasi D1	11 (0001–0011)	`migrations/`
Tabel dan view	11 tabel (`buildings, toilets, users, reports, work_logs, cleaning_areas, cleaning_assignments,`	`migrations/0008, 0009, 0011`

	`cleaning_reports, cleaning_report_tasks, daily_summaries, activity_log`) dan 2 view (`toilet_info, location_info`)	
Lokasi aktif (area kebersihan)	29: 9 halaman gedung (A–I), 19 lantai, 1 tempat di luar gedung (Lapang Voli)	`seed/cleaning-areas.sql`
Toilet aktif (data lama)	26, di gedung A, B, C, G, H, I	`seed/toilets.sql`
Nama petugas di seed	11 (10 punya penugasan area; satu tidak)	`seed/cleaning-areas.sql`
Tugas harian / kategori / prioritas / status / putusan bukti	6 / 6 / 3 / 3 / 3	`src/domain/types.ts`
Jenis aksi jejak audit	14	`src/repositories/activity.ts`
Rute halaman	12 rute berhalaman, 12 pengalihan (11 URL lama + `/clean`), 9 berkas halaman	`web/src/main.tsx`, `web/src/pages/`
Dependensi runtime Worker	2 (`hono`, `zod`); sisanya `devDependencies` (dibundel ke frontend atau alat bangun)	`package.json`

2. Tumpukan teknologi dan pembagian kerja dua model

Lapisan	Pilihan	Alasan
Bahasa	TypeScript 5, mode `strict`	Binding D1 dan R2 hanya tersedia di runtime JS/TS Workers
Runtime	Cloudflare Workers, `nodejs_compat` (`compatibility_date 2025-08-01`)	Tanpa server yang dikelola; satu Worker melayani API dan aset statis
Framework API	Hono 4	Router ringan yang berjalan native di Workers; middleware cookie, JWT, ETag, dan logger bawaan
Validasi	Zod 3	Satu pustaka untuk body HTTP dan untuk keluaran LLM
Basis data	Cloudflare D1 (SQLite), binding `DB`, database `kato`	Migrasi ter-versi, `json_each` untuk kolom kategori, `batch()` untuk penulisan atomik

Penyimpanan berkas	Cloudflare R2, binding `BUCKET`, bucket `kato`	Diakses lewat binding, disajikan ulang oleh Worker (bab 14)
Frontend	React 18 + Vite 6 + Tailwind 3, react-router 6	Dilayani Worker yang sama (`assets` + `single-page-application`), sehingga tanpa CORS
Kode QR	Pustaka `qrcode`	Di browser (tab QR lokasi) dan di skrip CLI (`npm run qr`)
i18n	Kamus buatan sendiri (`web/src/lib/i18n.tsx`)	Bahasa Indonesia mendefinisikan bentuk kamus; TypeScript menolak kunci yang belum diterjemahkan
LLM teks	DeepSeek `deepseek-chat` (`https://api.deepseek.com`)	JSON mode dan *function calling*; tidak bisa membaca gambar
LLM vision	Google Gemini `gemini-3.6-flash` lewat endpoint kompatibel OpenAI (`generativelanguage.googleapis.com/v1beta/openai`)	Menilai foto bukti; endpoint kompatibel OpenAI sehingga penyedia dapat diganti lewat variabel, tanpa mengubah kode
Dokumen	HTML + Chrome headless → PDF	Satu sumber untuk layar dan cetak (`npm run docs`)

Pembagian kerja dua model

Kedua model tidak pernah dipanggil untuk pekerjaan yang sama. Pemisahannya mengikuti jenis masukan: teks ke DeepSeek, gambar ke Gemini. Nama model dan alamat penyedia adalah variabel di `wrangler.jsonc` (`LLM_MODEL`, `VISION_MODEL`, dst.), bukan konstanta di kode.

	DeepSeek `deepseek-chat`	**Gemini `gemini-3.6-flash`**
Masukan	Teks keluhan beserta nama lokasi; daftar keluhan sehari; pertanyaan Kato Bot dan hasil alatnya	Foto bukti dari petugas + konteks: nama lokasi, kategori, dan teks keluhan asli
Pekerjaan	(1) kategori, (2) prioritas + ringkasan + rekomendasi, (3) ringkasan harian, (4) Kato Bot lewat *tool calling*	(5) Menilai foto penutup keluhan: benar lokasinya? sudah bersih?
Kapan dipanggil	Tiap keluhan masuk (asinkron, setelah respons dikirim); sekali sehari lewat cron atau tombol “Buat ulang”; tiap pertanyaan Kato Bot (sinkron, maks. 50/hari)	Tiap petugas menyelesaikan keluhan dengan foto (sinkron; petugas menunggu putusan). **Tugas harian tidak diperiksa model**

Parameter	`temperature 0.1`, `max_tokens` 500 (analisis) / 700 (ringkasan), `response_format json_object`, *timeout* 30 dtk; Kato Bot: `temperature 0.2`, `max_tokens 600`, `tool_choice auto` (maks. 4 putaran)	`temperature 0.1`, `max_tokens 2000`, `response_format json_object`, gambar `detail: low`, *timeout* 30 dtk
Bila gagal	Keluhan tetap tersimpan tanpa label (`ai_status='failed'`), dapat dianalisis ulang; Kato Bot membalas 502	Penutupan ditolak (502); foto dibuang; petugas diminta mencoba lagi
Fungsi di kode	`analyzeComplaint(), summarizeDay(), chatWithTools()`	`checkProofPhoto()`
Konfigurasi	`LLM_BASE_URL, LLM_MODEL, LLM_API_KEY`	`VISION_BASE_URL, VISION_MODEL, VISION_API_KEY (bila kosong, memakai LLM_API_KEY)`
Hasil disimpan di	`reports.categories, priority, summary, recommendation, ai_*; daily_summaries; activity_log (question)`	`reports.proof_*; activity_log (proof_rejected, verification_failed)`
Biaya dan latensi	Belum diringkas di dokumen ini. Datanya tersedia di basis data: `reports.ai_ms` dan `proof_ms` (latensi per panggilan), serta jumlah token tiap pertanyaan Kato Bot di `activity_log.details`.	

Analisis, ringkasan, dan pemeriksaan foto dipanggil lewat satu fungsi dasar, `chatJSON()` di `src/adapters/llm.ts`, karena kedua penyedia memakai format *chat completions* OpenAI. Yang berbeda hanya tujuan (URL, model, kunci) dan isi pesan. Kato Bot memakai fungsi kedua, `chatWithTools()`, yang menjalankan putaran *tool calling* (bab 8). Keduanya memanggil `postChat()`, yang **berpindah ke kunci berikutnya** bila satu kunci ditolak dengan 401, 403, atau 429: secret `LLM_API_KEY` dan `VISION_API_KEY` boleh berisi beberapa kunci dipisah koma.

Bahasa. Prosa prompt berbahasa Indonesia. Teks yang ditulis model untuk dibaca petugas dan SPV (ringkasan, rekomendasi, alasan penilaian foto, ringkasan harian) berbahasa Indonesia, apa pun bahasa antarmukanya; satu-satunya pengecualian adalah Kato Bot, yang menjawab dalam bahasa penanya (Indonesia atau Inggris). Kunci JSON dan token enum yang diminta dari model berbahasa Inggris (mis. `"priority":"high"`), dan prompt menyebutnya secara eksplisit.

3. Arsitektur dan struktur proyek

Backend disusun berlapis dengan arah ketergantungan hanya ke dalam. Semua nama berkas, pengenal, tabel, kolom, nilai enum, path API, dan URL berbahasa Inggris; teks yang dibaca pengguna (label antarmuka, pesan galat server, prosa prompt LLM, nama lokasi) tetap berbahasa Indonesia.

```
routes/  →  services/  →  repositories/  →  D1
                ↓
           adapters/            domain/
```

Lapisan	Tanggung jawab	Aturan yang dipatuhi
`domain/`	Kosakata (enum peran, kategori, prioritas, status, putusan bukti, tugas harian), entitas, pemetaan DTO, aturan penutupan (`canResolve`), rumus jarak dan status lokasi (`distanceMeters` , `locationStatus`)	Tidak mengimpor apa pun — tanpa Hono, D1, R2
`repositories/`	Seluruh pernyataan SQL, satu berkas per agregat	Tahu basis data, tidak tahu HTTP
`services/`	*Use case* dan aturan bisnis	Tidak menyebut Hono sama sekali
`adapters/`	LLM (teks + vision), R2, hash sandi, sesi JWT, konversi waktu WIB, cache memori	Membungkus dunia luar di balik antarmuka kecil
`routes/`	Parse, validasi, delegasi, format respons	Nol SQL; aturan bisnis hanya yang menyangkut bentuk permintaan
`index.ts`	*Composition root*: pasang route, validator ETag, fallback SPA, penangan cron	Satu-satunya berkas yang melihat semua lapisan

Konsekuensi yang dapat diuji: seluruh pernyataan SQL berada di `repositories/` . Berpindah dari D1 ke Postgres berarti menulis ulang satu direktori, bukan seluruh berkas route. Aturan “keluhan hanya boleh ditutup bila ada bukti *yang lolos verifikasi*” tinggal di lapisan domain (`canResolve(status, proofPhoto, proofVerdict)`), sehingga setiap pemanggil memutuskan dengan cara yang sama. Dua pengecualian jujur: `services/*` memanggil beberapa repositori sekaligus (mis. `report-service` memakai `repositories/cleaning` untuk memeriksa lokasi), dan `adapters/memo.ts` menyimpan cache yang dipakai lintas lapisan.

Struktur berkas

```
src/
  index.ts                 composition root: routes, ETag lemah, fallback SPA, penangan cron
  env.ts                   binding infrastruktur dan konfigurasi (Env, AppEnv)
  domain/types.ts          enum, ReportRow/ReportDTO, WorkLogRow/WorkLogDTO,
CleaningReportRow/DTO,
                           photoUrl(), toDTO(), canResolve(), distanceMeters(), locationStatus()
  adapters/
    llm.ts                 prompt, few-shot, validasi keluaran, postChat() (failover kunci),
chatJSON(),
                           chatWithTools(), analyzeComplaint(), summarizeDay(), checkProofPhoto()
    storage.ts             R2: prefix baru + lama, batas ukuran/tipe,
savePhoto/getPhoto/readPhoto/photoUploadedAt/deletePhoto
    session.ts             JWT dalam cookie `session`, currentSession(), requireSupervisor
    password.ts            PBKDF2-SHA256 lewat WebCrypto, perbandingan waktu-konstan
    clock.ts               wibDate(), wibDayRange(), isValidDate()
    memo.ts                cache memori per-isolate: memo(), forget(), reportsChanged(),
cleaningChanged(), HTTP_CACHE
  repositories/
    reports.ts             keluhan: daftar dashboard, publik, terbuka, milik pelapor, kembar,
analisis
    cleaning.ts            area, penugasan, laporan kebersihan, foto terpakai
    work-logs.ts           laporan pekerjaan lama (read-mostly)
    time-filter.ts         filter waktu bersama (from, to, hour_from, hour_to; WIB, inklusif)
    users.ts               akun SPV/pelapor dan nama petugas
    activity.ts            jejak audit (activity_log) dan 14 jenis aksinya
    summaries.ts           ringkasan harian, kartu angka, data grafik, papan peringkat
    analytics.ts           query agregat untuk Kato Bot (GROUPINGS, filter terikat)
  services/
    report-service.ts      kirim, ubah status (+ verifikasi bukti), hapus keluhan
    analysis-service.ts    analisis satu keluhan, dijalankan setelah respons dikirim
    cleaning-service.ts    daftar area, kirim laporan kebersihan, rekap harian (dailyRecap)
    work-log-service.ts    log pekerjaan lama (hanya baca)
    summary-service.ts     ringkasan harian, statistik, data grafik, papan peringkat
    ask-service.ts         definisi alat, prompt sistem, anggaran harian, pencatatan Kato Bot
    user-service.ts        masuk, daftar, tambah/ubah/nonaktif/hapus akun
    activity-service.ts    tulis (log) dan baca (history) jejak audit
  routes/                  auth, reports, work-logs, cleaning, staff, uploads, summary,
                           activity, users, leaderboard, ask

web/src/
  main.tsx                 router: 12 rute halaman + pengalihan URL lama; memasang FloatingChat
sekali
  lib/api.ts               klien API bertipe, ApiError (status + data), cachedReq() untuk daftar
yang jarang berubah
  lib/i18n.tsx             kamus ID/EN, useLanguage(), useRelativeTime(), useFormatTime()
  lib/session.tsx          konteks sesi (useSession)
  lib/theme.tsx            tema gelap/terang (useTheme), localStorage `theme`
  lib/staff.ts             nama petugas yang diingat HP ini (localStorage `selected-staff`)
  lib/places.ts            buildPlaces(), partOf(), locate(), reportIsAt(): pemilih Gedung /
Lantai-Halaman
  lib/polling.ts           usePolling(): polling yang berhenti saat tab tersembunyi
  lib/server-clock.ts      useServerClock(): selisih jam server vs HP, untuk cap waktu pada foto
  lib/motion.tsx           animasi (Reveal dst.)
  pages/                   Home, ReportForm, ReportStatus, PublicReports, StaffHome, Login,
Register,
                           Leaderboard, SupervisorDashboard
  components/              Header, Camera, FloatingChat, AskPanel, Select, ClockPicker,
TimeFilter,
                           LocationSelect, RoleSwitch, StaffPicker, ComplaintCard, DailyTasks,
Tracker,
```

```
                          Badges, CampusMap, ImageViewer, ZoomableImage, PasswordInput, Marks
                          panel dashboard: OverviewPanel, CleaningPanel, ActivityPanel,
WorkLogPanel,
                          AccountsPanel, QrCodesPanel
    charts/               AgingBars, AreaChart, BarChart, CountUp, Donut, Heatmap, HourClock,
KpiCard,
                          RadialBars, RadialGauge, Sparkline, StackedBarChart, StaffRings,
Treemap,
                          colors.ts (palet), motion.ts
  vite.config.ts          chunk vendor React terpisah, proxy /api ke Worker lokal
  public/                 _headers (cache aset), manifest, ikon, logo, peta-lokasi-gedung.jpg

migrations/               0001–0011 (bab 4)         seed/        toilets.sql, cleaning-areas.sql
scripts/                  generate-qr, build-frontend, build-docs, watch-db, free-dev-port
docs/                     dokumen ini, panduan-umum, prd, penjelasan-diagram, diagram/, bpmn/,
laporan-kemajuan/
design.md                 rujukan gaya (glassmorphism di atas gaya Apple)
backup.sql                ekspor D1 lama berskema Indonesia (sebelum migrasi 0008), ikut tercatat
di git
```

4. Model data: tabel, kolom, migrasi, seed

Basis data D1 (SQLite), skema berbahasa Inggris sejak migrasi 0008. Semua stempel waktu disimpan UTC dalam format `YYYY-MM-DD HH:MM:SS`; konversi ke hari WIB dilakukan di `adapters/clock.ts` dan di SQL lewat `'+7 hours'`. Nilai enum disimpan sebagai token bahasa Inggris; teks bebas (keluhan, ringkasan AI, kalimat jejak audit, nama lokasi) tetap bahasa Indonesia.

> **Data lama dan data baru berdampingan.** Sistem dimulai sebagai pelaporan toilet. Tabel `toilets` dan `buildings` masih ada, begitu pula `work_logs` yang menunjuk toilet, dan laporan lama tetap menunjuk toilet (`reports.toilet_id`). Laporan baru menunjuk **area kebersihan** (`reports.area_id`). Kedua jenis lokasi dibaca lewat satu view, `location_info`, sehingga tidak ada query yang perlu membedakan. Yang berubah hanya di mana lokasi *baru* didaftarkan (`seed/cleaning-areas.sql`).

buildings

Kolom	Tipe	Keterangan
`code`	TEXT PK	Satu huruf, A–J
`name`	TEXT NOT NULL	Nama dari poster “Peta Lokasi Gedung” (A Ki Hajar Dewantara, B Masjid At-Tarbiyah, C Dewi Sartika, D KH. Ahmad Dahlan, E Dr. Wahidin Sudirohusodo, F Daoed Joesoef, G RA. Kartini, H Mohamad Yamin, I Dr. Sutomo, J KH. Moh. Hasyim Ashari)
`sort_order`	INTEGER	Urutan tampil (bawaan 0); diisi 1–10 oleh migrasi 0002

cleaning_areas (lokasi yang dilaporkan)

Kolom	Tipe	Keterangan
`id`	TEXT PK	Yang dibawa QR: `C-2`, `C-HALAMAN`, `MASJID`, `LAPANG-VOLI`. Gedung A memakai awalan `ADM-` (`ADM-1` … `ADM-4`, `ADM-HALAMAN`) agar stiker dari versi sebelumnya tetap berlaku; lantai tunggal Gedung B bernama `MASJID`
`name`	TEXT NOT NULL	Nama tampilan, mis. “Gedung Dewi Sartika (Perkuliahan) - Lantai 2”
`building_code`	TEXT FK → buildings, NULL	NULL untuk tempat di luar gedung (Lapang Voli)
`floor`	INTEGER, NULL	`0` = halaman gedung, `NULL` = seluruh tempat (gedung yang dibersihkan utuh, atau lapangan), `n` = lantai n
`tasks`	TEXT NOT NULL (JSON)	Tugas harian yang berlaku di sini. Migrasi 0010 menyamakannya di semua area: `["sweep_mop","glass","toilet","trash","gallon","tidy"]`
`sort_order`	INTEGER	Urutan (A: 10–14, B: 20–21, …, Lapang Voli 900)
`active`	INTEGER	0/1. Area yang keluar dari daftar dinonaktifkan, tidak dihapus, agar riwayat tetap terbaca

cleaning_assignments

Kolom	Tipe	Keterangan
`area_id, staff_id`	TEXT, PK gabungan	Siapa ditugaskan di mana (surat penempatan 2026). Beberapa petugas boleh berbagi satu area; satu petugas boleh memegang banyak area. FK ke `cleaning_areas` dan `users`. Indeks `idx_cleaning_assignments_staff`

reports (keluhan)

Kolom	Tipe	Keterangan
`id`	TEXT PK	UUID; juga kunci halaman status `/reports/<id>`
`area_id / toilet_id`	TEXT FK, NULL	Tepat satu terisi (`CHECK ((area_id IS NULL) <> (toilet_id IS NULL))`): area untuk laporan sejak migrasi 0011, toilet untuk laporan sebelumnya
`description`	TEXT NOT NULL	Keluhan mentah (5–1000 karakter menurut validasi)
`photo_key`	TEXT	Kunci R2 foto keadaan (`report/…`). Wajib pada laporan baru
`status`	TEXT (CHECK)	`new` | `in_progress` | `resolved`; bawaan `new`
`staff_name`	TEXT	Nama petugas yang memegang atau menutup, disalin sebagai teks (bukan FK) agar tetap benar bila akun berubah
`resolved_at`	TEXT	Diisi saat `resolved`; dikosongkan lagi bila status bukan `resolved`
`ai_status`	TEXT (CHECK)	`pending` | `ok` | `failed`
`categories`	TEXT (JSON)	Larik, mis. `["odor","flooding"]`; nilai: `cleanliness, supplies, damage, odor, flooding, other`
`priority`	TEXT (CHECK)	`low` | `medium` | `high`; NULL sebelum dianalisis
`summary, recommendation`	TEXT	Dari LLM; maks. 300 dan 500 karakter setelah dipotong
`ai_error, ai_model, ai_ms`	TEXT, TEXT, INTEGER	Galat terakhir (maks. 500 karakter), model, dan latensi panggilan
`proof_photo_key`	TEXT	Foto bukti penyelesaian (`proof/…`); hanya foto yang lolos yang pernah disimpan
`proof_verdict`	TEXT (CHECK)	`clean` | `dirty` | `wrong_place`. Dalam praktik hanya `clean` yang tersimpan. Pada data lama, `not_toilet` diubah menjadi `wrong_place` oleh migrasi 0011

`proof_reason, proof_model, proof_ms`	TEXT, TEXT, INTEGER	Alasan satu kalimat dari model, nama model, latensi
`reporter_id`	TEXT FK → users, NULL	NULL untuk pelapor anonim; hanya laporan berpemilik yang masuk papan peringkat
`created_at, updated_at`	TEXT	UTC

Indeks: `idx_reports_created (created_at DESC)`, `idx_reports_status (status, created_at DESC)`, `idx_reports_area (area_id, created_at DESC)`, `idx_reports_toilet (toilet_id, created_at DESC)`, `idx_reports_ai (ai_status) WHERE ai_status <> 'ok'` (indeks parsial: hanya baris yang belum beres), dan `idx_reports_reporter (reporter_id)`.

cleaning_reports dan cleaning_report_tasks (tugas harian)

Kolom	Tipe	Keterangan
`cleaning_reports`: satu baris per pengiriman		
`id`	TEXT PK	UUID
`area_id, staff_id`	TEXT FK	Ke `cleaning_areas` dan `users`
`staff_name`	TEXT NOT NULL	Nama saat mengirim
`note`	TEXT	Catatan opsional (maks. 500 karakter), mis. “air mati”
`latitude, longitude, accuracy_m`	REAL, NULL	Posisi GPS seperti dilaporkan HP; NULL bila HP tidak memberi posisi
`distance_m`	REAL, NULL	Jarak ke pusat kampus, dihitung server (haversine)
`location_status`	TEXT (CHECK)	`ok` | `far` | `unknown` (bab 7)
`created_at`	TEXT	UTC
`cleaning_report_tasks`: satu baris per tugas yang dicentang, masing-masing dengan fotonya		
`report_id, task`	PK gabungan	`task` berbatas CHECK: `sweep_mop, glass, toilet, trash, gallon, tidy`, ditambah `sweep` dan `mop` yang hanya ada pada laporan sebelum migrasi 0010
`photo_key`	TEXT NOT NULL **UNIQUE**	Satu foto tidak dapat membuktikan dua tugas, atau dua laporan (`cleaning/…`)
`photo_at`	TEXT NOT NULL	Kapan server menerima foto (UTC), diambil dari cap unggah R2, bukan dari jam HP

Indeks pada `cleaning_reports`: `idx_cleaning_reports_created`, `_staff (staff_id, created_at DESC)`, `_area (area_id, created_at DESC)`, dan `idx_cleaning_reports_alerts (created_at DESC) WHERE location_status <> 'ok'` (parsial, untuk daftar peringatan).

users

Kolom	Tipe	Keterangan
`id`	TEXT PK	UUID (akun SPV bawaan: `adm-0001`)
`username`	TEXT UNIQUE COLLATE NOCASE, NULL	'Budi' dan 'budi' tidak boleh menjadi dua akun. NULL untuk petugas; UNIQUE di SQLite mengizinkan banyak NULL
`name`	TEXT NOT NULL	Nama tampilan (juga nama yang masuk jejak audit)
`role`	TEXT (CHECK)	`supervisor` | `staff` | `reporter`
`password_hash, password_salt`	TEXT, NULL	PBKDF2-SHA256 heksadesimal; NULL untuk petugas. `CHECK (role = 'staff' OR (username, password_hash, password_salt semuanya NOT NULL))`
`active`	INTEGER	0/1; akun yang dinonaktifkan tidak dapat masuk dan hilang dari dropdown petugas
`created_at`	TEXT	

Indeks `idx_users_role (role, active)`. Akun SPV bawaan `admin` disisipkan oleh migrasi 0004 dan ikut berubah peran (`admin → spv → supervisor`) lewat migrasi 0007 dan 0008; kata sandi awalnya tertulis di komentar migrasi 0004 dan **harus diganti** pada masuk pertama.

work_logs (laporan pekerjaan lama)

Kolom	Tipe	Keterangan
`id, toilet_id, staff_id, staff_name`		`toilet_id` menunjuk `toilets`; `staff_id` ke `users`
`description, photo_key`	TEXT	Apa yang dikerjakan, dalam kata petugas; foto (`work/…` atau `kerja/…` pada data lama)
`proof_verdict, proof_reason, proof_model, proof_ms`		Selalu `clean` pada data tersimpan
`created_at`	TEXT	Indeks pada `created_at`, `toilet_id`, dan `staff_id`

daily_summaries dan activity_log

Kolom	Tipe	Keterangan
`daily_summaries`		
`date`	TEXT PK	`YYYY-MM-DD` WIB; ditulis dengan `ON CONFLICT(date) DO UPDATE`, sehingga "Buat ulang" menimpa
`report_count, summary, highlights, created_at`		`highlights` berupa larik JSON string (maks. 4)

activity_log: sengaja *tanpa foreign key* ke `reports`		
`id`	INTEGER PK AUTOINCREMENT	
`created_at`	TEXT	UTC
`action`	TEXT	Salah satu dari 14 jenis (bab 11)
`report_id`	TEXT, NULL	Tetap tercatat walau keluhannya sudah dihapus
`actor`	TEXT	Nama seseorang, atau `reporter` / `system` / `visitor`
`summary, details`	TEXT, TEXT (JSON)	Satu kalimat yang langsung terbaca; rincian terstruktur (mis. salinan keluhan yang dihapus)

Indeks: `idx_activity_created`, `idx_activity_report`, `idx_activity_action (action, created_at DESC)`.

View: toilet_info dan location_info

`toilet_info` menyusun nama tampilan toilet di satu tempat, mis. “WC Pria - Lantai 2, Gedung A (Ki Hajar Dewantara)” (kolom: `id, building_code, building_name, building_order, floor, type, active, name`). `location_info` (migrasi 0011) menggabungkannya dengan area kebersihan lewat `UNION ALL` pada kolom yang sama; untuk area, `type` bernilai NULL dan `building_order` bernilai 999 bila tidak punya gedung. Semua query laporan bergabung dengan pola yang sama:

```
FROM reports r JOIN location_info t ON t.id = COALESCE(r.area_id, r.toilet_id)
```

Id area dan id toilet tidak pernah bertabrakan karena id toilet selalu berakhir dengan jenis toilet (`A–2–MEN`). View tidak menyaring `active`, sehingga laporan atas lokasi yang kemudian dinonaktifkan tetap tampil; penyaringan `active = 1` hanya dilakukan saat *menerima* laporan baru (`activeArea()`).

Migrasi

Berkas	Isi
`0001_init.sql`	Skema awal berbahasa Indonesia: `toilets`, `reports` (status `baru/diproses/selesai`, kolom AI), `daily_summaries`
`0002_gedung_upi.sql`	Nama gedung UPI Tasikmalaya (A–J) dalam tabel sendiri; satu QR per lantai (bukan per toilet); view `toilet_info`
`0003_bukti_dan_aktivitas.sql`	Foto bukti penyelesaian; jejak audit (tanpa FK ke keluhan)
`0004_pengguna.sql`	Akun berperan menggantikan satu sandi bersama; `pelapor_id` pada keluhan; akun admin bawaan (PBKDF2, 100.000 iterasi)
`0005_verifikasi_bukti.sql`	Putusan model vision atas foto bukti disimpan di keluhan

`0006_laporan_pekerjaan.sql`	Tabel `pekerjaan` (laporan pekerjaan petugas), terpisah dari keluhan agar tidak mengacaukan statistik
`0007_peran_spv.sql`	Tiga pihak: `admin` menjadi `spv`; petugas tidak lagi masuk (username dan sandi opsional, `CHECK` baru). Tabel dibangun ulang dengan trik `defer_foreign_keys` dan salinan sementara
`0008_english_schema.sql`	Seluruh skema diterjemahkan ke bahasa Inggris (tabel, kolom, nilai enum, isi kolom JSON, id toilet `-PRIA` → `-MEN`); tabel dibangun ulang anak dahulu. Foto di R2 tidak dipindah: awalan lama (`laporan/ bukti/ kerja/`) tetap dapat dibaca
`0009_cleaning_reports.sql`	`cleaning_areas`, `cleaning_assignments`, `cleaning_reports`, `cleaning_report_tasks`: laporan kebersihan per area dengan foto per tugas dan status GPS
`0010_daily_tasks.sql`	Satu daftar enam tugas di semua area (`sweep` + `mop` menjadi `sweep_mop`; ditambah `trash, gallon, tidy`); `CHECK` tugas dibangun ulang tetapi tetap menerima nilai lama
`0011_report_locations.sql`	Keluhan atas lokasi mana pun: `area_id` + `toilet_id` (tepat satu), `not_toilet` → `wrong_place`, view `location_info`; `reports` dibangun ulang karena SQLite tidak dapat melepas NOT NULL atau mengubah CHECK

Migrasi diterapkan dengan `npm run db:local` / `npm run db:remote` (`wrangler d1 migrations apply kato`). Pola yang berulang: SQLite tidak dapat mengubah `CHECK` atau `NOT NULL`, jadi tabel dibangun ulang, anak dahulu; D1 tidak dapat mematikan foreign key, hanya menundanya (`PRAGMA defer_foreign_keys = true`).

Seed

Dua berkas, keduanya aman dijalankan ulang kapan pun, termasuk setelah laporan ada.

Berkas / perintah	Isi dan perilaku
`seed/cleaning-areas.sql` `npm run db:seed:cleaning` / `:remote`	Sumber kebenaran lokasi hari ini, dari surat “Penempatan Lokasi Kerja PHL dan OB 2026”. Menonaktifkan semua area, lalu *upsert* 29 area (`ON CONFLICT(id) DO UPDATE … active = 1`); menambah 11 nama petugas hanya bila belum ada petugas aktif bernama sama (tanpa membedakan huruf besar); **membangun ulang** penugasan dari nol (`DELETE FROM cleaning_assignments` lalu isi). Gedung A lantai 1, E, dan F belum punya petugas; satu petugas (Romulus, teknisi) bekerja di seluruh kampus dan tidak punya area. Id lama `G`, `D`, `H` dinonaktifkan; frontend mengarahkan tautan lama ke gedungnya
`seed/toilets.sql` `npm run db:seed` / `:remote`	Daftar toilet lama: 26 toilet aktif (pria dan wanita) di gedung A, B, C, G, H, I; menonaktifkan semua dahulu. Tidak diperlukan untuk laporan baru; hanya menjaga nama lokasi pada data lama

5. Lokasi dan kode QR

Yang dilaporkan adalah **lokasi**, dan lokasi adalah baris `cleaning_areas` (bab 4). Satu lokasi punya satu QR. Daftar yang sama dipakai semua halaman (pelapor, petugas, SPV, stiker QR): backend menyajikannya di `GET /api/cleaning/areas` (di-*cache* 5 menit, bab 17), frontend mengubahnya menjadi dua dropdown di `web/src/lib/places.ts`.

Bagian lokasi: whole, yard, lantai

Nilai `floor`	Nama bagian (`partOf()`)	Label di dropdown "Lantai / Halaman"	Contoh id
`NULL`	`whole`	"Seluruh gedung" (bila punya kode gedung) atau "Seluruh area"	`LAPANG-VOLI`
`0`	`yard`	"Halaman"	`C-HALAMAN, ADM-HALAMAN`
`n ≥ 1`	`"n"`	"Lantai n"	`C-2, ADM-4, MASJID`

Dropdown pertama, "Gedung", berisi *tempat* (`Place`): gedung berkode huruf, lalu tempat di luar gedung. `buildPlaces(areas)` mengelompokkan area menurut `placeKeyOf()`: kode gedung bila ada; bila tidak, nama tanpa akhiran " - Lantai n" (diawali `~`), sehingga tempat di luar gedung yang terbagi per lantai tetap satu pilihan. Urutan: gedung menurut huruf, lalu tempat lain; bagian diurutkan *whole*, *yard*, lalu lantai dari bawah (`partRank`). Memilih tempat yang hanya punya satu bagian (Lapang Voli) langsung memilih bagiannya (`LocationSelect.pickPlace`). Pada halaman petugas tab Keluhan, kedua dropdown berfungsi sebagai *filter* ("Semua gedung", "Semua"), bukan pilihan wajib.

Dari QR ke lokasi: `locate(id, areas, places)`

Masukan (bagian URL)	Hasil
Id area persis (tak peka huruf besar-kecil), mis. `C-2`, `ADM-4`, `LAPANG-VOLI`	Tempat dan bagian area itu
Pola `<huruf>-<lantai>` dari stiker yang dicetak sebelum laporan mencakup seluruh kampus (`A-2`)	Lantai itu di gedung berkode huruf tersebut, bila ada
Satu huruf (`G`, id lama yang dinonaktifkan), atau lantai yang tidak punya area	Gedungnya saja; bagian *whole* bila gedung dibersihkan utuh, selain itu bagian dibiarkan kosong untuk dipilih
Selain itu (mis. kode toilet lama `A-1-PRIA`)	`null`: halaman menampilkan "QR tidak dikenal. Pilih lokasi di atas."

`reportIsAt()` memakai pemetaan yang sama untuk menyaring daftar keluhan terbuka di halaman petugas. Untuk keluhan lama (menunjuk toilet), kecocokan dihitung dari kode gedung dan lantai; bagian "halaman" tidak pernah cocok dengan keluhan toilet lama.

Satu QR per lokasi, satu tujuan

Setiap QR berisi `<domain>/report/<id-area>`, untuk pelapor *dan* petugas. Halaman yang terbuka:

1. `/report/<id>` (`ReportForm`) membuka formulir pelapor dengan lokasi sudah terpilih. Di atasnya ada sakelar `RoleSwitch` : “Saya pelapor” (`/report/<id>?as=student`) dan “Saya petugas” (`/staff/<id>`).
2. Bila HP itu pernah memilih nama petugas (`localStorage "selected-staff"`) dan URL tidak berisi `?as=student` , `ReportForm` langsung *mengalihkan* ke `/staff/<id>` : QR yang sama membawa petugas ke halamannya tanpa pilihan tambahan.
3. `/staff/<id>` (`StaffHome`) membuka tab *Keluhan* bila ada keluhan terbuka di lokasi itu, selain itu tab *Tugas harian*. `/clean/<areaId>` (alamat stiker petugas dari versi sebelumnya) langsung membuka tab Tugas harian.
4. Tanpa QR: `/student` (formulir pelapor tanpa lokasi) dan `/staff` (halaman petugas tanpa lokasi), dari Beranda.

Stiker lama tetap berfungsi

Stiker yang sudah tertempel menunjuk `/lapor/<lantai>` (atau `/petugas/…` , `/kerja/…`). Semuanya dialihkan oleh `LegacyRedirect` di `web/src/main.tsx` (`/lapor/:floorId → /report/:floorId` , dan seterusnya; parameter, *query*, dan *hash* ikut; `?sebagai=mahasiswa` menjadi `?as=student`). Agar tautan dalam yang tidak punya berkas statis (mis. `/report/C-2`) tidak dialihkan ke `/` , `wrangler.jsonc` menetapkan `run_worker_first: ["/api/*", "/__scheduled"]` dan `not_found_handling: "single-page-application"` , dan `app.notFound` di `src/index.ts` menyajikan `index.html` untuk path non-API.

Mencetak dari aplikasi: dashboard → tab QR lokasi

`QrCodesPanel` menggambar stiker langsung di browser dari daftar lokasi hidup (jadi lokasi baru di basis data langsung muncul): SVG lewat `QRCode.toString` dengan koreksi kesalahan `M` (tahan stiker tergores). Stiker memuat judul “Lapor kondisi kebersihan”, bagian dan nama lokasi, QR (di atas latar putih agar terbaca di tema gelap), ajakan “Pindai untuk melapor”, dan URL-nya. **Stiker dwibahasa**: karena kertas tidak dapat mengikuti sakelar bahasa, teks dicetak dalam Indonesia dan Inggris sekaligus (kamus `translatorFor('id')` dan `translatorFor('en')` ; satu baris bila kedua bahasa sama, mis. “Lapang Voli”). Ada filter per tempat, tombol *Cetak* (lembar A4 tiga stiker per baris, tidak terpotong lintas halaman), dan *Unduh PNG* per stiker (1024 piksel, margin 2).

Generator baris perintah: `npm run qr`

`scripts/generate-qr.mjs` mengambil `/api/cleaning/areas` dari `BASE_URL` (bawaan `http://127.0.0.1:8787` ; untuk produksi `BASE_URL=https://ai-complaint.yvrtz.workers.dev npm run qr`), **mengosongkan** folder `qr-codes/` (diabaikan git), lalu menulis satu PNG 900 piksel per lokasi (`qr-codes/<id>.png`) dan `qr-codes/cetak.html` (kartu dwibahasa, dua kolom, siap dicetak). Folder dikosongkan karena kode dari daftar lokasi yang lebih lama tidak boleh tertinggal: QR yang sudah tidak terdaftar akan ditolak sistem bila sudah terlanjur ditempel.

6. Alur keluhan dari ujung ke ujung

Foto hanya dari kamera langsung

Setiap foto di aplikasi (keluhan, bukti penyelesaian, tugas harian) diambil dengan komponen `Camera` (`getUserMedia`, kamera belakang sebagai pilihan), tanpa `<input type="file">` dan tanpa pilihan galeri. Bingkai diperkecil (sisi terpanjang maksimal 1600 piksel), dikodekan JPEG kualitas 0,85, dan diunggah ke `POST /api/uploads?kind=…` (`report`, `proof`, `work`, `cleaning`). Tanpa HTTPS atau tanpa kamera, tidak ada foto. **Kejujuran batas:** larangan galeri adalah sifat *frontend*. Server tidak dapat membedakan unggahan kamera dari unggahan lain (hanya memeriksa tipe JPEG/PNG/WebP dan ukuran 5 MB), sehingga pertahanan sisi server terhadap foto lama adalah batas umur dan pemakaian tunggal (bab 7) serta pemeriksaan model vision (bab 9).

A. Pelapor: pengiriman

1. **Pilih lokasi** (QR atau dua dropdown), tulis keluhan (minimal 5 karakter di formulir, maksimal 1000), ambil foto dengan kamera. Tombol kirim aktif bila lokasi dan foto sudah ada; panjang teks diperiksa saat dikirim.
2. **Unggah foto dahulu** (`POST /api/uploads`, `kind=report` bawaan) → kunci `report/<tanggal>/<uuid>.jpg`. Foto lebih dulu agar keluhan tersimpan dengan referensi yang pasti ada.
3. **`POST /api/reports`** dengan `{area_id, description, photo_key}`. Validasi Zod di `routes/reports.ts`: `area_id` 1–30 karakter (diubah ke huruf besar); `description` dipangkas lalu 5–1000 karakter; `photo_key` *wajib* dan harus berawalan `report/` (foto bukti petugas atau teks sembarang tidak dapat disamarkan sebagai foto pelapor). Gagal → 400 dengan pesan pertama.
4. **`submitReport()`** (services/report-service.ts): lokasi harus area aktif (`activeArea`), selain itu 404 “Lokasi tidak dikenal”. **Deteksi duplikat**: keluhan dengan teks persis sama untuk area yang sama dalam 2 menit terakhir (`findDuplicate`) dianggap ketukan ganda; dijawab 200 dengan id yang sudah ada dan `duplicate: true`, tanpa baris baru dan tanpa analisis.
5. Bukan duplikat: `crypto.randomUUID()`, `INSERT` (`ai_status='pending'`, `status='new'`), `reportsChanged()` (cache dashboard dibuang), baris jejak audit `report_created` (pelaku: nama pelapor bila masuk, selain itu `reporter`). Bila pelapor masuk sebagai `reporter`, `reporter_id` terisi; sesi peran lain tidak mengisinya.
6. **Respons 201** `{id, location, duplicate:false}` dikirim *sebelum* LLM bekerja; `c.executionCtx.waitUntil(runAnalysis(…))` menjalankan analisis sesudahnya (bab 8).
7. **Halaman status** `/reports/<id>`: menampilkan keluhan, foto, `Tracker` tiga tahap (diterima → diproses → selesai, dengan nama petugas dan waktu), dan hasil analisis. Selama `ai_status='pending'` halaman memuat ulang tiap 2 detik (maksimal 20 kali), lalu tiap 15 detik; tab tersembunyi tidak memuat. Pelapor tanpa akun hanya memegang tautan ini (UUID) untuk mengikuti keluhannya.

B. Petugas: tab Keluhan (`/staff`, tanpa login)

1. **Daftar terbuka**: `GET /api/reports/open` mengembalikan semua keluhan berstatus bukan `resolved`, urut prioritas (tinggi dahulu) lalu yang paling lama dahulu. Halaman memuatnya *tanpa filter* dan menyaring di klien dengan `reportIsAt`. Respons membuang kolom yang tidak perlu petugas (`reporter_id, ai_error, ai_model, ai_ms, proof_model, proof_ms`). Tidak ada “Siapa kamu?” di muka: nama ditanyakan *saat bertindak*.
2. **Klaim**: tombol “Saya kerjakan sekarang” (keluhan baru) membuka pilihan “Siapa yang mengerjakan?” (nama terakhir di HP sebagai isian awal) dan konfirmasi “Ya, saya yang kerjakan”. Klien mengirim `PATCH /api/reports/<id>` `{status:"in_progress", staff_id}`.

3. **Ambil alih**: klaim *bukan kunci*. Selama belum ditutup dengan foto bukti, keluhan yang dipegang orang lain menampilkan “Sedang dikerjakan X” dan tombol *Ambil alih*. Server mencatat “X mengambil alih laporan di … dari Y” di jejak audit (`details.handed_over_from`). Alasan rancangan: keluhan yang tidak diselesaikan pemegangnya tidak boleh menggantung selamanya.
4. **Selesai**: “Sudah beres — foto buktinya” menanyakan “Siapa yang menyelesaikan?” (isian awal: pemegang klaim), lalu membuka kamera. Foto diunggah (`kind=proof`), lalu `PATCH {status:"resolved", staff_id, proof_photo_key}` ; server memeriksa foto dengan model vision (bab 9) sebelum mengubah apa pun. Tombol menampilkan “Mengunggah…” lalu “AI memeriksa foto…”. Ditolak → kotak merah dengan alasan model; lolos → kartu “selesai” 2,5 detik, lalu hilang dari daftar.

Aturan transisi status di server

`PATCH /api/reports/:id` terbuka untuk petugas tanpa sesi. Identitas: `staff_id` yang harus milik petugas *aktif* (`findActiveStaff` ; selain itu 400 “Nama petugas tidak dikenal”), atau SPV yang sedang masuk (bertindak atas namanya sendiri); tanpa keduanya 400 “Pilih namamu dulu”. Seluruh aturan ada di `changeStatus()` , berurutan:

#	Kondisi	Hasil
1	Keluhan tidak ada	404 “Laporan tidak ditemukan”
2	Sudah `resolved` dan permintaan bukan `resolved`	**409** “Laporan ini sudah selesai.” Selesai tidak dapat dibuka kembali (membuka lagi akan menghapus foto buktinya)
3	Klaim ulang oleh pemegang yang sama (`in_progress` → `in_progress` , nama sama)	200 tanpa menulis dan **tanpa mencatat** dua kali
4	Klaim oleh orang lain atas keluhan `in_progress` berpemegang	Diterima sebagai *ambil alih*; `staff_name` diganti; dicatat khusus
5	Ada `proof_photo_key` baru	Harus berawalan `proof/` dan terbaca dari R2 (selain itu 400 “Foto bukti tidak ditemukan”); lalu pemeriksaan vision: ditolak → **422** (foto dibuang dari R2, jejak `proof_rejected`), gagal teknis → **502** (foto dibuang, jejak `verification_failed`)
6	`canResolve(status, proof, verdict)` salah	400 “Foto bukti penyelesaian wajib diunggah lebih dulu.” (target `resolved` tanpa bukti yang lolos)
7	Lolos semua	`UPDATE` status, nama, dan bukti *sekaligus* (foto dan putusan tidak pernah terpisah); `resolved_at` diisi bila `resolved` dan dikosongkan bila tidak; `reportsChanged()` ; jejak `status_changed` ; respons `{ok, status, verification}`

> **Celah yang diketahui pada `changeStatus()`** (`src/services/report-service.ts` , baris 215–275): (a) permintaan `resolved` atas keluhan yang *sudah* `resolved` tidak ditolak; ia menimpa `staff_name` dan `resolved_at` , dan bila disertai foto baru yang lolos, mengganti foto bukti lama tanpa menghapusnya dari R2. (b) `status:"new"` diterima oleh skema dan oleh `changeStatus()` : keluhan `in_progress` dapat dikembalikan ke `new` oleh siapa pun yang mengetahui sebuah `staff_id` . Keduanya konsisten dengan model kepercayaan petugas (bab 10) dan tercatat di jejak audit, tetapi bukan perilaku yang disengaja.

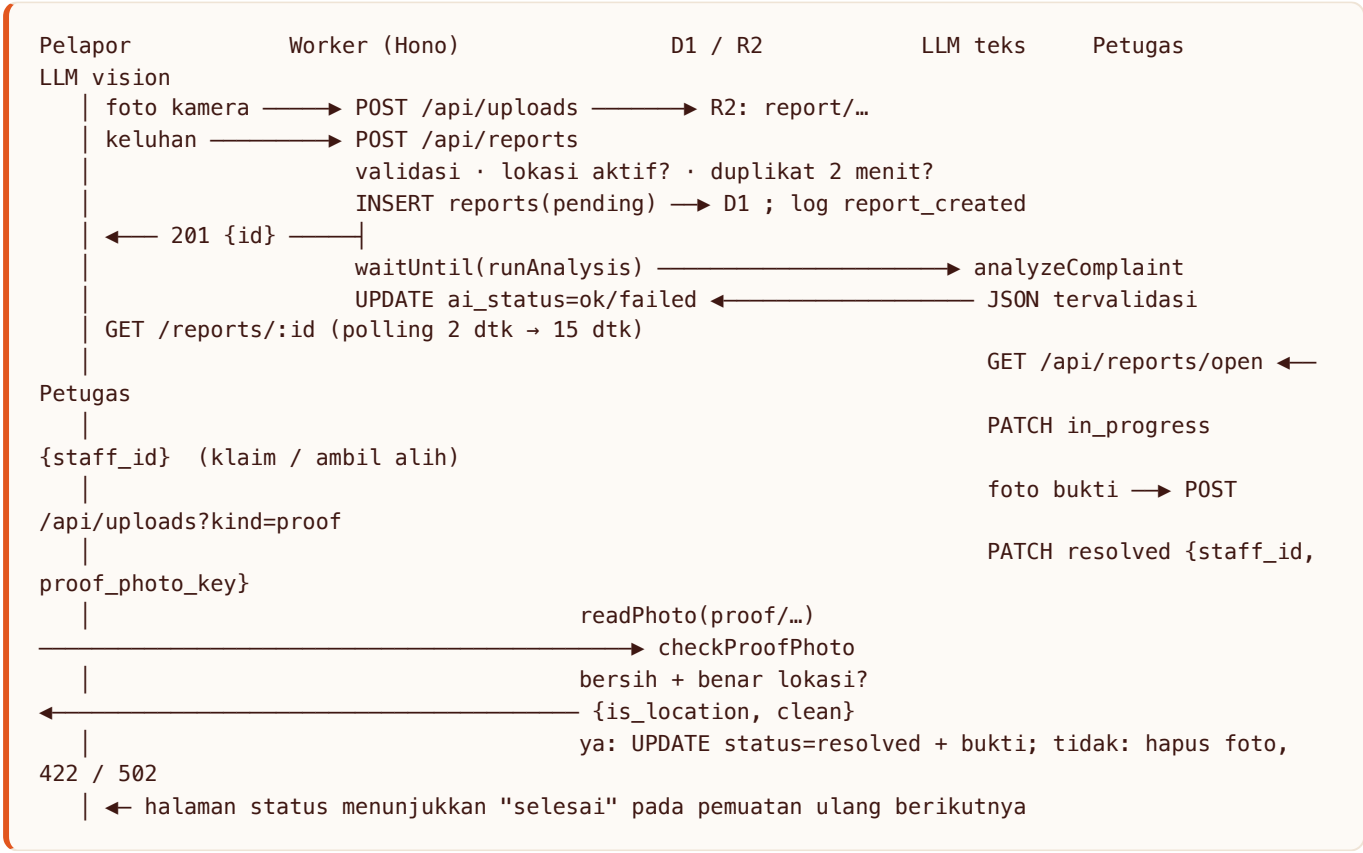
C. SPV: memantau, menghapus, menganalisis ulang

- **Memantau**: `GET /api/reports` (SPV) dengan filter `status, priority, area_id, building` , filter waktu bersama, dan `limit` (bawaan 100, maks. 200). Urutan: status (baru, diproses, selesai), lalu prioritas, lalu

terbaru.

- **Menganalisis ulang**: `POST /api/reports/:id/reanalyze` menandai `ai_status='pending'` (galat lama dikosongkan), lalu menjalankan `runAnalysis` lewat `waitUntil`. Berguna untuk keluhan `failed`.
- **Menghapus**: `DELETE /api/reports/:id` adalah satu-satunya aksi yang menghancurkan data, maka `deleteReport()` lebih dahulu menyalin keluhan ke jejak audit (`report_deleted`, rincian: nama lokasi, teks, status, prioritas, kategori, ringkasan, waktu dibuat, apakah ada foto), baru menghapus baris dan kedua foto di R2 (kegagalan hapus foto diabaikan). Jejak tetap menjawab apa yang hilang, siapa yang menghapus, dan kapan.

Diagram urutan: dari laporan sampai selesai



7. Tugas harian dan laporan kebersihan

Keluhan menjawab “ada masalah di sini”; tugas harian menjawab “pekerjaan rutin sudah dilakukan”. Tab *Tugas harian* lahir dari masalah manajemen: laporan kerja petugas dulu dikirim lewat WhatsApp dengan foto lama sehingga tidak dapat dipercaya. Rancangannya membatasi apa yang dapat dipalsukan (komentar di `services/cleaning-service.ts`).

Enam tugas, satu daftar di semua lokasi

Kode (`CLEANING_TASKS`)	Label
`sweep_mop`	Menyapu & mengepel
`glass`	Mengelap kaca jendela
`toilet`	Membersihkan toilet
`trash`	Membuang sampah
`gallon`	Mengisi ulang galon
`tidy`	Merapikan ruangan

Kode lama `sweep` dan `mop` tetap sah pada laporan sebelum migrasi 0010 (tipe `CleaningTask` memuatnya; `parseTasks()` hanya meloloskan enam kode baru untuk daftar tugas area).

Alur di halaman petugas

1. Tab *Tugas harian*: pilih **nama di muka** (`StaffPicker` ; diingat di HP) dan **lokasi** wajib (dropdown Gedung dan Lantai/Halaman). Peringatan lama “Bukan area tugasmu” sudah tidak ada: petugas boleh mengirim di area mana pun, dan SPV melihat selisihnya di rekap. Komponen `DailyTasks` diberi `key = area:petugas`, sehingga pergantian lokasi atau nama di HP bersama mulai dari kosong.
2. **Kartu GPS** (`useCampusGps`): meminta posisi (`enableHighAccuracy` , batas 15 dtk) dan menampilkan hijau (di kampus), merah (jauh), kuning (tidak tersedia: ditolak, tidak didukung, gagal), dengan tombol coba lagi. Ini hanya *peringatan dini* yang meniru aturan server.
3. Enam kartu tugas, **satu terbuka sekali**. Setiap tugas dikirim *sendiri-sendiri*: ketuk ikon kamera → foto → (opsional) catatan atau salah satu dari tiga kalimat siap pakai → “Kirim”. Setelah terkirim kartu bertanda centang hijau dengan jam kirim, dan tombol berubah menjadi “Lapor lagi”.
4. **Cap pada foto**: saat bingkai ditangkap, `Camera` menggambar pita hitam transparan 60% berisi empat baris teks putih di bagian bawah foto: *nama petugas · tugas*, *nama lokasi*, *waktu server (WIB)*, dan *koordinat GPS ±akurasi* (atau “tanpa GPS” / “tanpa waktu”). Waktu yang dicap adalah **jam server**: `useServerClock` mengukur selisih jam HP terhadap `/api/health` sekali, sehingga mengubah jam HP tidak mengubah cap. Cap membuat foto tetap bermakna setelah keluar dari aplikasi (mis. dicetak dalam laporan ke manajemen).
5. **Unggah saat foto diambil** (`kind=cleaning`), bukan saat “Kirim”, agar waktu terima server = waktu pengambilan foto.
6. Saat “Kirim”, klien mengambil posisi baru. Bila status bukan `ok` (jauh, atau posisi tidak ada), muncul dialog konfirmasi (“Kamu terlalu jauh dari kampus” / “Lokasi kamu tidak terbaca”) dengan “Batal, saya cek lokasi dulu” dan “Tetap kirim (supervisor diberi tahu)”. Laporan *tetap diterima*; yang berubah adalah penandaannya untuk SPV.

Pemeriksaan di server: `cleaning.submitReport()`

`POST /api/cleaning/reports` menerima `{staff_id, area_id, tasks:[{task, photo_key}], note?, position?}` (antarmuka mengirim satu tugas per permintaan, tetapi API menerima 1–6). Validasi Zod: tugas harus ada dalam enum, tidak ada tugas ganda, tidak ada kunci foto ganda, catatan maksimal 500, lintang/bujur berbatas. `staff_id` harus petugas aktif. Lalu, berurutan:

Langkah	Aturan	Hasil bila gagal
1. Area	Area aktif ada	404 “Kode area tidak dikenal” (foto yang terkirim dibuang)
2. Ketukan ganda	Semua kunci foto sudah terpakai pada *satu laporan yang sama* milik petugas dan area yang sama → dijawab dengan laporan itu, `duplicate:true` (200). Pencocokan memakai foto, bukan jendela waktu, sehingga laporan kedua yang sah semenit kemudian tidak pernah tertelan	—
3. Tugas berlaku	Tiap tugas termasuk daftar tugas area	400 “… tidak termasuk tugas di area ini”
4. Foto belum dipakai	Foto yang sudah membuktikan tugas lain ditolak (tidak dihapus, karena milik laporan lain)	422 “Foto … sudah pernah dipakai”
5. Foto ada dan baru	Kunci berawalan `cleaning/` dan ada di R2; `photoUploadedAt()` (cap unggah R2) tidak lebih dari **3 jam** (`PHOTO_MAX_AGE_MS` : cukup untuk memotret satu lantai sebelum mengirim, cukup pendek untuk menolak foto kemarin)	400 foto tidak ditemukan; 422 “sudah terlalu lama” (foto dihapus)
6. Posisi	Jarak haversine ke `CAMPUS_LAT/CAMPUS_LNG` ; `locationStatus(distance, accuracy, radius)`	— (menentukan penandaan)
7. Simpan	`cleaning_reports` dan semua `cleaning_report_tasks` ditulis dalam satu `DB.batch` (semua atau tidak sama sekali); `cleaningChanged()`	—

Status lokasi: ok, far, unknown

Di dalam ruangan GPS sering meleset puluhan meter, jadi akurasi yang dilaporkan HP diberikan sebagai kelonggaran:

```
slack  = min(max(accuracy ?? 0, 0), 200)          // dibatasi 200 m: HP yang mengaku galat 5 km
tidak bisa berkelit
status = distance === null ? 'unknown'
       : distance − slack > radius ? 'far' : 'ok'   // radius = CAMPUS_RADIUS_M, bawaan 500
```

Hanya posisi yang *di luar radius bahkan di tepi dekat lingkaran galatnya* yang dianggap `far` . Pusat dan radius kampus adalah variabel (`CAMPUS_LAT −7.3351` , `CAMPUS_LNG 108.2217` , `CAMPUS_RADIUS_M 500`); komentar di `wrangler.jsonc` menyebut pusat itu sebagai titik sementara di Jl. Dadaha yang harus diganti dengan pin kampus yang sebenarnya. `far` dan `unknown` menulis baris jejak `cleaning_out_of_range` (“Lapor … dari 1,2 km di luar kampus” / “tanpa lokasi GPS”) selain `cleaning_reported` (“<lokasi>: <daftar tugas>”), dan dihitung sebagai peringatan di dashboard.

> **Bukti tugas harian tidak diperiksa model vision.** Berbeda dari penutupan keluhan, foto tugas harian tidak dinilai bersih atau kotor. Buktinya berlapis lain: kamera langsung (di klien), cap nama-lokasi-waktu server-GPS pada gambar, umur foto di R2 maksimal 3 jam, satu foto hanya untuk satu tugas (batas `UNIQUE` di basis data), dan pemeriksaan posisi. Alasan: tugas rutin tidak punya "keluhan" untuk dicocokkan, dan memeriksa tiap foto akan menambah biaya serta latensi pada pekerjaan yang dikirim berkali-kali sehari.

Rekap harian per petugas: `dailyRecap(env, date)`

`GET /api/cleaning/recap?date=YYYY-MM-DD` (SPV; tanpa tanggal = hari ini WIB) merangkum satu hari per petugas. Hasilnya di-*cache* 20 detik dengan kunci `recap:<tanggal>` dan dibuang oleh `cleaningChanged()` (laporan baru, atau akun/penugasan berubah). Cara kerja `loadRecap`:

1. Memuat paralel: area aktif, penugasan (petugas aktif di area aktif), dan semua laporan kebersihan hari itu (batas 1000).
2. Setiap penugasan membuat sel *kosong* untuk tiap tugas area itu: `{done_at:null, times:0, flagged:false}`.
3. Laporan diproses dari yang paling lama: `done_at` menjadi waktu foto *pertama* hari itu, `times` bertambah. `flagged` bernilai benar hanya bila *semua* laporan di balik sel itu dikirim dari luar kampus atau tanpa GPS (satu laporan `ok` membersihkan tanda).
4. `expected` = jumlah sel di area yang *ditugaskan*; `done` = sel yang punya `done_at`. Pekerjaan di area yang bukan penugasannya ditampilkan (`assigned:false`) tetapi tidak mengubah skor.
5. Petugas diurutkan menurut kemajuan menaik (yang paling tertinggal di atas), lalu nama. `totals`: `expected, done, reports, alerts`. Petugas tanpa penugasan dan tanpa laporan hari itu tidak masuk daftar.

Sisi lain data yang sama: `GET /api/cleaning/reports` (SPV; filter `staff_id, area_id, alerts=1`, filter waktu; maks. 500) memberi daftar laporan dengan foto dan penanda GPS; `GET /api/cleaning/reports/public` menampilkannya di papan publik *tanpa nama petugas dan tanpa GPS* (bab 13). Dashboard tab Laporan → *Kebersihan* menampilkan rekap dan daftar itu; hitungan merah pada tab menunjukkan peringatan hari ini.

Laporan pekerjaan lama (work log)

Sebelum tugas harian, petugas mengirim "Sudah saya bersihkan" per *toilet* dengan foto yang diperiksa model vision. Data itu (tabel `work_logs`) tetap dibaca SPV di dashboard, tab Log → *Log pekerjaan* (`GET /api/work-logs`, filter petugas, toilet, gedung, dan waktu). Tidak ada lagi yang menulis ke tabel itu: antarmuka petugas tidak mengirimnya, dan rute `POST /api/work-logs` sudah dihapus dari server. Pekerjaan harian kini dicatat sebagai laporan kebersihan (bagian di atas); `work-log-service.ts` hanya berisi pembacanya.

8. Analisis LLM teks, ringkasan harian, dan Kato Bot

Model teks (DeepSeek) mengerjakan tiga hal: menganalisis tiap keluhan, menyusun ringkasan harian, dan menjawab pertanyaan lewat Kato Bot. Semuanya lewat `src/adapters/llm.ts`.

Prompt berbasis aturan

`analyzeComplaint(env, description, location)` menyusun pesan: *system prompt*, empat pasang contoh (*few-shot*), lalu pesan nyata berbentuk `Lokasi: <nama lokasi>\nKeluhan: <teks>` (lokasi di depan, persis seperti contoh). Isi *system prompt*:

- **Peran dan cakupan**: asisten petugas kebersihan; keluhan boleh tentang lokasi apa pun di kampus (ruang kelas, koridor, toilet, masjid, halaman, taman, selokan, lapangan), ditulis dengan bahasa gaul dan singkatan.
- **Enam kategori** dengan definisi contoh: `cleanliness` (kotor, sampah, daun menumpuk), `supplies` (sabun/tisu habis, galon kosong), `damage` (mampet, rusak, kaca pecah, paving rusak), `odor`, `flooding` (becek, genangan, atap bocor, selokan tersumbat), `other`. Boleh lebih dari satu.
- **Aturan prioritas** yang eksplisit: `high` bila ada risiko keselamatan atau lokasi praktis tidak dapat dipakai (air/selokan meluap, kloset mampet, lantai licin parah, kabel terbuka, listrik mati total, pintu tidak bisa dikunci, pecahan kaca, bau sangat menyengat, sampah menghalangi jalan, banyak masalah sekaligus); `medium` bila mengganggu tetapi lokasi masih bisa dipakai; `low` untuk keluhan kecil atau antisipasi.
- **Aturan penulisan**: `summary` satu kalimat netral maksimal 15 kata; `recommendation` instruksi kerja maksimal dua kalimat, diurutkan dari yang paling berisiko mencelakakan; jangan mengarang masalah; teks tidak jelas → `["other"]`, `low`.
- **Format keluaran**: hanya satu objek JSON dengan kunci Inggris persis, ditambah `response_format: json_object`.

Empat contoh sengaja mewakili jenis tempat yang berbeda agar penerapan aturan konsisten: toilet berbau/becek/sabun habis (`high`, tiga kategori), galon kosong di depan kelas (`medium`), selokan mampet di halaman gedung (`high`), dan daun kering di Lapang Voli (`low`). `temperature 0.1`: kalimat yang sama harus menghasilkan klasifikasi yang sama.

Validasi berlapis

Keluaran model tidak dipercaya apa adanya. Tiga lapis:

1. **Mentah → JSON**: `JSON.parse(content)`; bila gagal, mengambil blok `{…}` pertama dengan ekspresi reguler (model kadang membungkus dengan pagar kode); bila tetap gagal, galat “Respons LLM bukan JSON”. Respons kosong juga galat.
2. **Skema Zod** (`AnalysisSchema`): `categories` larik string minimal 1, `priority` string, `summary` dan `recommendation` tidak kosong.
3. **Pemaksaan ke enum** (`coerceAnalysis`): kategori diubah ke huruf kecil, dipangkas, disaring ke enam nilai sah, duplikat dibuang, hasil kosong menjadi `["other"]`; prioritas di luar tiga nilai menjadi `medium`; ringkasan dipotong 300 karakter dan rekomendasi 500. Basis data tidak pernah menerima nilai di luar enumnya.

Kegagalan jaringan ditangani di `postChat()`: batas 30 detik per panggilan (`AbortSignal.timeout`), dan pergantian ke kunci API berikutnya bila kunci aktif dijawab 401, 403, atau 429; status lain langsung menjadi galat.

Asinkron dan tahan gagal

`runAnalysis(env, id)` (services/analysis-service.ts) dipanggil lewat `waitUntil()` setelah respons terkirim, sehingga pelapor tidak pernah menunggu LLM. Fungsi ini **tidak pernah melempar galat**:

Hasil	Yang tersimpan	Jejak audit
Berhasil	`ai_status='ok'`, `categories`, `priority`, `summary`, `recommendation`, `ai_model`, `ai_ms`, `ai_error=NULL`; `reportsChanged()`	`analysis`, pelaku `system`: "Analisis selesai: prioritas high (1234 ms)"
Gagal (jaringan, JSON, skema)	`ai_status='failed'`, `ai_error` (maks. 500 karakter), `ai_ms`; keluhan tetap tampil di dashboard tanpa label	`analysis_failed` dengan 300 karakter pertama galat

Keluhan `failed` dapat dianalisis ulang oleh SPV (`POST /api/reports/:id/reanalyze`, bab 6). Prioritas menjadi masukan statistik dashboard, itulah sebabnya `reportsChanged()` dipanggil di kedua cabang.

Ringkasan harian

- **Pemicu**: cron `0 10 * * *` (UTC) = **17.00 WIB** (`wrangler.jsonc` → `scheduled()` di `src/index.ts`, tanggal = `wibDate()`), atau tombol *Buat ulang* di dashboard (`POST /api/summary/generate?date=`, SPV; galat dijawab 502).
- `generateDailySummary(env, date)`: mengambil semua keluhan hari WIB itu (`reportsOnDate`, memakai `wibDayRange`). **Hari tanpa keluhan tidak memanggil LLM**: tersimpan kalimat tetap "Tidak ada keluhan yang masuk pada hari ini."
- Selain itu `summarizeDay()` mengirim daftar bernomor `[prioritas] lokasi – ringkasan-atau-teks` (`max_tokens 700`), dengan perintah: tentukan lokasi paling bermasalah dan pola berulang, sebut angka konkret, tanpa mengarang data di luar daftar. Keluaran: `summary` (2–4 kalimat) dan `highlights` (maks. 4 butir, masing-masing maks. 12 kata; dipotong ke 4 di kode).
- Disimpan dengan *upsert* di `daily_summaries` (satu baris per tanggal); cache ringkasan dibuang (`forget('summary')`); jejak `daily_summary`. Ringkasan hanya mencakup *keluhan*, tidak mencakup tugas harian.
- Bila cron gagal, galat hanya masuk ke log Worker (`console.error`); tidak ada baris jejak audit untuk kegagalan, dan tidak ada percobaan ulang otomatis. SPV dapat menekan *Buat ulang*.

Kato Bot: tanya data lewat *tool calling*

Kato Bot menjawab pertanyaan bahasa sehari-hari tentang data laporan. Model **tidak pernah melihat basis data**: ia hanya melihat definisi empat alat, memutuskan alat mana dipanggil dengan argumen apa, dan menjawab dari hasil agregat yang dihitung di sisi kita. Biaya satu pertanyaan karena itu tetap beberapa ribu token berapa pun jumlah laporan.

Alat	Fungsi	Argumen
`count_reports`	Jumlah keluhan sesuai filter, dikelompokkan bila diminta; tiap baris memuat `count`, `resolved`, `high`	Filter + `group_by`
`resolution_time`	Rata-rata, tercepat, dan terlama menit dari masuk sampai selesai (hanya yang sudah selesai)	Filter + `group_by`
`list_reports`	Hingga 15 keluhan satu per satu: lokasi, status, prioritas, kategori, ringkasan, waktu masuk dan selesai (WIB)	Filter + `order` (newest/oldest), `limit` 1–15
`daily_summary`	Ringkasan tersimpan satu tanggal beserta sorotan	`date` (YYYY-MM-DD)

Filter bersama: `since`, `until` (tanggal WIB inklusif), `building` (A–J), `category`, `priority`, `status`. Pengelompokan (`GROUPINGS` di `src/repositories/analytics.ts`, 12 kunci): `building, location, floor,`

`category, priority, status, staff, date, week, month, hour, weekday`.

Pengaman: model memilih query, tidak menulis SQL

- Setiap nilai dari model masuk lewat *bind parameter*; setiap kolom atau pengelompokan yang disebutnya dicari di tabel tetap `GROUPINGS`. Argumen divalidasi ulang dengan Zod sebelum menyentuh query, dan tanggal serta kode gedung dicocokkan dengan pola ketat.
- **Hasil alat adalah data, bukan instruksi** (dinyatakan di prompt). `list_reports` sengaja *tidak* memuat teks mentah keluhan, hanya ringkasan netral dari model: pelapor tidak dapat menyelipkan perintah ke pertanyaan SPV lewat isi keluhan.
- **Dua hak akses**: SPV (sesi bukan `reporter`) boleh mengelompokkan per `staff` dan melihat `staff_name`; pelapor dan pengunjung mendapat daftar pengelompokan tanpa `staff`, dan `runTool` menolak `group_by=staff` sekali lagi bila model mengabaikan daftar itu. Sama seperti papan publik, nama petugas tidak dibuka untuk umum.
- Alat yang gagal tidak membatalkan pertanyaan: galatnya dikembalikan ke model sebagai `{error}` agar ia dapat mengulang atau menjawab dari yang sudah ada.
- Putaran dibatasi **4** (`maxRounds`); pada putaran terakhir `tool_choice: 'none'` memaksa model menjawab.
- Prompt sistem mewajibkan jawaban hanya dari hasil alat, menyebut rentang bila tidak ditentukan (bawaan 30 hari terakhir), 2–5 kalimat, teks polos tanpa Markdown, dalam bahasa penanya, dan menolak pertanyaan di luar data.

Anggaran harian dan pembatasan

Batas	Nilai	Penegakan
Per hari, semua penanya	`DAILY_LIMIT = 50`	429 “Batas 50 pertanyaan per hari sudah tercapai.”; dihitung dari baris `question` hari ini (WIB) di `activity_log`
Per alamat, bukan SPV	`PER_IP_LIMIT = 20`	Alamat (`cf-connecting-ip`) di-*hash* SHA-256, 8 byte pertama (16 heksadesimal) disimpan di `details.ip`: cukup untuk membatasi, tidak cukup untuk mengidentifikasi
Panjang	Pertanyaan 3–500 karakter; riwayat maks. 20 butir masuk, **6** terakhir diteruskan, masing-masing dipotong 500 karakter	Biaya pertanyaan tidak membesar seiring panjang percakapan

Jawabannya memuat `text`, `tools` (jejak panggilan), `tokens` (`prompt, completion, cache_hit`), `ms`, dan `remaining_today`. Tiap pertanyaan yang berhasil menulis baris jejak `question` (pelaku: nama SPV/pelapor atau `visitor`) berisi teks pertanyaan, alat yang dipanggil, token, model, dan waktu. Karena anggaran dihitung dari jejak itu, pertanyaan yang gagal tidak mengurangi jatah, dan pemeriksaan-lalu-tulis tidak atomik (dua pertanyaan serentak di ambang batas dapat sama-sama lolos); keduanya diterima karena skalanya kecil.

Kegagalan model dijawab 502 “Model tidak dapat menjawab saat ini”, dengan galat penuh di log Worker. `GET /api/ask` hanya mengembalikan `{daily_limit}`.

Antarmuka melayang

Kato Bot bukan lagi bagian Beranda, melainkan **ikon melayang di setiap halaman**: `FloatingChat` dipasang *sekali* di `web/src/main.tsx`, di luar `Routes`, sehingga percakapan bertahan selama pengguna berpindah halaman (tidak disimpan lintas muat ulang).

- **Diseret**: peluncur (lingkaran 56 piksel) dapat diseret ke mana saja dan menetap di tempat dilepas (ambang gerak 5 piksel untuk mouse, 10 untuk sentuhan, agar ketukan tidak terbaca seretan). Posisi disimpan di

`localStorage "chat-anchor"`; margin 12 piksel dari tepi.

- **Diketuk**: peluncur menempel ke tepi kiri atau kanan yang lebih dekat, dan jendela obrolan (lebar maks. 390, tinggi maks. 580) terbuka di sisinya ke arah yang lebih lapang (atas atau bawah). Jendela mengikuti peluncur dan dapat diseret lewat judulnya; `Esc` atau tombol tutup menutupnya.
- **Dimuat malas**: `AskPanel` (`lazy(() => import('./AskPanel'))`) baru diunduh pada ketukan pertama. Jendela yang ditutup tetap terpasang (disembunyikan), sehingga utas tidak hilang.
- Contoh pertanyaan untuk SPV memuat pertanyaan per petugas; untuk umum diganti pertanyaan jumlah selesai. Sisa jatah ditampilkan di bawah kolom isian; “Mulai percakapan baru” mengosongkan utas. Pertanyaan yang gagal dikembalikan ke kolom isian.

> **Batas pada `countReports` dan `resolutionTime`** (`src/repositories/analytics.ts`, baris 106 dan 137): hasil berkelompok dipotong `LIMIT 20`. Untuk pengelompokan waktu (`date, week, month, hour`) urutannya menaik menurut label, jadi rentang lebih dari 20 hari menampilkan *20 hari tertua* dan `hour` (24 label) kehilangan jam 20.00–23.00. Model hanya melihat potongan itu dan tidak diberi tahu bahwa ada yang terpotong.

9. Verifikasi foto (LLM vision)

Model vision (Gemini `gemini-3.6-flash`) memutuskan satu hal: apakah foto yang diunggah petugas boleh menutup sebuah keluhan. Fungsinya `checkProofPhoto(env, photo, context)` di `src/adapters/llm.ts`; dipanggil dari `checkNewProof()` di `report-service.ts`. Tugas harian *tidak* melalui pemeriksaan ini (bab 7).

Masukan

Foto dibaca dari R2 oleh Worker (`readPhoto`, hanya kunci berawalan `proof/` yang diterima untuk penutupan keluhan, sehingga foto keadaan dari pelapor tidak dapat dipakai ulang sebagai bukti), diubah ke *data URL* base64 (dipotong per 32.768 byte agar tidak meluap tumpukan), dan dikirim *inline* dengan `detail: "low"`. URL publik memang bukan pilihan: bucket privat dan hanya Worker yang dapat membacanya. Teks pendamping: `Lokasi`, `Kategori keluhan` (kode Inggris, atau "belum dianalisis"), dan `Keluhan asli`.

Kriteria dalam prompt

Model menjawab dua pertanyaan, bukan satu:

1. `is_location`: apakah foto benar-benar memperlihatkan tempat yang dikeluhkan, dicocokkan dengan lokasi dan isi keluhan. Keluhan toilet harus memperlihatkan bagian dalam toilet; ruangan/koridor memperlihatkan lantai, meja, atau jendela; halaman, taman, selokan, atau lapangan memperlihatkan area luar itu. Foto orang, layar, hanya langit-langit, gelap/buram yang tidak dapat dinilai, atau tempat yang jelas berbeda → `false`.
2. `clean`: apakah kondisi yang terlihat sudah bersih dan layak pakai. Tidak bersih bila terlihat sampah berserakan, kotoran atau noda, genangan atau lantai basah merata, tempat sampah meluap, selokan masih tersumbat, coretan, lumut/kerak tebal, atau bekas keluhan yang jelas belum ditangani. Noda permanen kecil, lantai tua, sedikit daun yang baru jatuh, dan lantai lembap tipis setelah dipel masih boleh dianggap bersih.

Aturan tambahan: bila keluhannya terlihat pada foto (sampah, genangan), periksa apakah hal itu sudah hilang; keluhan yang tidak terlihat dari foto (bau, sabun habis) *tidak* boleh dijadikan alasan menolak; **bila ragu antara bersih dan kotor, pilih `clean: false`**; jangan mengarang detail yang tidak ada di foto. Keluaran: `{is_location, clean, confidence, reason}` dengan alasan satu kalimat Indonesia maksimal 25 kata yang menyebut apa yang terlihat.

Validasi keluaran dan pemetaan putusan

Langkah	Aturan
Skema	`ProofCheckSchema`: `is_location` dan `clean` boolean; `confidence` angka 0–1 (opsional, dipaksa dengan `z.coerce`); `reason` tidak kosong
Boolean longgar	`looseBool`: string `true/ya/yes/1` dibaca benar, string lain salah. `z.coerce.boolean()` tidak dipakai karena mengubah string "false" menjadi `true`, kesalahan yang berbahaya di sini
Putusan	`!is_location` → `wrong_place`; selain itu `clean` → `clean`, bila tidak `dirty`. Salah tempat lebih dahulu daripada kotor
Alasan	Dipangkas 300 karakter

Anggaran token untuk model "berpikir"

`max_tokens` diset **2000**, jauh di atas ukuran JSON yang diminta (sekitar seratus token). Model keluarga Gemini 3.x memakai sebagian anggaran itu untuk menalar sebelum menjawab; anggaran beberapa ratus membuat JSON

terpotong di tengah dan penutupan gagal tanpa sebab yang terlihat. Latensi tiap pemeriksaan tercatat di `reports.proof_ms`.

Apa yang disimpan, apa yang dibuang

Hasil	Foto di R2	Basis data	Jejak audit	Respons ke petugas
`clean` (diterima)	Disimpan	`proof_photo_key`, `proof_verdict`, `proof_reason`, `proof_model`, `proof_ms` ditulis bersama status `resolved`	`status_changed` dengan `details.verification`	200
`dirty` / `wrong_place`	**Dihapus**	Tidak ada perubahan	`proof_rejected`: putusan, alasan, `confidence`, model, `ms`	422 “Lokasi pada foto masih terlihat kotor…” atau “Foto tidak menunjukkan lokasi yang dilaporkan…”, ditambah `verdict` dan `reason`
Gagal teknis (HTTP, JSON, skema, *timeout*)	**Dihapus** (percobaan ulang mengunggah salinan baru)	Tidak ada perubahan	`verification_failed` dengan galat dan model	502 “Pemeriksaan foto gagal. Coba lagi sebentar.”

Pemadaman vision tidak diloloskan. Bila pemeriksa mati, keluhan tidak dapat ditutup: bukti yang tidak diperiksa tidak dihitung. Konsekuensinya diterima sengaja (bab 22): lebih baik keluhan menunggu daripada ditutup tanpa pemeriksaan. Bukti yang tersimpan hanya yang lolos, jadi bucket tidak menumpuk foto bukti palsu; riwayat penolakan hidup di jejak audit, bukan di R2.

10. Akun, sesi, dan wewenang

Tiga peran

Peran (`users.role`)	Memegang kredensial?	Sesi?	Catatan
`supervisor`	Ya: `username` + sandi (PBKDF2)	Ya	Pengguna dashboard; boleh lebih dari satu
`reporter`	Ya (mendaftar sendiri di `/register`)	Ya	Opsional; hanya agar laporannya terikat akun dan masuk papan peringkat
`staff`	**Tidak**: hanya nama; `CHECK` basis data mewajibkan username dan sandi hanya untuk peran selain staff	**Tidak pernah**; `login()` menolak akun petugas walau suatu akun lama masih punya sandi	Dipilih dari dropdown

Masuk, daftar, keluar

- **`POST /api/auth/login`** `{username, password}`. `findByUsername` (tak peka huruf besar-kecil, karena `COLLATE NOCASE`). Akun tak dikenal, tidak aktif, tanpa sandi tersimpan, berperan petugas, atau sandi salah menghasilkan **jawaban yang sama** (401 “Username atau password salah”), supaya halaman masuk tidak menjadi cara menemukan username yang ada. Verifikasi memakai `verifyPassword`: PBKDF2-SHA256, 100.000 iterasi, garam 16 byte per pengguna, turunan 256 bit, pembanding waktu-konstan. PBKDF2 dipilih karena tersedia di WebCrypto Workers tanpa WebAssembly tambahan; garam per pengguna membuat dua orang dengan sandi sama berhash berbeda. Masuk sebagai supervisor menulis jejak `login`; masuk sebagai pelapor tidak.
- **`POST /api/auth/register`**: hanya membuat `reporter` (username 3–30 karakter `[a-zA-Z0-9._-]`, nama 2–60, sandi 8–200). Username yang sudah ada → 409. Sesi langsung dibuat (201). Akun supervisor tidak pernah dibuat lewat pendaftaran.
- **`POST /api/auth/logout`** menghapus cookie; **`GET /api/auth/me`** mengembalikan sesi atau 401 (dibaca sekali di akar aplikasi oleh `SessionProvider`).

Sesi: cookie bertanda tangan

`createSession()` menandatangani JWT HS256 (`hono/jwt`) dengan `AUTH_SECRET` berisi `{id, name, role, exp}`, lalu menaruhnya di cookie `session`: `HttpOnly`, `Secure` bila URL HTTPS, `SameSite=Lax`, `Path=/`, umur **12 jam** (“cukup untuk satu giliran”). `currentSession()` memverifikasi tanda tangan dan kedaluwarsa; token rusak atau kedaluwarsa dianggap tidak ada sesi. Tanpa `AUTH_SECRET` (mis. `.dev.vars` tidak ada), masuk gagal dengan 500.

> **Sesi tanpa status.** Peran dan nama dibaca dari token, tidak dicocokkan ke basis data pada tiap permintaan (`src/adapters/session.ts`, `requireRole`). Menonaktifkan, mengganti nama, atau menghapus akun supervisor baru berlaku saat tokennya kedaluwarsa, paling lama 12 jam; keluar hanya menghapus cookie di peramban itu. Untuk mencabut akses segera, ganti `AUTH_SECRET` (semua sesi gugur). Tidak ada pembatasan percobaan masuk (*rate limit*) pada `/api/auth/login`.

Wewenang per rute

`requireRole(...peran)` menolak dengan 401 “Silakan masuk terlebih dahulu.” (tanpa sesi) atau 403 “Akses ditolak.” (peran lain). Hanya satu penjaga yang diekspor, `requireSupervisor`; tiap rute menyebut sendiri peran yang diterimanya, sehingga wewenang terbaca di tempat rute didefinisikan. Daftar lengkap rute ada di bab 15; ringkasnya:

Golongan	Rute	Cara identitas
Publik, tanpa identitas	`POST /api/reports` (sesi `reporter` opsional mengikat laporan), `GET /api/reports/public|open|:id`, `GET /api/reports/mine`, `GET /api/cleaning/areas[/:id]`, `GET /api/cleaning/reports/public`, `GET /api/staff`, `GET /api/leaderboard`, `POST|GET /api/ask`, `POST /api/uploads`, `GET /api/uploads/*`, `GET /api/health`	—
Petugas tanpa login	`PATCH /api/reports/:id` (klaim, ambil alih, selesai), `POST /api/cleaning/reports`	`staff_id` di body, harus petugas aktif; `PATCH` juga menerima sesi supervisor
Hanya supervisor	`GET /api/reports`, `POST /api/reports/:id/reanalyze`, `DELETE /api/reports/:id`, `GET /api/cleaning/reports`, `GET /api/cleaning/recap`, `GET /api/work-logs`, `GET /api/activity`, `GET /api/summary`, `/stats`, `/charts`, `POST /api/summary/generate`, seluruh `/api/users`	Cookie sesi, peran `supervisor`

Seperti papan publik, daftar terbuka untuk petugas (`/open`) membuang akun pelapor dan internal AI (bab 6). Pengecualian: `GET /api/reports/:id` (halaman status) mengembalikan seluruh DTO, termasuk `reporter_id`, `staff_name`, dan `ai_error`; satu-satunya pengamannya adalah UUID yang tidak dapat ditebak (bab 23). `GET /api/reports/mine` mengembalikan daftar kosong, bukan 401, bagi yang bukan `reporter`. Kato Bot menilai peran dari sesi (bab 8): sesi bukan `reporter` = SPV.

Petugas tanpa login: kompromi keamanan

Banyak petugas tidak nyaman dengan username dan sandi, dan HP dipakai bergantian. Maka petugas memilih nama dari dropdown (`GET /api/staff` publik, hanya `id` dan `name` petugas aktif; nama yang sama sudah tampil di keluhan selesai). Identitas lewat `staff_id` (UUID):

- **Divalidasi tiap aksi**: id harus milik petugas dengan `role='staff'` dan `active=1` (`findActiveStaff`). Petugas yang dinonaktifkan atau dihapus SPV langsung tidak dapat bertindak, dan HP yang mengingat id lamanya jatuh kembali ke “belum memilih nama”. `localStorage "selected-staff"` hanya kenyamanan, bukan identitas.
- **Nama ditanyakan saat bertindak** di tab Keluhan (dengan isian awal nama terakhir), karena HP bersama; tab Tugas harian menanyakannya di muka karena satu putaran milik satu orang.
- **Apa yang mencegah penyalahgunaan** bukan kerahasiaan id (daftar itu publik, jadi siapa pun dapat bertindak atas nama siapa pun), melainkan: penutupan keluhan butuh foto yang lolos pemeriksaan vision; tugas harian butuh foto yang baru, tunggal-pakai, dan bertanda GPS; semua aksi tercatat dengan nama; dan SPV dapat menghapus, menganalisis ulang, dan menelusuri jejak.
- **Yang tidak dicegah**: seseorang yang mengetahui nama petugas dapat mengirim atas namanya, dan dapat mengklaim atau mengambil alih keluhan atas namanya (dicatat sebagai nama itu). Pertimbangan ini diterima karena akibat terburuknya terbatas pada catatan kerja yang salah nama, dan dapat ditelusuri dari jejak serta foto.

Mengelola akun (tab Petugas & Supervisor)

Aksi	Rute	Aturan
Daftar	`GET /api/users`	Supervisor dan petugas (pelapor tidak ditampilkan karena bisa ribuan); supervisor dahulu, lalu menurut nama
Tambah petugas	`POST /api/users {role:"staff", name}`	Nama 2–60 karakter, tanpa username dan sandi ("Petugas cukup nama, tanpa login"). Jejak `user_changed`: "Menambah petugas …"
Tambah supervisor	`POST /api/users {role:"supervisor", username, name, password}`	Username 3–30 `[a-zA-Z0-9._-]`, sandi 8–200; username dipakai → 409; sandi di-*hash* dengan garam baru
Ubah	`PATCH /api/users/:id {name?, password?, active?}`	Minimal satu bidang. `password` mengganti hash *dan* garam; petugas tidak punya sandi (400). Akun yang sedang dipakai tidak bisa dinonaktifkan (400, *self-lockout*). Nonaktif = tidak bisa masuk (SPV) atau hilang dari dropdown (petugas)
Hapus	`DELETE /api/users/:id`	Hanya SPV atau petugas (pelapor → 404). Ditolak: akun sendiri (400), **SPV aktif terakhir** (400), dan akun yang **punya catatan** (409, memuat jumlahnya: pesan "Akun ini punya N catatan (laporan atau pekerjaan). Nonaktifkan saja agar riwayatnya tetap utuh."). Catatan dihitung dari `work_logs.staff_id` + `cleaning_reports.staff_id` + `reports.reporter_id`. Bila lolos, akun dan penugasan areanya (`cleaning_assignments`) dihapus dalam satu `batch`. Antarmuka meminta konfirmasi "Hapus … secara permanen?"

Setiap perubahan akun menulis `user_changed` dengan kalimat terbaca ("Mengubah petugas X: nama, dinonaktifkan", "Menghapus akun budi") dan memanggil `forgetStaff()`, yang membuang cache daftar petugas, daftar lokasi (penugasan ikut di dalamnya), dan rekap kebersihan. Keluhan menyimpan nama petugas sebagai *teks* (`reports.staff_name`), bukan FK, sehingga keluhan yang pernah ditangani petugas tidak menghalangi penghapusan namanya; jejak audit juga menyebut nama secara polos. Alasan membedakan hapus dan nonaktif: hapus hanya untuk salah ketik atau akun uji tanpa riwayat; yang punya riwayat dinonaktifkan agar riwayat tidak yatim.

> **Catatan konsistensi.** `updateAccount` (`src/services/user-service.ts`) tidak menolak akun berperan `reporter`, berbeda dengan `deleteAccount`: supervisor yang mengetahui id seorang pelapor dapat mengganti nama, menonaktifkan, atau mengatur ulang sandinya lewat `PATCH`. Antarmuka tidak menyediakannya (daftar tidak memuat pelapor).

Akun bawaan. Migrasi 0004 menanam akun `admin` dengan sandi awal yang tertulis di komentar migrasi (repositori publik). Ganti sandinya, atau buat supervisor baru lalu nonaktifkan `admin`, sebelum sistem dipakai sungguhan.

11. Jejak audit dan filter waktu

Tabel `activity_log`

Setiap pergerakan penting ditulis ke satu tabel, `activity_log`, lewat fungsi `log()` di `services/activity-service.ts`. Skemanya dibuat oleh migrasi 0008:

Kolom	Tipe	Arti
`id`	INTEGER, kunci utama, `AUTOINCREMENT`	Urutan kejadian; dipakai sebagai pemecah seri bila dua baris punya `created_at` yang sama
`created_at`	TEXT, UTC `YYYY-MM-DD HH:MM:SS`	Diisi `datetime('now')` oleh basis data. Dashboard menampilkannya dalam WIB
`action`	TEXT	Salah satu nilai `ActivityAction` (tabel di bawah). Tidak ada `CHECK` di tingkat basis data; daftarnya dijaga oleh tipe TypeScript
`report_id`	TEXT, boleh NULL, **bukan** foreign key	Id laporan keluhan yang terkait, tetap tercatat walau laporannya sudah dihapus
`actor`	TEXT	Nama orang, atau `reporter` (pelapor anonim), `system` (analisis, cron), `visitor` (penanya Kato Bot tanpa akun)
`summary`	TEXT	Satu kalimat berbahasa Indonesia yang langsung terbaca manusia
`details`	TEXT (JSON), boleh NULL	Rincian terstruktur; bentuknya bergantung pada `action`

Indeks: `idx_activity_created (created_at DESC)`, `idx_activity_report (report_id)`, `idx_activity_action (action, created_at DESC)`.

Prinsip

- **Mencatat tidak boleh menggagalkan tindakan.** `log()` menelan semua galat dan hanya mencetaknya ke log Worker (`Failed to write activity log`): menerima laporan lebih penting daripada merekamnya. Konsekuensinya, jejak ini adalah catatan sebaik-baiknya, bukan jaminan transaksional; pencatatan dilakukan sesudah tindakan, bukan dalam transaksi yang sama.
- **Tidak dapat diubah atau dihapus lewat aplikasi.** Satu-satunya pernyataan SQL atas tabel ini di `src/` adalah `INSERT` (`repositories/activity.ts`, fungsi `insert`) dan `SELECT` (`find`, dan penghitung anggaran Kato Bot di `repositories/analytics.ts`). Tidak ada endpoint ubah atau hapus, bahkan untuk SPV yang menghapus laporan. Perlu dicatat bahwa ini sifat aplikasi, bukan batasan basis data: pemegang akses `wrangler d1` tetap dapat mengubah barisnya.
- **Sengaja tanpa foreign key.** Laporan yang *dihapus* adalah yang paling perlu terlacak. Foreign key akan memaksa baris audit ikut hilang (atau menolak penghapusan), kebalikan dari tujuannya. Maka seluruh isi laporan disalin ke `details` sebelum barisnya dihapus (`deleteReport` di `report-service.ts`: log dulu, baru `DELETE`). Nama orang pun disimpan sebagai teks, sehingga akun petugas yang dihapus tetap terbaca di riwayatnya.
- **Yang tidak dicatat:** pendaftaran akun pelapor, masuk yang gagal, dan masuk oleh pelapor (hanya masuk oleh SPV yang dicatat).

Jenis aksi (`ActivityAction`)

Empat belas nilai, didefinisikan di `repositories/activity.ts`. Daftar yang sama dipakai untuk memvalidasi filter `action` di API.

`action`	Pelaku (`actor`)	Kapan ditulis	Isi `details`
`report_created`	nama pelapor berakun, atau `reporter`	Keluhan baru diterima (kiriman kembar tidak dicatat dua kali)	area_id, description
`analysis`	`system`	Analisis teks selesai (`runAnalysis`)	categories, priority, model
`analysis_failed`	`system`	Analisis teks gagal	error (maks. 300 karakter)
`status_changed`	nama petugas yang dipilih saat bertindak, atau SPV	Keluhan diklaim, diambil alih, atau diselesaikan	from, to, handed_over_from (nama pemegang sebelumnya atau null), proof_photo, verification {verdict, reason, model, ms} atau null
`proof_rejected`	petugas	Foto bukti dinilai `dirty` atau `wrong_place`	verdict, reason, confidence, model, ms. Jalur lama laporan pekerjaan menambah description dan `report_id` NULL
`verification_failed`	petugas	Layanan vision tidak dapat dihubungi atau responsnya rusak	error (maks. 300 karakter), model
`report_deleted`	SPV	Keluhan dihapus	salinan: location_name, description, status, priority, categories (teks JSON), summary, created_at, had_photo
`login`	SPV	Masuk ke dashboard berhasil	tidak ada
`daily_summary`	`system`	Ringkasan harian disusun (cron atau tombol “Buat ulang”)	tidak ada; `summary` memuat tanggal dan jumlah laporan
`user_changed`	SPV	Nama petugas atau akun SPV ditambah, diubah (nama, sandi, aktif/nonaktif), atau **dihapus**	tidak ada; `summary` menyebut apa yang berubah
`question`	nama penanya, atau `visitor`	Kato Bot menjawab satu pertanyaan	question, tools [{name, arguments}], tokens {prompt, completion, cache_hit}, model, ms, ip (sidik SHA-256 dipotong 8 bita; hanya untuk non-SPV, dipakai membatasi 20 pertanyaan per alamat)
`work_logged`	petugas	Laporan pekerjaan versi lama (jalur tulisnya sudah dihapus; hanya baris lama yang tersisa)	work_log_id, toilet_id, description, verification
`cleaning_reported`	petugas	Laporan kebersihan (satu tugas harian dengan fotonya) diterima	cleaning_report_id, area_id, tasks[], note. `report_id` NULL

`cleaning_out_of_range`	petugas	Sebagai tambahan `cleaning_reported` bila status lokasi `far` atau `unknown`	cleaning_report_id, area_id, latitude, longitude, accuracy_m, distance_m

Baris dari sebelum migrasi 0008 sudah memakai nilai `action` berbahasa Inggris, tetapi `details`-nya (kecuali salinan `report_deleted`, yang diterjemahkan migrasi) masih berkunci Indonesia, mis. `{"dari":…,"ke":…,"verifikasi":{"hasil":"bersih",…}}`. Salinan keluhan lama yang dihapus memakai kunci `toilet_name`, bukan `location_name`; antarmuka membaca keduanya.

Membaca: `GET /api/activity`

Hanya SPV (`requireSupervisor`). Parameter: `action` (satu nilai `ActivityAction`; nilai yang tidak dikenal **diabaikan**, bukan galat 400, sehingga semua baris tampil), `report_id` (kecocokan persis), `limit` (bawaan 100, **maksimum 300**), serta filter waktu bersama di bawah (`from, to, hour_from, hour_to, date`). Urutan: `created_at DESC, id DESC`. Setiap baris dikembalikan dengan `details` yang sudah di-*parse* menjadi objek (atau null). Antarmuka meminta `limit=300` (bab 16).

Filter waktu bersama (`repositories/time-filter.ts`)

Satu fungsi, `buildTimeFilter(kolom, filter)`, menyusun klausa `WHERE` untuk kolom `created_at` dan dipakai oleh empat daftar: keluhan (`GET /api/reports`), laporan pekerjaan lama (`GET /api/work-logs`), jejak audit (`GET /api/activity`), dan laporan kebersihan (`GET /api/cleaning/reports`). Agar arti “kapan” sama di semua tab, tidak ada daftar yang menulis aturan waktunya sendiri. `readTimeFilter(query)` membaca parameter dari *query string*.

Parameter	Arti
`from, to`	Tanggal WIB `YYYY-MM-DD`, **inklusif di kedua ujung**. Boleh hanya salah satu. Nilai yang tidak valid (format salah atau tanggal yang tak dapat di-*parse*) diabaikan, bukan galat
`date`	Satu hari WIB; hanya dipakai bila `from` atau `to` tidak valid atau tidak ada (menjadi pengganti ujung yang kosong)
`hour_from, hour_to`	Jam dalam sehari, WIB, **inklusif**, berbentuk `H`, `HH`, atau `HH:MM`. Bentuk jam saja berarti awal jam itu bila sebagai batas awal (`7` = 07:00) dan menit ke-59 bila sebagai batas akhir (`12` = 12:59). Bentuk `HH:MM` tepat ke menit. Jam > 23, menit > 59, atau format lain diabaikan

Arti inklusif pada ujung menit: `hour_from=07:30&hour_to=12:15` mencakup 07:30:00 sampai 12:15:59. Bila `hour_from` lebih besar daripada `hour_to` (mis. `22:00` sampai `06:00`), jendela dianggap **melewati tengah malam**: yang dipilih adalah menit ≥ 22:00 *atau* ≤ 06:00.

SQL yang dihasilkan

```
-- tanggal: batas hari WIB diterjemahkan ke UTC oleh wibDayRange() (adapters/clock.ts)
created_at >= '2026-10-28 17:00:00'     -- from=2026-10-29 (00:00 WIB)
created_at <  '2026-10-30 17:00:00'     -- to=2026-10-29 (awal hari WIB berikutnya, eksklusif)

-- jam: menit sejak tengah malam WIB, dihitung dari kolom UTC
(CAST(strftime('%H', created_at, '+7 hours') AS INTEGER) * 60
 + CAST(strftime('%M', created_at, '+7 hours') AS INTEGER))  AS minuteOfDay

minuteOfDay >= ? AND minuteOfDay <= ?          -- jendela biasa (lo ≤ hi)
(minuteOfDay >= ? OR minuteOfDay <= ?)         -- jendela lewat tengah malam (lo > hi)
```

Hanya satu batas jam pun sah: hanya `hour_from` berarti “mulai jam itu sampai akhir hari”, hanya `hour_to` berarti “dari tengah malam sampai jam itu”. Semua nilai masuk lewat *bind parameter*; kolom yang dipakai tidak berasal dari masukan pengguna.

> **Dua hal yang perlu diketahui pembaca.** (1) Jendela jam diterapkan pada *setiap hari kalender* dalam rentang tanggal. Jendela malam `22:00–06:00` dengan `from=to=` satu tanggal memilih 00:00–06:00 dan 22:00–23:59 pada tanggal itu, bukan satu malam yang bersambung dari tanggal itu ke hari berikutnya. (2) Ekspresi menit memakai fungsi pada kolom, sehingga bagian jam tidak dapat memakai indeks; bagian tanggal tetap dapat. Pada volume data sekarang ini tidak terasa, tetapi belum diukur pada volume besar.

Filter tambahan per daftar: keluhan menerima `status, priority, area_id, building`; laporan kebersihan menerima `staff_id, area_id, alerts=1`. Rincian ada di bab 15.

12. Ringkasan harian, statistik, dan grafik dashboard

Cron dan ringkasan harian

`wrangler.jsonc` menetapkan `"crons": ["0 10 * * *"]` : 10.00 UTC = **17.00 WIB** setiap hari. Penangan `scheduled()` di `src/index.ts` memanggil `generateDailySummary(env, wibDate())` lewat `ctx.waitUntil()` ; hasilnya (jumlah laporan) atau galatnya hanya dicetak ke log Worker. Tombol “Buat ulang” di tab Ringkasan (`POST /api/summary/generate` , boleh dengan `?date=`) menjalankan fungsi yang sama kapan saja. Secara lokal cron dapat dipicu dengan `wrangler dev --test-scheduled` lalu membuka `/__scheduled` (rute ini dikecualikan dari fallback SPA lewat `run_worker_first`).

Langkah `generateDailySummary(env, date)` di `services/summary-service.ts` :

1. Ambil semua keluhan pada hari WIB itu (`reportsOnDate` : lokasi, prioritas, ringkasan AI atau teks asli, urut waktu).
2. Bila kosong, **tidak ada panggilan LLM**: teksnya tetap “Tidak ada keluhan yang masuk pada hari ini.” dengan sorotan kosong. Bila ada, `summarizeDay()` (bab 8) meminta JSON `{summary, highlights[]}` .
3. *Upsert* ke `daily_summaries` (`date` kunci utama; `report_count` , `summary` , `highlights` berupa teks JSON, `created_at` diperbarui). Hanya **satu ringkasan per tanggal**; membuat ulang menimpanya.
4. `forget('summary')` membuang cache ringkasan, lalu satu baris `daily_summary` ditulis ke jejak audit.

`GET /api/summary` (`storedSummary`) membaca ringkasan tersimpan dan menyimpannya di cache memori 60 detik per tanggal (bab 17). Bentuk respons: `{date, exists:false}` bila belum ada, atau `{date, report_count, summary, highlights[], created_at, exists:true}` . Karena cron berjalan pukul 17.00, keluhan yang masuk sesudahnya tidak ikut sampai ringkasan dibuat ulang; ringkasan hari itu tidak otomatis diperbarui lagi.

Kartu angka: `GET /api/summary/stats`

Untuk satu hari WIB (`?date=YYYY-MM-DD` , bawaan hari ini; format salah → 400). Dihitung oleh `dailyCounts` dalam satu *batch* D1 dan disimpan di cache memori 15 detik per tanggal:

```
{ "date": "2026-10-29",
  "today": { "total": 9, "high": 2, "medium": 4, "low": 3, "ai_failed": 0 },
  "unresolved": 14,
  "top_locations": [ { "location": "Gedung Dewi Sartika (Perkuliahan) – Lantai 2", "count": 3 }, …
] }
```

`today` menghitung keluhan yang *dibuat* pada hari itu (`high/medium/low` menurut prioritas hasil AI; keluhan yang belum dianalisis tidak masuk salah satunya, sehingga jumlahnya bisa kurang dari `total`); `unresolved` adalah semua keluhan berstatus bukan `resolved` sepanjang waktu; `top_locations` maksimal lima lokasi terbanyak hari itu (digabung lewat `location_info`).

Data grafik: `GET /api/summary/charts?days=`

`days` bawaan 14, **maksimum 90** (nilai bukan angka kembali ke 14). Respons dihitung oleh `chartData(env, days)` , disimpan di cache memori 60 detik per nilai `days` , dan dikirim dengan `Cache-Control: private, max-age=60` . Query dibagi dua *batch* D1 (`chartCounts` dan `advancedCounts` di `repositories/summaries.ts`) yang dijalankan paralel. Hari kosong diisi nol, bukan dilewati: celah terbaca sebagai data hilang, nol terbaca sebagai “tidak ada laporan”, dan itulah kebenarannya. Grafik hanya mencakup keluhan, bukan laporan kebersihan.

Bidang	Bentuk	Cakupan	Dipakai oleh
`daily`	`[{date, total, resolved}]`, tepat *days* elemen, tanggal WIB, terlama dulu	jendela *days* hari	Grafik area “masuk vs selesai”; sparkline kartu “Hari ini” dan “Belum selesai” (`total − resolved`)
`dailyByPriority`	`[{date, high, medium, low}]`, *days* elemen	jendela	Batang bertumpuk “komposisi prioritas per hari”; sparkline kartu prioritas tinggi
`statuses`	`[{status, count}]`	jendela (keluhan yang *dibuat* dalam *days* hari)	Donat status
`aging`	larik 5 angka: keluhan terbuka menurut lama menunggu, pita <2 j, 2–8 j, 8–24 j, 1–3 h, >3 h	semua keluhan berstatus bukan `resolved`, sepanjang waktu	Kolom “Lama menunggu”
`hourByDay`	`[{day, hour, count}]`; `day` 0 = Minggu, `hour` 0–23, WIB	sepanjang waktu	Jam radial 24 jam; peta panas hari × jam
`matrix`	`[{building_code, category, count}]`; tempat di luar gedung memakai namanya sebagai kode	sepanjang waktu	Peta panas gedung × kategori
`resolutionByPriority`	`[{priority, count, minutes}]` (rata-rata menit)	sepanjang waktu, hanya yang sudah selesai	Batang “Waktu selesai” per prioritas
`buildings`	`[{building_code, building_name, count}]`, 10 teratas	sepanjang waktu	Treemap gedung
`categories`, `priorities`	`[{category, count}]` / `[{priority, count}]`	sepanjang waktu	Cincin kategori; donat prioritas. Kategori dihitung dengan `json_each`: satu keluhan dapat masuk beberapa kategori
`resolution`	`{count, minutes}`	sepanjang waktu, yang sudah selesai	Kartu “Waktu selesai”
`trend`	`{days, reports, previous_reports, change, resolved, high}`; `change` dalam persen, `null` bila jendela sebelumnya nol	*days* hari terakhir vs *days* hari sebelumnya	Pengukur “Selesai” (`resolved / reports`); panah perubahan pada kartu “Hari ini”

Waktu penyelesaian = `resolved_at − created_at` dalam menit (`julianday`). Pengelompokan per hari memakai `date(created_at, '+7 hours')` sehingga “hari” berarti hari WIB.

> **Cakupan waktu tidak seragam.** Pemilih rentang 7/14/30/90 hari hanya menggeser `daily`, `dailyByPriority`, `statuses`, dan `trend`. Cincin kategori, donat prioritas, treemap, kedua peta panas, jam radial, dan waktu selesai menghitung *sepanjang waktu* sehingga jumlahnya tidak sama dengan donat status pada rentang pendek. Jendela `trend` dan `statuses` adalah jendela bergulir (`datetime('now', '-N days')`) yang dimulai pada jam yang sama dengan saat query dijalankan, sedangkan `daily` memotong per hari WIB; jumlah `daily` dan `trend.reports` dapat berbeda sedikit karena hari parsial di ujung jendela.

Bagaimana tiap visual di tab Ringkasan memakainya

`OverviewPanel.tsx` memanggil empat endpoint sekaligus (`charts(days)`, `stats()`, `cleaning/recap?date=hari ini`, `summary()`) dan memuat ulang tiap 60 detik selama tab terlihat. Tidak ada angka yang dihitung ulang di peramban selain penjumlahan sederhana dan pengelompokan ke ember tiga jam.

Visual	Sumber data	Cara membacanya
Kartu ringkasan AI	`summary()`; `stats.top_locations`	Paragraf dan sorotan hasil cron/tombol; di sampingnya batang lokasi terbanyak hari ini. Tanda “!” menjelaskan jadwal 17.00 WIB
Pengukur “Selesai” (`RadialGauge`)	`trend.resolved / trend.reports`	Persen keluhan dalam jendela yang sudah selesai (0 bila tidak ada laporan)
Empat `KpiCard`	`stats.today.total`, `stats.today.high`, `stats.unresolved`, `resolution`	“Hari ini” (dengan sparkline `daily.total` dan perubahan `trend.change`), prioritas tinggi hari ini, belum selesai, rata-rata waktu selesai
Area chart	`daily`	Dua kurva: masuk dan selesai per hari
Donat status	`statuses`	Baru / diproses / selesai
Cincin kebersihan per petugas (`StaffRings`) dan pengukur kedua	rekap `cleaning/recap` hari ini	Tugas terlapor dibanding tugas yang diharapkan; lencana merah bila ada laporan di luar kampus (bab 7)
Jam radial 24 jam (`HourClock`)	`hourByDay` dijumlahkan per jam	Panjang baji = banyaknya keluhan pada jam itu; jam tersibuk menjadi judul di tengah
Cincin kategori (`RadialBars`)	`categories`	Cincin terpanjang = masalah paling sering; daftar angka di sampingnya
Donat prioritas	`priorities`	Tinggi / sedang / rendah
Treemap gedung	`buildings`	Luas petak sebanding jumlah keluhan; tanda “!” menerangkan arti huruf gedung dari daftar lokasi (`cleaning/areas`, cache 5 menit)
Kolom umur (`AgingBars`)	`aging`	Lima kolom dari baru sampai terabaikan
Peta panas gedung × kategori	`matrix`	Gedung mana menjadi sumber masalah apa; nol dibiarkan kosong, bukan langkah terpucat

Batang waktu selesai	`resolutionByPriority`	Uji nyata: prioritas tinggi seharusnya selesai paling cepat
Peta panas hari × jam	`hourByDay` dikelompokkan per tiga jam	Dasar jadwal ronda
Komposisi prioritas per hari (`StackedBarChart`)	`dailyByPriority`	Apakah beban berat bertambah, atau hanya jumlahnya

13. Papan publik dan papan peringkat

Papan publik (`/reports`)

Dapat dibaca siapa pun tanpa login. Halaman `PublicReports.tsx` punya dua tab di bawah satu penyaring lokasi (Gedung, Lantai/Halaman):

- **Pelapor**: `GET /api/reports/public` (filter `status, priority, limit` ; limit bawaan 100, maks. 200). Kolom yang dikembalikan *hanya*: id, area_id, status, priority, categories, summary, ai_status, created_at, resolved_at, location_name, building_code, floor, dan `proof_photo_url` ; ditambah `counts: {total, resolved}` atas *seluruh* keluhan (bukan hanya yang difilter), untuk kalimat “x dari y laporan sudah selesai ditangani”.
- **Petugas**: `GET /api/cleaning/reports/public` (maks. 200, bawaan 60): area, catatan petugas, waktu, dan tiap tugas dengan fotonya serta jam server penerimaan foto.

Keduanya dikirim dengan `Cache-Control: public, max-age=10, stale-while-revalidate=60` . Halaman ini tidak melakukan polling; ia memuat ulang saat filter berubah atau halaman dibuka lagi.

Data yang sengaja tidak ditampilkan

- Teks mentah keluhan, foto keadaan dari pelapor, dan `reporter_id` tidak ikut: papan tidak boleh menjadi saluran teks yang belum disaring, atau memuat wajah orang yang tak sengaja terfoto.
- **Tidak ada nama petugas** pada kedua tab. Foto bukti penyelesaian ditampilkan terbuka karena itulah yang membuat klaim “sudah ditangani” dapat diperiksa siapa pun.
- Pada laporan kebersihan, posisi GPS, jarak, dan penanda “di luar kampus” tetap hanya untuk SPV; yang publik hanya area, tugas, foto, dan waktu.

> **Satu pengecualian yang disengaja:** petugas tidak login, sehingga `GET /api/reports/open` (keluhan yang belum selesai) memuat teks asli dan foto keadaan dari pelapor tanpa sesi. Isinya dibatasi (tanpa `reporter_id` dan rincian AI), tetapi itu bukan papan yang tertutup. Teks bebas `note` dari petugas pada laporan kebersihan juga tampil di papan publik apa adanya.

Papan peringkat (`/leaderboard`)

`GET /api/leaderboard` mengembalikan 20 pelapor teratas (`reporterLeaderboard`): urutan jumlah keluhan, lalu jumlah yang selesai, lalu nama; hanya keluhan dengan `reporter_id` milik akun berperan `reporter` yang dihitung. Yang tampil hanya nama tampilan akun (bukan username), jumlah keluhan, dan jumlah yang selesai; keluhan anonim tidak masuk peringkat. Pelapor yang sedang masuk juga menerima `me: {rank, reports}` (rank = 1 + jumlah pelapor yang keluhannya lebih banyak; 0 bila belum melapor), termasuk bila berada di luar 20 besar. Daftar 20 besar dibagikan ke semua orang lewat cache memori 30 detik, dan hanya `me` yang dihitung per orang. Peringkat berdasar jumlah, tetapi jumlah yang *selesai* ditampilkan berdampingan agar yang dihargai bukan sekadar volume.

14. Penyimpanan foto

Semua foto berada di satu *bucket* R2 (`kato` , *binding* `BUCKET`) yang **tidak punya akses publik**. Kodenya ada di `adapters/storage.ts` (fungsi `savePhoto` , `getPhoto` , `readPhoto` , `photoUploadedAt` , `deletePhoto`) dan `routes/uploads.ts` .

Aspek	Penerapan
Folder (`FOLDER`)	Empat jenis foto, dipisah menurut awalan kunci: `report/` (foto keadaan dari pelapor), `proof/` (foto bukti penyelesaian keluhan), `cleaning/` (foto satu tugas harian), `work/` (foto laporan pekerjaan versi lama; tidak lagi dibuat, tetap dapat dibaca)
Memilih jenis	`POST /api/uploads?kind=report | proof | cleaning` . Nilai tak dikenal, kosong, atau `work` (jenis lama) menjadi `report` . Server memeriksa awalan itu lagi saat foto dipakai: keluhan hanya menerima `report/` , penutupan hanya `proof/` dan laporan kebersihan hanya `cleaning/` . Foto satu jenis tidak dapat dipakai sebagai jenis lain
Kunci objek	`<folder>/<YYYY-MM-DD>/<uuid>.<jpg|png|webp>` . Tanggalnya tanggal UTC saat diunggah; awalan tanggal memudahkan penelusuran dan pembersihan per periode
Awalan lama	`laporan/` , `bukti/` , `kerja/` : objek dari sebelum penggantian nama tetap dapat disajikan dan dihapus, tetapi tidak ada yang baru ditulis ke sana
Batas unggah	≤ 5 MB; tipe `image/jpeg` , `image/png` , `image/webp` (415 bila lain, 413 bila terlalu besar, 400 bila tanpa berkas). Kamera di halaman memperkecil bingkai ke sisi terpanjang 1600 px sebelum diunggah (bab 16)
Unggah	Lewat *binding*: Worker mengalirkan isi berkas ke bucket (`file.stream()`), bukan *presigned URL*, sehingga tidak ada kredensial S3 di mana pun. Endpoint ini **terbuka tanpa login** karena pelapor dan petugas tidak login
Penyajian	`GET /api/uploads/<kunci>` hanya untuk tujuh awalan di atas (`isAllowedKey`); lainnya 404. `Cache-Control: public, max-age=31536000, immutable` karena nama UUID tidak pernah ditimpa; `ETag` objek dan `If-None-Match` menghasilkan 304 (R2 `onlyIf`)
Pembacaan untuk vision	`readPhoto()` memuat seluruh objek ke memori lalu dikodekan base64 menjadi *data URL*: 5 MB menjadi sekitar 6,7 MB, jauh di bawah batas memori Worker
Waktu foto	`photoUploadedAt()` membaca waktu penerimaan objek di R2 (`head().uploaded`). Inilah “jam foto” laporan kebersihan: jam server, bukan jam ponsel (bab 7)
Penghapusan	Keluhan yang dihapus: foto keadaan dan foto bukti dihapus *setelah* log ditulis dan baris dihapus. Foto bukti yang ditolak model atau gagal diperiksa dihapus seketika; foto laporan kebersihan yang terlalu tua (> 3 jam) atau berlokasi tak dikenal juga dihapus. `deletePhoto()` menelan galatnya sehingga tidak pernah menggagalkan operasi induk
Mengapa bukan `r2.dev`	DNS `pub-*.r2.dev` dibajak sebagian ISP Indonesia: terverifikasi saat pengembangan, resolver lokal mengembalikan alamat non-Cloudflare dan TLS gagal, sedangkan lewat 1.1.1.1 benar. Menyajikan dari *origin* sendiri menghilangkan ketergantungan itu, dan bucket tidak perlu dibuka ke publik

> **Batasan yang jujur:** unggahan tidak dibatasi laju dan tidak butuh login, serta foto yang sudah diunggah tetapi tidak pernah dilampirkan ke laporan (pengguna menutup halaman di tengah jalan) tidak dibersihkan otomatis. Foto yang sudah diterima (bukti bersih, foto keadaan, foto tugas harian) disimpan tanpa batas waktu; tidak ada kebijakan retensi. Lihat bab 23.

15. Referensi API lengkap

Seluruhnya **36 rute**, dicocokkan satu per satu dengan `src/index.ts` dan `src/routes/*.ts`. Semua respons berupa JSON kecuali penyajian foto. Galat selalu `{"error": "pesan"}` (berbahasa Indonesia), kadang dengan bidang tambahan. Isi permintaan JSON kecuali unggah (*multipart*). Sesi SPV dibawa oleh *cookie* `session` (JWT HS256, HttpOnly, SameSite=Lax, berlaku 12 jam, `Secure` pada HTTPS). Arti kolom "Siapa": **Publik** = tanpa sesi; **Pelapor** = sesi opsional yang diperhatikan bila berperan pelapor; **Nama** = tanpa login, tetapi memakai `staff_id` petugas aktif; **SPV** = `requireSupervisor` (401 tanpa sesi, 403 untuk peran lain). Daftar lokasi ada di `/api/cleaning/areas` (tidak ada lagi `/api/locations`).

Kode status

Kode	Dipakai untuk
`200` / `201`	Sukses / dibuat. Kiriman kembar dijawab **200** dengan `duplicate:true` (bukan 201)
`304`	ETag cocok (GET tertentu) atau foto tidak berubah
`400`	Validasi gagal, atau aturan bisnis sederhana (nama petugas tak dikenal, foto tak ditemukan, bukti belum ada, akun sendiri tak bisa dinonaktifkan/dihapus, SPV aktif terakhir tak bisa dihapus)
`401` / `403`	Belum masuk, atau salah sandi / peran tidak berhak
`404`	Tidak ditemukan (lokasi, keluhan, akun, berkas). Path `/api/*` yang tak dikenal juga 404 JSON "Endpoint tidak ditemukan"
`409`	**Konflik**: username sudah dipakai; keluhan sudah selesai dan tak bisa dibuka lagi; akun yang punya riwayat tak bisa dihapus
`413` / `415`	Berkas > 5 MB / tipe bukan JPG, PNG, WebP
`422`	**Foto ditolak** (foto bukti dinilai kotor atau bukan lokasi; foto tugas harian terlalu lama atau sudah pernah dipakai); permintaan benar bentuknya, hanya fotonya yang tidak lolos
`429`	Batas pertanyaan Kato Bot (50 per hari, 20 per alamat)
`500`	Galat tak tertangani (`app.onError`): "Terjadi kesalahan pada server". Termasuk login bila `AUTH_SECRET` kosong (bab 19)
`502`	Layanan LLM atau vision gagal (pemeriksaan foto, ringkasan, Kato Bot)

Path di luar `/api/` yang bukan aset statis dijawab `index.html` (fallback SPA, `app.notFound` dan `not_found_handling`). Path API lama berbahasa Indonesia (`/api/lokasi`, `/api/pengguna`, …) **tidak** dialihkan dan menjawab 404.

Sistem, akun, dan sesi

Metode, jalur, siapa	Permintaan	Respons	Galat
`GET /api/health` Publik	—	`{ok:true, time}`	

POST /api/auth/login Publik	{username, password}	{id, name, role} + *cookie*	400 kosong; 401 salah, akun nonaktif, atau peran staff (pesan sama untuk semuanya)
POST /api/auth/register Publik	{username, name, password} : username 3–30 [a-zA-Z0-9._-] ; nama 2–60; sandi 8–200	201 {id, name, role:'reporter'} + *cookie*	400 validasi; 409 username dipakai
POST /api/auth/logout Publik	—	{ok:true}	
GET /api/auth/me Sesi	—	{id, name, role}	401 belum masuk

Keluhan (/api/reports)

Metode, jalur, siapa	Permintaan	Respons	Galat
POST /api/reports Pelapor	{area_id, description, photo_key} : area_id 1–30 (diubah ke huruf besar); description 5–1000; photo_key wajib, ≤ 200 dan harus berawalan report/	201 {id, location, duplicate:false} ; 200 {…, duplicate:true} bila keluhan identik di area yang sama dalam 2 menit. Analisis AI dijalankan sesudah respons	400 validasi (“Keluhan terlalu pendek”, “Foto keadaan wajib dilampirkan”, “Foto keadaan tidak valid”); 404 lokasi tak dikenal
GET /api/reports/public Publik	?status, priority, limit (bawaan 100, maks. 200)	{data:[PublicReport], counts:{total, resolved}}	
GET /api/reports/open Publik	?building (satu huruf), floor (1–2 digit), limit (bawaan 100, maks. 200)	{data:[Report]} berstatus bukan resolved , prioritas tinggi dulu lalu terlama; tanpa reporter_id, ai_error, ai_model, ai_ms, proof_model, proof_ms	
GET /api/reports/mine Pelapor	—	{data:[Report]} , 50 terbaru milik akun; kosong bila bukan pelapor	
GET /api/reports/:id Publik	—	Report (DTO lengkap, lampiran A)	404
GET /api/reports SPV	?status, priority, area_id, building, from, to, hour_from, hour_to, date, limit (bawaan 100, maks. 200).	{data:[Report]} : baru, lalu diproses, lalu selesai; di dalamnya prioritas tinggi dulu, lalu terbaru	401/403

	Nilai `status` / `priority` di luar enum diabaikan		
`PATCH /api/reports/:id` Nama atau SPV	`{status: new | in_progress | resolved, staff_id?, proof_photo_key?}`. Dengan `staff_id` (petugas aktif) pelakunya petugas itu; tanpanya hanya sesi SPV yang diterima, atas nama SPV	`{ok:true, status, verification: {verdict, reason, model, ms}|null}`	400 status tak valid / nama petugas tak dikenal / "Pilih namamu dulu" / foto bukti tak ditemukan (juga bila awalan bukan `proof/`) / bukti belum ada; 404; **409** "Laporan ini sudah selesai." (membuka kembali keluhan selesai); **422** `{error, verdict:'dirty' | 'wrong_place', reason}`; **502** pemeriksaan foto gagal
`POST /api/reports/:id/reanalyze` SPV	—	`{ok:true}`	404
`DELETE /api/reports/:id` SPV	—	`{ok:true}`	404

Semantik `PATCH /api/reports/:id` (klaim, ambil alih, selesai)

Satu endpoint, tiga makna, diputuskan oleh `changeStatus()` di `report-service.ts` menurut urutan ini:

1. Keluhan tak ada → 404.
2. Keluhan sudah `resolved` dan permintaannya status lain → **409**. Keluhan selesai tidak dapat dibuka lagi, sebab membukanya akan menghapus foto bukti yang menutupnya.
3. **Klaim** (`in_progress`): bila keluhan sudah `in_progress` atas nama orang yang sama, tidak ada yang ditulis maupun dicatat (200, `verification:null`). Bila dipegang orang lain, klaim ini adalah **ambil alih**: `staff_name` diganti dan log `status_changed` mencatat "X mengambil alih laporan di … dari Y" dengan `details.handed_over_from`. Klaim bukan kunci.
4. **Selesai** (`resolved`): bila ada `proof_photo_key` baru, foto diperiksa model vision lebih dulu (bab 9); yang tidak lolos dihapus dan menghasilkan 422. Tanpa foto lolos yang tersimpan, hasilnya 400 "Foto bukti penyelesaian wajib diunggah lebih dulu." (`canResolve` di `domain/types.ts`). Bila lolos, status, nama, foto, putusan, alasan, model, dan waktu ditulis bersama (`updateStatus`); `resolved_at` diisi waktu server.

Nilai `new` diterima server tetapi tidak dikirim oleh antarmuka mana pun. Setelah penulisan berhasil, cache statistik, grafik, dan papan peringkat di isolate itu dibuang (`reportsChanged()`).

Log pekerjaan versi lama, hanya baca (`/api/work-logs`)

Metode, jalur, siapa	Permintaan	Respons	Galat
`GET /api/work-logs` SPV	`?staff_id, toilet_id, building, from, to, hour_from, hour_to, date, limit` (bawaan 100, maks. 200)	`{data:[WorkLog]}`, terbaru dulu	401/403

Hanya pembacaan yang tersisa; dipakai tab Log → Log pekerjaan. Jalur tulisnya (`POST`) dan fungsi kliennya (`createWorkLog`) dihapus karena tidak ada lagi yang memakainya; pekerjaan harian dicatat lewat `POST /api/cleaning/reports`.

Lokasi dan laporan kebersihan (`/api/cleaning`)

Metode, jalur, siapa	Permintaan	Respons	Galat
`GET /api/cleaning/areas` Publik	—	`{data:[{id, name, building_code, building_name, floor, tasks[], staff[{id,name}]}], campus:{lat, lng, radius_m}}` . `floor` : 0 = halaman, null = gedung utuh atau tempat di luar gedung	
`GET /api/cleaning/areas/:id` Publik	id `[A-Za-z0-9-]{1,30}` (huruf besar dipakai)	satu area + `campus`	404 “Kode area tidak dikenal”
`POST /api/cleaning/reports` Nama	`{staff_id, area_id, tasks:[{task, photo_key}], note?, position?:{latitude, longitude, accuracy?}}` ; `task` salah satu dari enam: `sweep_mop, glass, toilet, trash, gallon, tidy` ; 1–6 tugas tanpa duplikat; satu foto hanya untuk satu tugas; `note` ≤ 500	201 `{id, area, duplicate:false, location_status:'ok' | 'far' | 'unknown', distance_m}` ; 200 dengan `duplicate:true` bila foto-fotonya persis milik laporan sebelumnya oleh petugas dan area yang sama	400 validasi / nama tak dikenal / tugas tak termasuk di area itu / foto tak ditemukan (berisi `task`); 404 area tak dikenal; **422** foto > 3 jam sejak diterima server, atau foto sudah dipakai laporan lain (berisi `task`)
`GET /api/cleaning/reports/public` Publik	`?limit` (bawaan 60, maks. 200)	`{data:[{id, area_id, area_name, note, created_at, tasks:[{task, photo_url, photo_at}]}]}` ; tanpa nama petugas dan tanpa GPS	
`GET /api/cleaning/reports` SPV	`?staff_id, area_id, alerts=1, from, to, hour_from, hour_to, date, limit` (bawaan 200, maks. 500)	`{data:[CleaningReport + tasks]}` , terbaru dulu; memuat posisi, jarak, `location_status`	401/403
`GET /api/cleaning/recap` SPV	`?date=YYYY-MM-DD` (bawaan hari ini WIB; tanggal tak valid dianggap hari ini)	`{date, staff:[{staff_id, staff_name, areas[], expected, done, reports, alerts}], totals:{expected, done, reports, alerts}}` ; urut dari penyelesaian terendah	401/403

Foto, petugas, ringkasan, jejak audit, akun, papan peringkat, Kato Bot

Metode, jalur, siapa	Permintaan	Respons	Galat
`POST /api/uploads` Publik	`?kind=report | proof | cleaning` ; *multipart* bidang `file`	201 `{key, url}`	400 tanpa berkas; 415 tipe; 413 > 5 MB
`GET /api/uploads/:key` Publik	kunci dengan garis miring (`:key{.+}`)	biner + `Cache-Control` immutable + `ETag`	304; 404 awalan salah atau objek tak ada
`GET /api/staff` Publik	—	`{data:[{id, name}]}` , petugas aktif, urut nama; `Cache-Control: public, max-age=60, stale-while-revalidate=600`	
`GET /api/summary` SPV	`?date` (bawaan hari ini WIB)	`{date, exists:false}` atau `{date, report_count, summary, highlights[], created_at, exists:true}`	400 format tanggal
`GET /api/summary/stats` SPV	`?date`	`{date, today:{total, high, medium, low, ai_failed}, unresolved, top_locations: [{location, count}]}`	400
`GET /api/summary/charts` SPV	`?days` (bawaan 14, maks. 90)	ChartData (bab 12)	
`POST /api/summary/generate` SPV	`?date`	`{date, report_count, summary, highlights[], exists:true}`	400; 502 LLM gagal
`GET /api/activity` SPV	`?action, report_id, from, to, hour_from, hour_to, date, limit` (bawaan 100, maks. 300)	`{data:[{id, created_at, action, report_id, actor, summary, details}]}`	401/403
`GET /api/users` SPV	—	`{data:[{id, username, name, role, active, created_at}]}` , hanya SPV dan petugas; SPV dulu, lalu nama	401/403
`POST /api/users` SPV	`{role:'staff', name}` atau `{role:'supervisor',`	201 `{id, name, role, username, active:1}`	400; 409 username dipakai

	`username, name, password}`	(`username` null untuk petugas)	
`PATCH /api/users/:id` SPV	`{name?, password?, active?}` , minimal satu; sandi ≥ 8	`{ok:true}`	400 “Tidak ada yang diubah” / menonaktifkan akun yang sedang dipakai / sandi untuk petugas; 404
`DELETE /api/users/:id` SPV	—	`{ok:true}`	400 akun sendiri, atau SPV aktif terakhir; **409** `{error, records}` bila akun punya catatan; 404 (juga untuk akun pelapor, yang tidak dikelola di sini)
`GET /api/leaderboard` Pelapor	—	`{data:[{id, name, reports, resolved}], me: {rank, reports}|null}`	
`GET /api/ask` Publik	—	`{daily_limit}`	
`POST /api/ask` Publik (hak menurut sesi)	`{question (3–500), history:[{role:'user' | 'assistant', text}]}` , maks. 20 giliran, 6 terakhir yang dikirim ke model	`{text, tools:[{name, arguments}], tokens: {prompt, completion, cache_hit}, ms, remaining_today}`	400 validasi; **429** batas harian atau per alamat; 502 model gagal

Aturan `DELETE /api/users/:id`

Urutan pemeriksaan di `deleteAccount()` (`services/user-service.ts`): akun tak ada atau berperan pelapor → 404; akun sendiri → 400; SPV aktif terakhir → 400; akun yang masih dirujuk `work_logs.staff_id` , `cleaning_reports.staff_id` , atau `reports.reporter_id` → **409** dengan jumlahnya di bidang `records` dan saran menonaktifkan. Bila lolos, akun dan penugasan areanya (`cleaning_assignments`) dihapus dalam satu *batch*, cache nama petugas dan lokasi dibuang, dan `user_changed` dicatat. Menonaktifkan (`PATCH active:false`) menyembunyikan nama sama efektifnya dan menjaga riwayat. Perhatikan: keluhan yang diklaim atau diselesaikan petugas hanya menyimpan *nama* (`reports.staff_name`), jadi tidak ikut dihitung sebagai “catatan”.

Header pada GET tertentu

ETag lemah dipasang pada `GET` di bawah `/api/summary, /api/activity, /api/cleaning, /api/reports, /api/work-logs, /api/users, /api/staff, /api/leaderboard` ; permintaan dengan `If-None-Match` yang cocok dijawab 304 tanpa isi. Foto dikecualikan karena ETag-nya datang dari R2. `Cache-Control` per rute ada di bab 17.

16. Antarmuka pengguna

Frontend adalah aplikasi React 18 satu halaman (Vite, Tailwind, React Router 6) di `web/src/`, disajikan oleh Worker yang sama dengan API. Titik masuknya `web/src/main.tsx` memasang penyedia tema, bahasa, dan sesi, lalu router, lalu `FloatingChat` sekali untuk seluruh aplikasi.

Peta rute

Rute	Halaman	Isi
`/`	Home	Kepala halaman dengan masuk/daftar (atau sapaan, tombol ke dashboard bila SPV, dan keluar). Bila pelapor yang masuk: daftar "Laporan saya" (`GET /api/reports/mine`). Dua pintu peran tanpa login, **"Saya pelapor"** (→ `/student`) dan **"Saya petugas"** (→ `/staff`), serta tautan ke papan publik dan papan peringkat. Tidak ada bagian chatbot atau kode QR di sini lagi
`/student`	ReportForm	Formulir keluhan tanpa QR: lokasi dipilih di halaman (Gedung, lalu Lantai/Halaman)
`/report/:locationId`	ReportForm	Tujuan QR. Sakelar peran (`RoleSwitch`) Pelapor/Petugas, lokasi sudah terpilih, teks keluhan (≤ 1000), foto dari kamera, kirim. Bila HP menyimpan nama petugas, dialihkan ke `/staff/:locationId` kecuali ada `?as=student` . Setelah terkirim menuju `/reports/:id`
`/reports/:id`	ReportStatus	Konfirmasi, linimasa tiga tahap, hasil analisis AI. Memuat ulang tiap 2 detik selama analisis berlangsung (maks. 20 kali), sesudahnya tiap 15 detik; tab tersembunyi tidak memuat
`/reports`	PublicReports	Papan publik, dua tab (bab 13)
`/staff`, `/staff/:locationId`, `/clean/:areaId`	StaffHome	Halaman petugas, satu halaman untuk keluhan dan tugas harian (di bawah). `/clean/:areaId` membuka langsung tab Tugas harian; `/clean` tanpa id dialihkan ke `/staff`
`/login`, `/register`	Login, Register	Masuk untuk SPV atau pelapor yang ingin ikut peringkat; pendaftaran pelapor
`/leaderboard`	Leaderboard	20 besar dan posisi sendiri
`/supervisor`	SupervisorDashboard	Hanya untuk SPV yang masuk; lainnya dialihkan ke `/login` . Server tetap menolak walau halaman dipaksa tampil (401/403)
`*`	—	Path lain yang tak dikenal dialihkan ke `/`

Semua halaman selain Home dimuat malas (`React.lazy` dengan `Suspense fallback={null}`), sehingga pengunjung pertama tidak mengunduh dashboard dan grafiknya. Router memahami awalan lokasi dari stiker lama: `locate()` di `lib/places.ts` menerjemahkan id area (`C-2` , `C-HALAMAN` , `LAPANG-VOLI`), bentuk `<gedung>-<lantai>` dari stiker pintu toilet (`A-2`), dan huruf gedung saja (`G`) ke gedung/lantai yang sesuai.

Pengalihan URL lama

Stiker yang sudah tertempel dan tautan yang beredar tetap berfungsi: `LegacyRedirect` di `main.tsx` mengalihkan (parameter, query, dan hash ikut terbawa; `?sebagai=mahasiswa` menjadi `?as=student`). Pengalihan terjadi di *router peramban*, bukan di server.

URL lama	URL baru	URL lama	URL baru
`/lapor/:id`	`/report/:id`	`/laporan`	`/reports`
`/mahasiswa`	`/student`	`/laporan/:id`	`/reports/:id`
`/petugas`	`/staff`	`/masuk`	`/login`
`/petugas/:id`	`/staff/:id`	`/daftar`	`/register`
`/kerja/:id`	`/staff/:id`	`/peringkat`	`/leaderboard`
`/spv`	`/supervisor`		

Dashboard SPV (`/supervisor`)

Lima tab dalam satu baris (juga di ponsel; tab dengan label panjang mendapat kolom lebih lebar): **Ringkasan** (halaman awal), **Laporan**, **Log**, **Petugas & Supervisor**, **QR lokasi**. Dua tab memuat dua daftar yang dibedakan oleh sakelar kecil (`SubTabs`). Tiap tab adalah *chunk* sendiri yang diunduh saat pertama dibuka. Lencana merah di tab Laporan menghitung laporan kebersihan hari ini yang berlokasi di luar kampus atau tanpa GPS (`GET /api/cleaning/reports?alerts=1` untuk hari ini, tiap 60 detik di semua tab).

Tab / sub-tab	Isi
Ringkasan	`OverviewPanel` : kartu ringkasan AI hari ini di paling atas (tombol “Buat ulang”, tanda “!” berisi jadwal 17.00 WIB, dan batang lokasi terbanyak hari ini); pemilih rentang 7/14/30/90 hari (bawaan 90); pengukur tingkat penyelesaian dan empat kartu angka; grafik area, donat status; cincin kemajuan kebersihan per petugas; jam radial; cincin kategori; donat prioritas; treemap gedung (tanda “!” menerangkan kode gedung); kolom umur; peta panas gedung × kategori; batang waktu selesai per prioritas; peta panas hari × jam; komposisi prioritas per hari. Pemetaan visual ke data ada di bab 12. Memuat ulang tiap 60 detik
Laporan › Keluhan	Filter status dan prioritas (`Select`), **filter waktu**, hitungan “*n* data ditampilkan”, dan kartu keluhan (di bawah). Memuat paling banyak 200 baris; memuat ulang tiap 30 detik *hanya* saat sub-tab ini sedang tampil
Laporan › Kebersihan	`CleaningPanel` . *Rekap harian*: pilih tanggal (maju/mundur sehari), kartu per petugas dengan tabel area × enam tugas (jam pertama tugas dilaporkan, tanda untuk laporan yang berlokasi di luar kampus atau tanpa GPS, area di luar penugasan diberi catatan “tambahan”), tiga angka ringkas, spanduk peringatan, dan tombol cetak. *Laporan & foto*: filter petugas dan “hanya yang ditandai”, setiap laporan dengan foto per tugas, jam server, jarak dari kampus, dan tautan peta. Memuat ulang tiap 30 detik
Log › Log aktivitas	`ActivityPanel` : pilih jenis aksi (13 pilihan), filter waktu, hingga 300 baris dengan waktu persis WIB. Salinan keluhan yang dihapus dibentangkan langsung (bab 11)
Log › Log pekerjaan	`WorkLogPanel` : data lama (laporan pekerjaan sebelum laporan kebersihan), hanya dibaca. Filter petugas aktif dan filter waktu, hingga 200 baris
Petugas & Supervisor	`AccountsPanel` : tambah nama petugas (nama saja), tambah akun SPV (username, nama, sandi), ubah nama, ganti sandi SPV, aktifkan/nonaktifkan, dan **hapus** dengan konfirmasi. Hapus hanya berhasil untuk akun tanpa riwayat (409 bila punya catatan); akun sendiri tidak bisa dinonaktifkan atau dihapus, dan SPV aktif terakhir tidak bisa dihapus
QR lokasi	`QrCodesPanel` : satu stiker per lokasi, disaring per gedung. Kode dibuat di peramban dengan pustaka `qrcode` (SVG, koreksi galat M) dan menunjuk `<situs ini>/report/<id area>` . Stiker memuat

judul dan petunjuk **dwibahasa** (Indonesia dan Inggris sekaligus, karena stiker cetak tidak dapat mengikuti sakelar bahasa). Cetak A4 (tiga stiker per baris) atau unduh PNG 1024 px per lokasi (`qr-<id>.png`)

Filter waktu

`TimeFilter.tsx` , dipakai tab Keluhan, Log aktivitas, dan Log pekerjaan. Preset **Semua, Hari ini, Kemarin, 7 hari, 30 hari, Pilih tanggal…** (menampilkan "Dari tanggal" / "Sampai tanggal"), ditambah dua pemilih jam "Dari jam" dan "Sampai jam" bergaya jam dial (`ClockPicker` , di bawah), dan keterangan WIB. Preset dihitung menurut hari WIB apa pun zona waktu perangkatnya (`todayWIB()` : UTC+7 dari jam peranti). "Hapus filter waktu" mengembalikan ke Semua. Nilai dikirim sebagai `from, to, hour_from, hour_to` (bab 11).

Kartu keluhan di dashboard

Lencana prioritas, status, dan kategori; waktu relatif; stempel waktu persis WIB "Masuk: …" dan, bila selesai, "Selesai: …" (`useFormatTime()` , zona `Asia/Jakarta`); nama lokasi; ringkasan (atau teks mentah bila belum dianalisis) dan teks asli; kotak "Tindakan" dari AI; foto keadaan; foto bukti berbingkai hijau dengan keterangan "Diverifikasi AI" dan alasan model; tombol "Analisis ulang" (hanya bila gagal) dan "Hapus" (dengan konfirmasi), serta "Ditangani: *nama*". SPV hanya memoderasi; menyelesaikan keluhan adalah pekerjaan petugas di halaman petugas.

Halaman petugas (`StaffHome.tsx`)

Urutan blok: kepala halaman; sakelar peran (hanya bila dibuka lewat `/staff/:locationId`); peta kampus; dua tab **Keluhan** (dengan lencana jumlah) dan **Tugas harian**; pemilih lokasi (Gedung, lalu Lantai/Halaman); lalu isi tab. Tab awal: *Keluhan*, kecuali dibuka dari stiker petugas `/clean/:areaId` (langsung *Tugas harian*) atau dari QR lokasi yang tidak punya keluhan menunggu.

Tab Keluhan

- Semua keluhan yang belum selesai di seluruh kampus langsung tampil tanpa memilih apa pun (`GET /api/reports/open` , sekali saat halaman dibuka dan sesudah tiap tindakan). Gedung dan Lantai/Halaman hanya **penyaring** ("Semua gedung" / "Semua"), bukan syarat.
- **Tidak ada "Siapa kamu?" di halaman ini.** Nama ditanyakan *saat bertindak*, karena satu ponsel dipakai bergantian. Tombol **"Saya kerjakan sekarang"** membuka "Siapa yang mengerjakan?" dengan nama terakhir di HP (`selected-staff`) sebagai isian awal, lalu konfirmasi "Ya, saya yang kerjakan".
- **Klaim bukan kunci.** Selama belum selesai dengan foto bukti, petugas lain boleh **Ambil alih** ("Sedang dikerjakan *nama*" dan petunjuk "Belum ada foto bukti. Siapa pun boleh mengambil alih sampai selesai."). Pengambilalihan tercatat di log aktivitas; klaim ulang oleh orang yang sama tidak dicatat dua kali. Kartu menandai keluhan milik pengguna HP itu dengan "Kamu yang mengerjakan ini".
- **Selesai:** tombol "Sudah beres — foto buktinya" menanyakan "Siapa yang menyelesaikan?" (isian awal: pemegang klaim bila ada), lalu "Lanjut foto bukti" membuka kamera. Foto diunggah sebagai `proof` , lalu `PATCH` dengan `resolved` ; selama itu tombol berubah menjadi "Mengunggah…" dan "Memeriksa foto…". Foto yang tak lolos (bersih dan lokasi benar) menampilkan kotak "Foto ditolak" dengan alasan model; kegagalan layanan menampilkan "Pemeriksaan gagal". Keluhan yang sudah selesai tidak dapat dibuka kembali (409).
- Nama yang dipilih saat bertindak diingat sebagai nama HP itu (`lib/staff.ts`); nama yang kemudian dinonaktifkan SPV dianggap "belum dipilih".

Tab Tugas harian

Pemilih nama di muka (`StaffPicker`), lokasi wajib dipilih, lalu enam tugas (`DailyTasks.tsx`): menyapu & mengepel, mengelap kaca jendela, membersihkan toilet, membuang sampah, mengisi ulang galon, merapikan ruangan. Tiap tugas dibuka sendiri, difoto dari kamera langsung dengan cap (nama · tugas, area, jam *server*, koordinat GPS), diunggah seketika sebagai `cleaning` , lalu dikirim sendiri dengan catatan opsional. Posisi GPS dibaca ulang saat tombol kirim ditekan; bila di luar radius atau tak terbaca, petugas diperingatkan dan boleh

“Tetap kirim (supervisor diberi tahu)”. Petugas boleh mengirim di area mana pun; tidak ada lagi peringatan “bukan area tugasmu”, dan SPV melihat selisihnya di rekap per petugas. Aturan server ada di bab 7.

Kato Bot melayang (`FloatingChat.tsx`)

Chatbot tanya-data bukan lagi bagian halaman beranda, melainkan ikon bundar di setiap halaman, dipasang sekali di `main.tsx` lewat `createPortal` ke `document.body`.

- **Ikon** 56 × 56 px (`SIZE`), tidak pernah lebih dekat dari 12 px ke tepi layar (`MARGIN`), jendela chat berjarak 12 px dari ikon (`GAP`).
- **Seret:** ikon dapat diseret ke mana saja dan *tetap di tempat dilepas*. Ambang geser 5 px untuk mouse dan 10 px untuk sentuhan, supaya jari yang bergoyang tetap dihitung ketukan.
- **Ketuk:** mengetuk (menekan tanpa menggeser) membuka jendela dan **menempelkan** ikon ke tepi kiri atau kanan yang lebih dekat. Ketukan diproses saat jari/penunjuk *diangkat* (`onPointerUp`), bukan lewat `click` ; `click` hanya diterima dengan `detail === 0` (papan ketik: Enter atau Spasi). Alasannya di bab 22.
- **Posisi** disimpan di `localStorage` kunci `chat-anchor` sebagai `{side?, x, y}` ; tanpa nilai tersimpan, ikon berada di kanan bawah. Posisi menempel ikut tepi saat jendela peramban diubah ukurannya.
- **Jendela:** lebar maksimal 390 px, tinggi maksimal 580 px, terbuka ke arah (atas atau bawah, kiri atau kanan) yang lebih lega, mengikuti ikon, dan dapat diseret lewat judulnya (ikon ikut). `Esc` atau tombol tutup menutupnya. Jendela tetap terpasang saat tertutup, sehingga percakapan bertahan selama halaman dipakai (hilang saat muat ulang).
- **Muat malas:** isi chat (`AskPanel`) diunduh pertama kali ikon dibuka, bukan bersama halaman.
- Isinya: contoh pertanyaan, sisa pertanyaan hari ini (`remaining_today`), tombol percakapan baru. SPV yang masuk mendapat jawaban dengan hak SPV (boleh per petugas); lainnya data agregat tanpa nama petugas (bab 8).

Komponen `Select`

Semua dropdown di aplikasi memakai `Select.tsx` buatan sendiri, bukan `<select>` bawaan. Alasannya: peramban memilih arah buka sendiri (ke atas bila merasa tak muat, di tengah layar pada ponsel) dan CSS tidak dapat mengubahnya. Pilihan ditulis sebagai anak `<option>` seperti biasa; `onChange` menerima nilainya.

- **Selalu membuka ke bawah.** Bila ruang di bawah kolom kurang dari sekitar 300 px (`ROOM_WANTED`), *halaman* yang digulir naik, bukan daftar yang dibalik ke atas.
- **Memperpanjang halaman sementara:** bila halaman terlalu pendek untuk digulir sejauh itu, sebuah blok kosong ditambahkan ke `body` selama daftar terbuka dan dibuang saat ditutup.
- **Portal:** daftar digambar di `document.body` (`position: fixed` , `z-[70]`), sehingga tidak terpotong atau tertutup kartu di sekitarnya, dan mengikuti kolom saat halaman digulir atau diubah ukurannya.
- Papan ketik: panah, Home/End, Enter atau Spasi untuk memilih, Esc menutup, huruf melompat ke pilihan berawalan itu. Atribut ARIA `combobox` / `listbox` / `option` . Klik di luar menutup.

`ClockPicker`

Pemilih jam bergaya dial ponsel untuk filter waktu: pilih **jam** di dial (cincin luar 00–11, cincin dalam 12–23), lalu **menit** (ketelitian satu menit) dengan mengetuk atau menyeret; panah papan ketik menggeser satu langkah. Nilai dikirim sebagai `HH:MM` saat menekan “Selesai” atau mengklik di luar; `Esc` membuang perubahan; “Hapus” mengosongkannya. Keluarannya persis format yang dibaca server (`hour_from/hour_to` , bab 11).

Komponen grafik (`web/src/components/charts/`)

Seluruhnya SVG buatan sendiri tanpa pustaka grafik, agar kecil dan terkendali. Warna dan gerak dikumpulkan di `colors.ts` dan `motion.ts` .

Komponen	Fungsi
`RadialGauge`	Satu persentase sebagai dial: busur gradien, cincin tanda yang menyala sampai nilai, angka yang menghitung naik
`KpiCard, Sparkline, CountUp`	Angka utama dengan bentuk beberapa hari terakhir (sparkline tanpa sumbu: hanya arah) dan arah perubahan (tanda ±% terhadap jendela sebelumnya, dihilangkan bila tidak ada pembanding); `CountUp` menghitung naik
`AreaChart`	Kurva monoton (Fritsch–Carlson) yang tidak pernah turun di bawah nilai sebenarnya; dua deret: masuk dan selesai
`Donut`	Bagian dari keseluruhan; menyorot satu irisan (atau chip legendanya) meredupkan yang lain dan menampilkan angkanya di tengah
`StaffRings`	Satu cincin kemajuan per petugas; lencana merah bila ada laporan dari luar kampus
`HourClock`	24 jam sebagai jam: tiap jam satu baji yang panjangnya sebanding banyaknya keluhan; jam tersibuk jadi judul
`RadialBars`	Cincin konsentris per kategori, terpanjang untuk yang paling sering; angka dicetak di daftar di samping
`Treemap`	Treemap *squarified*: luas sebanding nilai dan petak tetap mendekati persegi
`AgingBars`	Kolom dari garis dasar, berwarna dari segar sampai terabaikan
`Heatmap`	Matriks hitungan dengan satu rona berurutan; nol tetap berupa permukaan kosong, bukan langkah terpucat, agar “tidak ada” dan “sedikit” tak terlihat sama
`BarChart`	Peringkat besaran, batang mendatar karena label panjang; nilai dicetak di ujung batang sehingga terbaca tanpa warna atau penunjuk
`StackedBarChart`	Komposisi per hari (tinggi, sedang, rendah), bertumpuk karena pertanyaannya komposisi; irisan berjarak dua piksel

Palet (`colors.ts`) mengikuti `design.md` : halaman hampir monokrom dengan satu warna biru, biru menandai yang penting (selesai), abu baja membawa volume (masuk); keduanya berbeda tajam dalam terang dan rona sehingga tetap terbedakan bagi buta warna dan pada cetak hitam-putih. Jangan ganti warna tanpa memeriksa ulang keterbacaannya.

Bahasa, tema, tampilan, responsivitas

- **Dwibahasa ID/EN** penuh pada antarmuka (`lib/i18n.tsx` , kamus `id` dan `en` dengan kunci berbahasa Inggris, mis. `staff.who_are_you`). Pilihan disimpan di `localStorage` kunci `language` ; tanpa pilihan, peramban berbahasa Inggris mendapat EN, lainnya ID. **Hasil AI tetap Indonesia** (ringkasan, rekomendasi, alasan model, jawaban Kato Bot) karena pembaca akhirnya petugas kebersihan; stiker QR selalu dwibahasa.
- **Tema gelap adalah bawaan**; terang hanya bila dipilih lewat sakelar di kepala halaman (`localStorage` kunci `theme` = `light`). Skrip kecil di `index.html` memasang kelas `dark` pada `<html>` sebelum React dimuat agar tidak berkedip putih; `ThemeProvider` menjaga warna `theme-color` peramban ponsel.
- **Glassmorphism** (rujukan `design.md` , gaya Liquid Glass tanpa warna): permukaan kaca buram (`.glass` , `backdrop-filter` dengan `blur` 36 px), tepi spekular, bayangan berlapis, di atas latar aurora. Seluruh warna adalah variabel CSS (RGB) yang dibalik oleh kelas `.dark` ; font Inter dari Google Fonts.

- **Kepala halaman:** lencana UPI, sakelar bahasa dan tema, tombol kembali; setelah digulir, versi ringkas berbentuk pil melayang.
- **Ponsel lebih dulu:** formulir dibuka dari kamera QR; tombol dan dropdown petugas sengaja besar dan polos. Dashboard memakai kolom hingga lebar `max-w-6xl` dan grid yang bertumpuk di layar sempit; bagian yang dicetak (rekap kebersihan, lembar QR) diatur dengan kelas `print:` dan `.print-sheet`.
- **Kamera** (`Camera.tsx`): `getUserMedia` (kamera belakang dipilih bila ada), hanya di HTTPS atau `localhost` ; bingkai digambar ke *canvas* dan diperkecil ke sisi terpanjang 1600 px, dengan cap bila diminta. Tidak ada pemilih berkas, sehingga tidak ada jalur galeri; tanpa kamera tidak ada foto sama sekali.
- **Penampil foto** (`ZoomableImage` → `ImageViewer`): ketuk untuk memperbesar 1–5 ×, geser dan cubit, ketuk dua kali untuk 2,5 ×.

17. Caching dan performa

Bab ini mengumpulkan semua mekanisme yang membuat sistem ringan, dengan penjelasan batasnya. Dasarnya satu fakta platform: **Cache API Cloudflare tidak bekerja di `*.workers.dev`**, tempat aplikasi ini berjalan. Karena itu cache sisi server berupa memori isolate, bukan Cache API, dan sisanya mengandalkan header HTTP ke peramban.

1. Cache memori per-isolate (`adapters/memo.ts`)

`memo(kunci, ttlMs, muat)` menyimpan *janji* hasil muat di sebuah `Map` selama `ttlMs`. Sifatnya:

- **Per isolate.** Setiap instans Worker punya salinannya sendiri. Penulisan di isolate yang sama membuang salinan itu seketika; isolate lain baru tahu saat salinannya kedaluwarsa (paling lama sebesar TTL). Tidak ada penyiaran pembatalan antar-isolate.
- **Pemanggil bersamaan berbagi satu pemuatan** (yang disimpan adalah *promise*-nya), jadi sepuluh dashboard yang memuat bersamaan menghasilkan satu query.
- **Pemuatan yang gagal tidak disimpan** (entri dihapus bila promise ditolak), sehingga galat sesaat tidak membeku selama TTL.

Kunci cache	TTL	Fungsi / data	Dibuang oleh
`areas`	5 menit	`listAreas` : lokasi dan penugasan petugas	`forget('areas')` saat akun petugas berubah (`forgetStaff`)
`staff`	60 detik	`activeStaff` : nama petugas aktif	`forget('staff')` saat akun berubah
`charts:<days>`	60 detik	`chartData` (query terberat)	`reportsChanged()`
`stats:<tanggal>`	15 detik	`dailyStats`	`reportsChanged()`
`summary:<tanggal>`	60 detik	`storedSummary`	`forget('summary')` saat ringkasan dibuat ulang
`leaderboard`	30 detik	20 pelapor teratas	`reportsChanged()`
`recap:<tanggal>`	20 detik	`dailyRecap` : rekap kebersihan harian	`cleaningChanged()` saat laporan kebersihan masuk atau penugasan berubah

`forget(awalan)` membuang semua kunci berawalan itu. `reportsChanged()` membuang `stats`, `charts`, dan `leaderboard` dan dipanggil dari setiap penulisan tabel `reports` : keluhan baru, perubahan status, analisis AI selesai atau gagal, analisis ulang, penghapusan. `cleaningChanged()` membuang `recap`. Ada pula cache ringan di peramban untuk daftar yang jarang berubah (`cachedReq` di `lib/api.ts` : daftar petugas 60 detik, daftar area 5 menit, dibuang oleh klien saat akun diubah).

2. Header `Cache-Control` per rute (`HTTP_CACHE`)

Rute	Header	Alasan

`GET /api/cleaning/areas`	`public, max-age=300, stale-while-revalidate=86400`	Berubah hanya saat seed dijalankan ulang
`GET /api/staff`	`public, max-age=60, stale-while-revalidate=600`	Berubah saat SPV mengubah nama
`GET /api/reports/public`, `GET /api/cleaning/reports/public`	`public, max-age=10, stale-while-revalidate=60`	Papan publik: beberapa detik usang tidak masalah, dan meringankan basis data pada saat ramai
`GET /api/summary/charts`	`private, max-age=60`	Khusus SPV; dipakai per orang
`GET /api/uploads/<kunci>`	`public, max-age=31536000, immutable`	Nama UUID tidak pernah ditimpa
aset di `/assets/*`	`public, max-age=31536000, immutable`	Nama berkas memuat *hash* isi (`web/public/_headers`)
gambar statis (`.svg`, `.png`, `peta-lokasi-gedung.jpg`)	`public, max-age=604800, stale-while-revalidate=86400`	Berubah jarang; seminggu
`index.html`	bawaan (selalu divalidasi ulang)	Deploy baru terlihat pada kunjungan berikutnya

Rute lain (keluhan, jejak audit, akun, statistik, ringkasan, peringkat, Kato Bot) tidak memberi `Cache-Control` sehingga peramban tidak menahannya selain lewat ETag.

3. ETag lemah

`src/index.ts` memasang `etag({weak: true})` (`hono/etag`) pada `/api/summary`, `/activity`, `/cleaning`, `/reports`, `/work-logs`, `/users`, `/staff`, dan `/leaderboard` (beserta anak jalurnya). Dashboard memuat ulang tiap setengah menit; bila isinya sama, peramban mengirim `If-None-Match` dan mendapat **304 tanpa isi**. Foto dikecualikan: ETag-nya datang dari R2 dan menghitung *hash* isi gambar lebih mahal daripada manfaatnya. Batasannya perlu dipahami: ETag menghemat *byte yang dikirim*, bukan kerja server; Worker tetap menjalankan penangannya (dan query, kecuali kena cache memori) sebelum membandingkan. Pada rute yang sudah di-cache memori, hasil kedua mekanisme ini bertumpuk. Terverifikasi di lingkungan lokal: `GET /api/staff`, `/api/reports/public`, dan `/api/cleaning/areas` mengirim ETag dan `Cache-Control` di atas, dan `If-None-Match` yang cocok dijawab 304.

4. Polling yang sopan (`lib/polling.ts`)

`usePolling(fn, ms, enabled)` menjalankan `fn` segera lalu tiap `ms`, **hanya saat `document.hidden` bernilai salah**, dan menjalankannya lagi seketika saat tab terlihat kembali (`visibilitychange`). Dashboard di tab latar tidak meminta apa pun ke server.

Pemakai	Selang	Catatan
Tab Ringkasan	60 detik	Empat endpoint sekaligus
Daftar keluhan	30 detik	`enabled` hanya saat tab Laporan › Keluhan tampil
Hitungan peringatan kebersihan	60 detik	Murah; berjalan di semua tab

Halaman status keluhan	2 detik (maks. 20 kali) lalu 15 detik	Penjadwalan sendiri; melewati putaran saat tab tersembunyi
Tab Kebersihan	30 detik	**Memakai `setInterval` biasa, bukan `usePolling`**: tetap memuat saat tab peramban tersembunyi selama sub-tab ini terpasang (`CleaningPanel.tsx`)

5. Pemecahan kode dan muat malas

- Setiap halaman selain Home, enam panel dashboard, dan isi Kato Bot adalah *chunk* terpisah (`React.lazy` + `import()`). Pelapor yang memindai QR tidak mengunduh dashboard, pustaka QR, atau grafik. Hasil `npm run build` memuat 25 berkas di `dist/assets`.
- Chunk **vendor React** terpisah (`web/vite.config.ts`, `manualChunks: {react: [react, react-dom, react-router-dom]}`): berkas ini jarang berubah, sehingga *hash*-nya bertahan antar-deploy dan tersimpan di peramban pengunjung yang kembali.
- **Font** dimuat dengan `<link rel="preconnect">` dan `<link rel="stylesheet">` di `index.html`, bukan `@import` di CSS: peramban memulai unduhan font lebih awal alih-alih menunggu CSS selesai.
- **Aset immutable:** berkas ber-*hash* di `/assets/*` disimpan setahun (bagian 2).

6. Basis data

- Indeks pada tabel utama (dari migrasi): `reports` menurut waktu, status+waktu, area+waktu, toilet+waktu, status AI (parsial, hanya yang bukan `ok`), dan pelapor; `activity_log` menurut waktu, laporan, dan aksi+waktu; `cleaning_reports` menurut waktu, petugas+waktu, area+waktu, dan parsial untuk `location_status <> 'ok'` (peringatan); `cleaning_report_tasks.photo_key` `UNIQUE`; `users` menurut peran+aktif.
- Kumpulan query dashboard dijalankan sebagai *batch* D1 (`env.DB.batch`) dan dua *batch* grafik paralel; daftar dengan banyak parameter dipecah menjadi irisan 90 (`tasksOf`) karena D1 membatasi parameter per pernyataan.
- Analisis AI berjalan sesudah respons (`waitUntil`), jadi pelapor tidak menunggu model.

Apa yang sudah diukur, apa yang belum

> **Diukur (lokal, saat pengembangan):** respons statistik yang kena cache memori sekitar 3 ms. Angka ini berasal dari server pengembangan (`wrangler dev`) pada satu mesin dengan basis data lokal; ia dicatat oleh pengembang dan tidak diulang saat dokumen ini disusun, dan belum diulang di produksi.
> **Terverifikasi dari kode dan lewat header:** TTL, kunci, dan pembatalan di atas; header `Cache-Control` dan ETag pada rute publik; polling berhenti saat tab tersembunyi.

> **Belum diukur di produksi:** waktu respons nyata dari kampus, tingkat kena cache isolate (berapa isolate yang aktif di Cloudflare tidak diketahui), ukuran dan waktu muat tiap *chunk* pada jaringan seluler, dampak filter jam (tanpa indeks) pada jejak audit yang besar, dan waktu rata-rata kueri grafik pada data ratusan ribu baris. Aplikasi ini tidak punya pemantauan kinerja sendiri selain log bawaan (`observability` diaktifkan dan `wrangler tail`). Jangan mengutip angka performa produksi dari dokumen ini sebelum diukur.

Konsekuensi caching yang perlu diingat

- Setelah menjalankan ulang seed lokasi, daftar area dapat tertinggal hingga 5 menit di server dan, karena `stale-while-revalidate=86400`, peramban dapat menampilkan daftar lama sekali lagi sebelum menyegarkan di latar.

- Setelah SPV menonaktifkan nama petugas, daftar nama di HP lain dapat memuat nama itu hingga sekitar satu menit (cache) sampai sebelas menit (*stale*); namun server selalu memeriksa `staff_id` pada setiap tindakan, sehingga nama itu ditolak (400) walau masih terlihat.
- Grafik di dashboard dapat terlambat hingga sekitar dua menit terhadap keadaan sebenarnya dari susunan cache server (60 detik) dan peramban (`max-age=60`), tambah selang polling. Tombol “Buat ulang” ringkasan langsung membuang cache ringkasan di isolate yang menanganinya.

18. Konfigurasi, variabel, dan rahasia

Konfigurasi terbagi tiga: *vars* publik di `wrangler.jsonc` (ter-commit), *secrets* yang dipasang dengan `wrangler secret put` (tidak pernah ada di repositori), dan *binding* ke sumber daya Cloudflare. Tipenya ada di `src/env.ts` (`interface Env`).

Nama	Jenis	Nilai / keterangan
`LLM_BASE_URL`	var	`https://api.deepseek.com` : model teks, mengikuti format *chat completions* OpenAI
`LLM_MODEL`	var	`deepseek-chat`
`VISION_BASE_URL`	var	`https://generativelanguage.googleapis.com/v1beta/openai` : model vision (DeepSeek tidak dapat melihat gambar). Titik akhir apa pun yang kompatibel dengan OpenAI dan menerima `image_url` dapat dipakai
`VISION_MODEL`	var	`gemini-3.6-flash`
`CAMPUS_LAT`, `CAMPUS_LNG`	var (teks)	`-7.3351` , `108.2217` : pusat kampus untuk pemeriksaan GPS laporan kebersihan. Komentar di `wrangler.jsonc` menyebut nilainya **penanda sementara** di Jl. Dadaha; ganti dengan koordinat sebenarnya dari Google Maps
`CAMPUS_RADIUS_M`	var (teks)	`500` meter. Bila bukan angka atau 0, kode memakai 500 (`campus()` di `cleaning-service.ts`)
`LLM_API_KEY`	secret	Kunci model teks. Boleh berisi **beberapa kunci dipisah koma** (`kunci1,kunci2`)
`VISION_API_KEY`	secret, opsional	Kunci model vision; bila kosong jatuh ke `LLM_API_KEY` (hanya berguna bila kedua model di penyedia yang sama). Juga boleh berisi beberapa kunci dipisah koma: bila satu kunci ditolak dengan 401, 403, atau 429, kunci berikutnya dicoba berurutan (`postChat` di `adapters/llm.ts`)
`AUTH_SECRET`	secret	String acak panjang untuk menandatangani JWT sesi. Menggantinya membatalkan semua sesi (bab 20). **Tanpa nilai ini login gagal 500**
`DB`	binding D1	Basis data `kato` ; `migrations_dir: "migrations"`
`BUCKET`	binding R2	Bucket `kato` (bab 14)
`ASSETS`	binding aset	Berkas statis dari `./dist` (hasil build frontend)

Pengaturan lain di `wrangler.jsonc`

Kunci	Nilai dan alasan
`main`	`src/index.ts`
`compatibility_date`, `compatibility_flags`	`2025-08-01` ; `nodejs_compat`
`observability`	`{enabled: true}` : log dan jejak Worker tersimpan di dashboard Cloudflare
`workers_dev`, `preview_urls`	`true` , `false` : aplikasi hanya di `https://ai-complaint.yvrtz.workers.dev` , tanpa URL pratinjau per versi
`build.command`	`node scripts/build-frontend.mjs` : membangun frontend ke `./dist` sebelum deploy (memasang dependensi dulu bila `node_modules/vite` tidak ada), sehingga `wrangler deploy` tidak pernah menemukan `./dist` kosong
`assets`	`run_worker_first: ["/api/*", "/__scheduled"]` (API selalu ke Worker); `not_found_handling: "single-page-application"` (path yang bukan aset menjadi `index.html` agar router React bekerja)
`triggers.crons`	`["0 10 * * *"]` : 10.00 UTC = 17.00 WIB, ringkasan harian (bab 12)
`account_id`	Ditetapkan eksplisit karena token OAuth dapat menjangkau beberapa akun; harus akun pemilik D1 dan R2 `kato`

Variabel di luar Worker

- `SITE_URL` (saat `npm run build`): mengganti `%SITE_URL%` di `index.html` (tag pratinjau tautan perlu URL absolut). Bawaan: `https://ai-complaint.yvrtz.workers.dev` .
- `BASE_URL` (saat `npm run qr`): asal Worker yang daftar areanya dibaca; bawaan `http://127.0.0.1:8787` .

Konstanta yang tertanam di kode (mengubahnya butuh deploy)

Nilai	Di mana	Arti
50 / 20	`services/ask-service.ts`	Pertanyaan Kato Bot per hari (semua orang) / per alamat non-SPV. SPV hanya terkena batas 50
3 jam	`services/cleaning-service.ts`	`PHOTO_MAX_AGE_MS` : usia maksimum foto tugas harian sejak diterima server
200 m	`domain/types.ts`	Batas toleransi akurasi GPS yang dihitung (`locationStatus`)
5 MB	`adapters/storage.ts`	`MAX_BYTES` unggahan foto
12 jam	`adapters/session.ts`	Masa berlaku sesi SPV
100.000 putaran	`adapters/password.ts`	PBKDF2-SHA256 dengan garam per pengguna
2 menit	`repositories/reports.ts`, `work-logs.ts`	Jendela deteksi kiriman kembar
30 detik	`adapters/llm.ts`	Batas waktu tiap panggilan ke model (`AbortSignal.timeout`)
TTL cache	`services/*.ts`	Bab 17

Berkas `.dev.vars`

Untuk pengembangan lokal, salin `.dev.vars.example` menjadi `.dev.vars` (diabaikan git) dan isi tiga rahasia: `LLM_API_KEY`, `VISION_API_KEY`, `AUTH_SECRET`. Berkas ini membaca nilai persis seperti *secrets* produksi. Mengganti penyedia vision: ubah dua *vars* dan satu *secret*; format panggilan yang dikirim adalah OpenAI *chat completions* dengan bagian `image_url` (didukung antara lain OpenAI dan OpenRouter). Mengganti model teks sama caranya.

19. Pengembangan lokal

```
npm install
cp .dev.vars.example .dev.vars    # isi LLM_API_KEY, VISION_API_KEY, AUTH_SECRET
npm run db:local                  # menerapkan migrasi 0001–0011 ke D1 lokal
npm run db:seed:cleaning          # lokasi, nama petugas, penugasan (wajib agar ada lokasi)
npm run db:seed                   # opsional: daftar toilet lama (hanya untuk data lama)
npm run dev                       # Vite (5173) + Worker (8787) berdampingan
```

Buka `http://localhost:5173`. Perintah `npm run dev` menjalankan tiga proses sekaligus (`concurrently`): `db` (`scripts/watch-db.mjs`), `api` (`wrangler dev --port 8787`), dan `web` (Vite, yang meneruskan `/api` ke `127.0.0.1:8787`, sehingga tidak perlu CORS). Sebelumnya, skrip `predev` (`scripts/free-dev-port.mjs`) menghentikan Worker lama *dari proyek ini* yang masih memegang port 8787.

- `watch-db.mjs` menerapkan migrasi yang belum dijalankan saat mulai dan setiap kali berkas di `migrations/` berubah, serta menjalankan ulang berkas di `seed/` setiap kali disimpan. Seed tidak dijalankan otomatis saat mulai; basis data kosong butuh `db:seed:cleaning` sekali.
- Kedua seed aman dijalankan ulang. `seed/cleaning-areas.sql` menonaktifkan semua area lalu mengaktifkan yang tercantum, menambah nama petugas hanya bila belum ada nama aktif yang sama, dan **menghapus lalu membangun ulang seluruh penugasan area**.
- Migrasi yang sudah diterapkan tidak dijalankan ulang bila isi berkasnya diubah (D1 mencatat namanya). Perubahan skema = berkas migrasi baru.
- Akun SPV bawaan berasal dari migrasi 0004 (username `admin`; kata sandinya tercatat di berkas itu dan karena itu publik). Untuk lokal tidak masalah; untuk produksi wajib diganti (bab 20).
- `build.command` di `wrangler.jsonc` juga membangun `./dist` saat `wrangler dev` dimulai; direktori aset itu harus ada. Perubahan frontend terlihat lewat Vite (5173), bukan lewat 8787.

Halaman	URL lokal
Beranda	`http://localhost:5173/`
Formulir keluhan, tujuan QR (contoh area C-2)	`http://localhost:5173/report/C-2`
Halaman petugas; stiker tugas harian	`http://localhost:5173/staff`, `/staff/C-2`, `/clean/C-2`
Papan publik / peringkat	`http://localhost:5173/reports`, `/leaderboard`
Dashboard SPV	`http://localhost:5173/supervisor`

Kamera hanya berjalan di HTTPS atau `localhost`; membuka server pengembangan dari ponsel lewat alamat IP LAN biasa (HTTP) tidak akan menyalakan kamera, dan GPS pun sering diblokir pada konteks yang tak aman.

Perintah npm

Skrip	Fungsi
`dev`	Pemantau basis data + Worker (8787) + Vite (5173) bersamaan; `predev` membebaskan port 8787 lebih dulu
`dev:api` / `dev:web`	Worker saja / Vite saja

`build`	`vite build` ke `./dist`
`typecheck`	`tsc --noEmit` untuk `src/`, `web/src/`, dan `scripts/`. Satu-satunya pemeriksaan otomatis; tidak ada uji unit
`db:local / db:remote`	Menerapkan migrasi yang belum dijalankan (`wrangler d1 migrations apply kato --local|--remote`)
`db:seed / db:seed:remote`	Menjalankan `seed/toilets.sql` (toilet lama)
`db:seed:cleaning / db:seed:cleaning:remote`	Menjalankan `seed/cleaning-areas.sql` (lokasi, petugas, penugasan)
`qr`	`scripts/generate-qr.mjs`: membaca `/api/cleaning/areas` dari `BASE_URL` dan menulis satu PNG per lokasi serta lembar cetak ke `qr-codes/` (diabaikan git; folder dikosongkan lebih dulu). Alternatif tanpa skrip: tab QR lokasi di dashboard
`docs`	Merender dokumen di `docs/` ke PDF lewat Chrome headless
`cf-typegen`	Menghasilkan tipe *binding* dari wrangler
`deploy`	`wrangler deploy` (membangun frontend lebih dulu lewat `build.command`)

Jebakan yang nyata

- **Login gagal 500** padahal sandi benar: `AUTH_SECRET` kosong atau `.dev.vars` tidak terbaca. Penandatanganan JWT melempar galat dan `app.onError` menjawab “Terjadi kesalahan pada server”. Isi `AUTH_SECRET` (string acak apa pun) dan mulai ulang `wrangler dev`; perubahan `.dev.vars` tidak terbaca tanpa mulai ulang.
- **Dua proses `wrangler dev`.** Proses kedua tidak mendapatkan port 8787 sementara Vite terus meneruskan `/api` ke proses pertama, jadi perubahan kode backend “tidak berpengaruh”; keduanya juga berbagi satu berkas SQLite lokal di `.wrangler/`. `predev` hanya menghentikan sisa Worker dari proyek ini; proses dari tempat lain hanya dilaporkan. Hentikan yang lama, lalu jalankan ulang. Vite memakai `strictPort`: `npm run dev` kedua gagal alih-alih membuka 5174.
- **Daftar lokasi kosong** atau QR menjawab “Kode lokasi tidak dikenal”: `db:seed:cleaning` belum dijalankan pada basis data itu.
- **Tanpa kunci vision sungguhan** penutupan keluhan selalu gagal 502 (“no API key configured” dicatat sebagai `verification_failed`), dan analisis teks berakhir `ai_status = failed`. Aplikasi lain tetap berjalan; gunakan server tiruan (di bawah).
- Cron lokal tidak berjalan sendiri: `wrangler dev --test-scheduled` lalu buka `/__scheduled` (pada port Worker).

Menguji tanpa kunci API sungguhan

Kedua model dipanggil lewat URL yang dapat dikonfigurasi, sehingga keduanya dapat diganti server tiruan lokal yang menjawab format *chat completions* (`choices[0].message.content` berisi JSON). Jalankan Worker dengan `wrangler dev --var VISION_BASE_URL:http://127.0.0.1:8799/v1` (dan `LLM_BASE_URL` bila perlu), dengan kunci apa pun yang tidak kosong di `.dev.vars`. Skenario yang layak diperiksa:

- **Keluhan:** `resolved` tanpa foto (400); foto bersih (200, kolom `proof_*` terisi); foto kotor (422 `dirty`) dan bukan lokasi (422 `wrong_place`), dengan fotonya hilang dari R2 (404 saat dibuka); kunci foto berawalan `report/` (400); `staff_id` tak dikenal (400); layanan mati (502, tercatat `verification_failed`).

- **Klaim:** `in_progress` oleh petugas A (200), oleh B (ambil alih, tercatat dengan `handed_over_from`), oleh B lagi (200 tanpa baris log baru), membuka kembali keluhan selesai (409).
- **Tugas harian:** foto dipakai dua tugas (400), foto yang sudah dipakai laporan lain (422), kirim ganda dengan foto sama (200, `duplicate:true`), tanpa GPS atau di luar radius (201 dengan `location_status` `unknown` / `far` dan baris `cleaning_out_of_range`).
- **Akun:** hapus akun sendiri (400), SPV terakhir (400), akun dengan riwayat (409 dengan `records`), akun kosong riwayat (200).

20. Penyebaran dan operasi

Urutan penyebaran

```
npx wrangler secret put LLM_API_KEY      # sekali, atau saat kunci diganti
npx wrangler secret put VISION_API_KEY
npx wrangler secret put AUTH_SECRET

npm run typecheck
npx wrangler d1 export kato --remote --output ~/kato-cadangan.sql   # cadangan dulu, di luar
repositori
npm run db:remote                   # migrasi D1 produksi: SEBELUM deploy
npm run db:seed:cleaning:remote     # hanya bila daftar lokasi/petugas/penugasan berubah
npm run deploy                      # wrangler deploy: membangun frontend lalu mengunggah Worker +
aset
```

- **Frontend dibangun otomatis.** `wrangler deploy` menjalankan `build.command` (`scripts/build-frontend.mjs`) lebih dulu; pada checkout bersih tanpa `node_modules/vite` ia memasang dependensi dengan `npm ci`. Tidak perlu `npm run build` manual.
- **Urutan penting: `db:remote` sebelum `deploy`.** Kode membaca tabel dan view hasil migrasi terbaru (mis. `location_info` dan `reports.area_id` dari 0011). Bila kode baru ter-deploy lebih dulu, API gagal dengan galat SQL (`no such table`, `no such column`) sampai migrasi diterapkan. Periksa status dengan `npx wrangler d1 migrations list kato --remote`; setiap berkas diterapkan atomik.
- **Cadangkan sebelum migrasi yang membangun ulang tabel** (0007, 0008, 0010, dan 0011 melakukannya). Migrasi tidak dapat dibatalkan; satu-satunya jalan pulang adalah memulihkan dari ekspor. Simpan ekspor di luar repositori: berkas `backup.sql` yang pernah ter-commit memuat baris akun beserta *hash* sandinya dan jejak audit.
- Galat sesaat `code 7403` pernah muncul sekali saat `db:remote` padahal izin benar; jalankan ulang perintahnya.
- **Cron** (`0 10 * * *`) terdaftar otomatis dari `wrangler.jsonc` saat deploy; tidak ada langkah tambahan. Verifikasi esok harinya: baris `daily_summary` pada pukul 17.00 WIB di tab Log aktivitas, atau kartu ringkasan di tab Ringkasan.
- **Deploy otomatis dari GitHub** (Workers Builds), bila disambungkan, memakai perintah deploy `npx wrangler deploy` dan `build.command` yang sama. Konfigurasi itu ada di dashboard Cloudflare, bukan di repositori, dan tidak dapat diverifikasi dari sini. Rahasia dan migrasi **tidak** ikut otomatis: jalankan `db:remote` dengan tangan sebelum mendorong perubahan yang membawa migrasi baru.

Pemeriksaan sesudah deploy

1. `curl https://ai-complaint.yvrtz.workers.dev/api/health` mengembalikan `{"ok":true,"time":…}`.
2. `curl -I https://ai-complaint.yvrtz.workers.dev/api/staff` menunjukkan `Cache-Control` dan `ETag` (bab 17); `/api/cleaning/areas` mengembalikan daftar lokasi.

3. `npx wrangler deployments list` menunjukkan versi baru; `npx wrangler secret list` memuat ketiga rahasia; `npx wrangler d1 migrations list kato --remote` bersih.
4. Dari ponsel: buka QR sebuah lokasi, kirim satu keluhan berfoto, lihat hasil analisis AI muncul, klaim lalu selesaikan dengan foto bukti di halaman petugas, kirim satu tugas harian, lalu hapus keluhan uji dari dashboard (salinannya tetap ada di log aktivitas).
5. Masuk ke `/supervisor`; semua tab terbuka tanpa galat. Log langsung: `npx wrangler tail`.

Risiko dan pembatalan

- **Tab yang terbuka sebelum deploy dapat menjadi layar kosong (tampak hitam pada tema gelap).** Berkas JavaScript dibangun dengan nama ber-*hash*; deploy baru mengganti namanya dan berkas lama tidak lagi disajikan. Tab lama yang kemudian membuka halaman atau panel yang dimuat malas meminta *chunk* yang sudah tidak ada, impor gagal, dan karena aplikasi tidak punya penangkap galat (tidak ada *error boundary* atau penangan `vite:preloadError`), pohon React runtuh. Pemulihannya cukup memuat ulang tab. Deploy sebaiknya tidak dilakukan saat dashboard dan petugas sedang aktif; perbaikan jangka panjang tercantum di bab 23.
- `npx wrangler rollback` mengembalikan kode Worker ke versi sebelumnya, tetapi **tidak** membatalkan migrasi D1. Rollback kode ke versi yang tidak cocok dengan skema yang sudah dimigrasi akan gagal; karena itu migrasi harus kompatibel mundur atau dipasangkan dengan rencana pemulihan dari ekspor.
- **Setelah mengganti nama cookie atau `AUTH_SECRET`**, semua orang keluar sekali; SPV cukup masuk lagi. Pilihan bahasa, tema, dan nama petugas tersimpan di peramban masing-masing dan tidak terpengaruh.
- **Menonaktifkan atau menghapus akun SPV tidak mengakhiri sesinya** yang sedang berjalan (JWT tanpa pemeriksaan basis data, maksimal 12 jam). Untuk memutuskan akses seketika, ganti `AUTH_SECRET`.
- `preview_urls` dimatikan: tidak ada tempat mencoba versi baru sebelum menjadi produksi. Uji dulu secara lokal (bab 19).

> **Wajib dilakukan sebelum dipakai sungguhan:** ganti kata sandi SPV bawaan (tab Petugas & Supervisor → Ubah, atau `PATCH /api/users/adm-0001`). Nilainya tercatat di berkas migrasi 0004 yang ter-commit, sehingga harus dianggap publik. Sesudah itu tambahkan atau periksa nama-nama petugas di tab yang sama agar muncul di pilihan nama, dan pastikan koordinat `CAMPUS_LAT` / `CAMPUS_LNG` adalah pusat kampus yang sebenarnya.

21. Pemantauan dan pemecahan masalah

Satu-satunya pemantauan bawaan adalah jejak audit di basis data (bab 11), log Worker (`npx wrangler tail` , dashboard *observability* Cloudflare), dan `GET /api/health` . Semua *query* di bawah dijalankan dengan `npx wrangler d1 execute kato --remote --command "…"` (tambahkan `--local` untuk basis data lokal). `details` adalah JSON, jadi `json_extract` dapat dipakai; baris dari sebelum migrasi 0008 berkunci Indonesia dan akan menghasilkan NULL.

Menghitung pemakaian model vision dari aplikasi sendiri

```
SELECT date(created_at, '+7 hours') AS hari, action, COUNT(*) AS jumlah
  FROM activity_log
 WHERE action IN ('proof_rejected', 'verification_failed')
    OR (action = 'status_changed'
        AND json_extract(details, '$.verification.verdict') = 'clean')
 GROUP BY hari, action ORDER BY hari DESC;
```

`status_changed` dengan putusan `clean` adalah foto yang diterima, `proof_rejected` yang ditolak, `verification_failed` panggilan yang gagal; jumlahnya per hari kira-kira sama dengan jumlah panggilan ke penyedia vision hari itu. (Foto tugas harian tidak diperiksa model, jadi tidak ikut.) Angka batas dan pemakaian resmi ada di konsol penyedia. `LLM HTTP 429` di log berarti kuota habis.

Membaca galat verifikasi terakhir

```
SELECT created_at, actor, summary, details FROM activity_log
 WHERE action = 'verification_failed' ORDER BY id DESC LIMIT 3;
```

Menelusuri tindakan atas nama seorang petugas

```
SELECT datetime(created_at, '+7 hours') AS wib, action, summary, details
  FROM activity_log
 WHERE actor = 'Nama Petugas'
   AND action IN ('status_changed', 'cleaning_reported', 'cleaning_out_of_range',
                  'proof_rejected', 'verification_failed')
 ORDER BY id DESC LIMIT 50;
```

Pengambilalihan, laporan kebersihan, dan Kato Bot

```
-- berapa kali klaim diambil alih, dan dari siapa ke siapa
SELECT datetime(created_at, '+7 hours') AS wib, actor,
       json_extract(details, '$.handed_over_from') AS dari, summary
  FROM activity_log
 WHERE action = 'status_changed' AND json_extract(details, '$.handed_over_from') IS NOT NULL
 ORDER BY id DESC;

-- laporan kebersihan 7 hari terakhir per petugas, dan berapa yang ditandai
SELECT staff_name, COUNT(*) AS laporan, SUM(location_status <> 'ok') AS ditandai
  FROM cleaning_reports WHERE created_at >= datetime('now', '-7 days')
 GROUP BY staff_name ORDER BY laporan DESC;

-- pemakaian Kato Bot hari ini (hari WIB)
SELECT COUNT(*) FROM activity_log
 WHERE action = 'question' AND created_at >= datetime(date('now', '+7 hours'), '-7 hours');
```

Latensi dan tingkat keberhasilan (bahan bab hasil)

```
-- analisis teks
SELECT ai_status, COUNT(*) n, ROUND(AVG(ai_ms)) avg_ms, MIN(ai_ms) min_ms, MAX(ai_ms) max_ms
  FROM reports GROUP BY ai_status;
-- verifikasi foto bukti yang diterima
SELECT COUNT(*) n, ROUND(AVG(proof_ms)) avg_ms FROM reports WHERE proof_verdict = 'clean';
-- tingkat penolakan foto
SELECT action, COUNT(*) FROM activity_log
 WHERE action IN ('proof_rejected', 'verification_failed') GROUP BY action;
-- sebaran prioritas dan kategori
SELECT priority, COUNT(*) FROM reports GROUP BY priority;
SELECT je.value AS category, COUNT(*) FROM reports, json_each(categories) je GROUP BY je.value;
```

Gejala → penyebab → tindakan

Gejala	Penyebab yang lazim	Tindakan
“Pemeriksaan foto gagal. Coba lagi sebentar.” (502) terus-menerus	Lihat `verification_failed`: `LLM HTTP 404 … no longer available` = nama model kedaluwarsa; `401/403` = kunci salah atau kosong; `429` = kuota habis; “Respons LLM bukan JSON” = JSON terpotong atau model tidak patuh; *timeout* 30 detik; “no API key configured” = rahasia belum dipasang	Ganti `VISION_MODEL` ke nama yang disebut galat; pasang ulang `VISION_API_KEY` (boleh beberapa kunci dipisah koma agar yang kena batas digantikan); pindah ke model lebih ringan atau aktifkan penagihan
Foto bukti ditolak berulang (422 “masih terlihat kotor” atau “tidak menunjukkan lokasi yang dilaporkan”)	Foto gelap atau buram, sudut sempit, lantai masih basah merata, atau foto tempat lain. Alasan model tampil di kotak “Foto ditolak”	Foto ulang dari lokasi yang dilaporkan dengan cahaya cukup, setelah benar-benar bersih; ikuti alasan model
“Kamera tidak bisa dibuka”	Situs dibuka lewat HTTP, izin kamera ditolak, perangkat tanpa kamera, kamera dipakai aplikasi lain; sebagian *WebView* di dalam aplikasi chat tidak mengizinkan kamera	Buka lewat HTTPS di Chrome/Safari; izinkan kamera di pengaturan situs; tutup aplikasi lain; tekan “Coba lagi”

“Nama petugas tidak dikenal. Pilih ulang namamu.” (400)	Nama sudah dinonaktifkan atau dihapus SPV, atau HP menyimpan id dari basis data lain	Pilih nama lagi; bila tidak ada, minta SPV menambah atau mengaktifkannya
“Belum ada nama petugas.” di daftar nama	Belum ada akun berperan `staff` yang aktif (basis data baru tanpa seed)	SPV: tab Petugas & Supervisor → Tambah petugas; atau `db:seed:cleaning`
“Laporan ini sudah selesai.” (409) saat mengklaim atau menutup	Petugas lain menutupnya lebih dulu; keluhan selesai tidak dapat dibuka lagi	Muat ulang halaman; tidak ada yang perlu dilakukan
Dua petugas mengerjakan keluhan yang sama, atau “diambil alih” di log	Klaim bukan kunci; siapa pun boleh mengambil alih sebelum ada foto bukti	Sesuai rancangan. Telusuri pemegang klaim lewat `handed_over_from` (query di atas)
Foto tugas harian ditolak 422 (“sudah terlalu lama” atau “sudah pernah dipakai”)	Lebih dari 3 jam sejak foto diterima server; atau foto yang sama sudah membuktikan tugas lain	Ambil foto baru lalu kirim. Foto lama tidak dapat dihidupkan kembali
Banyak laporan kebersihan bertanda “di luar kampus” atau “tanpa GPS”	Izin lokasi ditolak atau GPS lemah di dalam gedung; atau `CAMPUS_LAT/LNG/RADIUS_M` belum sesuai dengan kampus	Periksa koordinat di `wrangler.jsonc`, naikkan radius bila perlu, minta petugas mengizinkan lokasi. Laporan tetap diterima dan terhitung di rekap
Kartu keluhan tanpa label AI, “Analisis gagal” > 0	DeepSeek tidak terjangkau atau kunci salah; lihat `ai_error`	Perbaiki kunci, tekan “Analisis ulang”
Kato Bot: “Batas 50 pertanyaan per hari sudah tercapai” (429) atau “Model tidak dapat menjawab saat ini” (502)	Anggaran harian habis (dihitung dari baris `question`), atau DeepSeek bermasalah	Tunggu hari WIB berikutnya, atau ubah `DAILY_LIMIT` dan deploy; periksa kunci dan log
Ringkasan harian kosong pukul 17.05	Cron gagal (LLM), belum ada keluhan, atau cron belum aktif sejak deploy terakhir	Lihat `daily_summary` di jejak audit dan log Worker; tekan “Buat ulang”
Login gagal 500	`AUTH_SECRET` kosong atau tidak terpasang	`wrangler secret list`; pasang bila hilang. Lokal: isi `.dev.vars` dan mulai ulang
Semua pengguna terlempar keluar	`AUTH_SECRET` diganti, atau sesi 12 jam habis	Wajar; masuk kembali
SPV yang dinonaktifkan atau sandinya diganti masih bisa memakai dashboard	Sesi JWT tidak diperiksa ke basis data dan berlaku sampai 12 jam	Ganti `AUTH_SECRET` untuk memutus semua sesi seketika
Layar kosong/hitam pada tab yang sudah	*Chunk* JavaScript lama sudah tidak ada (bab 20)	Muat ulang tab

lama terbuka, padahal baru ada deploy		
Angka dashboard tertinggal atau berbeda antar-perangkat	Cache memori per-isolate (15–60 detik) dan cache peramban (grafik 60 detik)	Tunggu satu sampai dua menit; bila terus berbeda, periksa data langsung dengan query di atas
Galat SQL `no such table` / `no such column` / `location_info`	Deploy sebelum migrasi	`npm run db:remote`
`db:remote` gagal dengan `code 7403`	Galat sesaat di sisi Cloudflare	Jalankan ulang; migrasi per berkas bersifat atomik
QR menjawab “Kode lokasi tidak dikenal”	Area itu dinonaktifkan oleh seed, atau QR dari daftar lama yang idnya sudah tidak ada	Periksa `/api/cleaning/areas`; perbarui seed, jalankan `db:seed:cleaning:remote`, cetak ulang dari tab QR lokasi. Stiker lama `/lapor/…` dan bentuk `<gedung>–<lantai>` tetap dikenali
Daftar lokasi atau nama petugas belum berubah setelah seed atau pengubahan akun	Cache area 5 menit dan nama 60 detik per isolate, ditambah cache peramban	Tunggu beberapa menit; muat ulang
Menghapus akun ditolak 409	Akun punya laporan kebersihan, laporan pekerjaan lama, atau keluhan sebagai pelapor	Gunakan Nonaktifkan; sesuai rancangan
Foto tidak tampil	Seharusnya tidak terjadi (disajikan dari *origin* sendiri); periksa awalan kunci	Hanya `report/ proof/ work/ cleaning/ laporan/ bukti/ kerja/` yang disajikan
Ikon Kato Bot menutupi tombol di halaman	Ikon melayang di semua halaman	Seret ke tempat lain (posisi tersimpan di `localStorage` `chat-anchor`); hapus kunci itu untuk mengembalikannya ke kanan bawah
Skrip atau alat lama memanggil `/api/lokasi` dan mendapat 404	Path API lama tidak dialihkan	Ganti ke path baru (bab 15)
Push ke GitHub tidak men-deploy	Workers Builds belum tersambung, cabang salah, atau build gagal (konfigurasi ada di dashboard Cloudflare)	Periksa tab Builds di dashboard; sementara itu `npm run deploy`
Lokal: `npm run dev` gagal karena port terpakai	Proses `wrangler dev` atau Vite lain masih hidup	Hentikan yang lama (bab 19)

22. Keputusan rancangan yang berbiaya

Masing-masing adalah keputusan yang mengorbankan sesuatu di tempat lain, sehingga alasan dan konsekuensinya dicatat. Yang paling baru ada di paruh kedua daftar.

Analisis berjalan setelah respons dikirim

Pelapor menerima konfirmasi seketika sementara panggilan LLM berlanjut di latar (`ctx.waitUntil`); halaman konfirmasi melakukan polling. Panggilan sinkron akan menaruh jeda beberapa detik antara pelapor dan tombol kirim. Biayanya: hasil analisis belum ada saat laporan pertama kali terbaca; bila model gagal, laporan ditandai gagal (tetap tersimpan, dapat dianalisis ulang).

Panggilan LLM yang gagal tidak pernah menghilangkan laporan

Baris ditandai `failed` beserta teks galatnya, tetap tampil, dan dapat dianalisis ulang. Sistem menurun menjadi alat pelaporan biasa, bukan gagal.

Keluaran LLM tidak pernah dipercaya apa adanya

Setiap respons divalidasi Zod lalu dipaksa ke enum yang dikenal basis data, untuk teks maupun vision. Kata dari pelapor tidak pernah menjadi perintah: Kato Bot memilih dari daftar tetap *query* agregat dan tidak menulis SQL.

Penyelesaian menuntut bukti yang diperiksa, ditolak di server

Selesai tanpa foto → 400; foto yang dinilai kotor atau bukan lokasi yang dilaporkan → 422 beserta alasan. Aturan ada di server (`canResolve`), bukan sekadar tombol yang disembunyikan. Biayanya: petugas menunggu beberapa detik dan sistem bergantung pada penyedia model kedua. Sebaliknya, **pemadaman layanan vision tidak dilooloskan** (502): bukti yang tak dapat diperiksa tidak dianggap lolos.

Foto hanya dari kamera

`<input type="file" capture>` hanya petunjuk: banyak peramban tetap menawarkan galeri, sehingga foto lama dapat diaku sebagai foto baru. Mengambil bingkai langsung dari `getUserMedia` meniadakan galeri. Biayanya: tanpa kamera, tanpa HTTPS, atau tanpa izin, tidak ada foto sama sekali, dan itu disengaja tanpa cadangan. Kamera juga memperkecil foto sebelum unggah sehingga batas 5 MB tidak menjadi masalah di jaringan seluler.

Jejak audit sengaja tanpa foreign key

Laporan yang *dihapus* adalah yang paling perlu terlacak. Foreign key akan memaksa baris audit ikut terhapus, kebalikan dari tujuannya. Seluruh isi laporan disalin ke `details` sebelum barisnya hilang, dan nama orang disimpan sebagai teks. Biayanya: tidak ada jaminan integritas referensial, jadi `report_id` bisa menunjuk laporan yang tak ada, dan itulah maksudnya.

Melapor tidak butuh akun, tetapi akun dihargai

Dinding pendaftaran di pintu menurunkan jumlah laporan. Laporan berakun masuk papan peringkat; laporan anonim tetap diterima dan ditangani, hanya tidak diperingkat. Laporan menempel pada akun, bukan perangkat, agar tidak hilang saat data situs dibersihkan. Biayanya: pelapor anonim mudah disalahgunakan untuk spam (bab 23).

Foto tidak melewati domain pihak ketiga

Disajikan dari *origin* sendiri lewat Worker (bab 14), karena DNS `r2.dev` dibajak sebagian ISP Indonesia. Biayanya: setiap foto melewati Worker (dimitigasi header `immutable`).

Antarmuka dwibahasa, analisis tetap Indonesia

Label mengikuti pembaca; ringkasan, rekomendasi, dan alasan model selalu Indonesia karena yang bertindak adalah petugas kebersihan. Keluhan berbahasa Inggris tetap dipahami dan diringkas dalam bahasa Indonesia.

Semua kode berbahasa Inggris, teks pengguna tetap Indonesia

Satu kosakata dari basis data sampai frontend (tabel, kolom, enum, path API, URL, kunci penyimpanan, cookie); yang dibaca manusia tetap Indonesia. Biayanya: satu migrasi besar (0008) yang membangun ulang setiap tabel dan satu kali keluar-sesi untuk semua orang.

URL lama dialihkan

Stiker QR sudah tertempel dengan `/lapor/A-1` dan tautan lama beredar. Mencetak ulang semua stiker mahal dan pasti ada yang terlewat, jadi sebelas rute pengalihan dipertahankan di router frontend, dan `locate()` memahami id lantai bentuk lama. Biayanya: pengalihan hanya di peramban; path API lama tidak ikut dialihkan.

Pertanyaan dijawab lewat alat, bukan membaca tabel

Memasukkan seluruh baris ke prompt tumbuh linear dengan data dan membuka jalan teks pelapor masuk ke prompt. Dengan *tool calling*, biaya per pertanyaan tetap (kira-kira 3–4 ribu token), model hanya memilih query dari daftar tetap, hak akses (nama petugas) ditegakkan di server, dan teks mentah keluhan tidak pernah dikirim ke model. Batas 50 pertanyaan per hari dan 20 per alamat menjaga agar fitur publik ini tidak menguras anggaran.

Warna grafik dihitung, bukan dikira-kira

Palet grafik diperiksa terhadap rentang terang, ambang chroma, keterpisahan buta warna, dan kontras, dan ramp berurutan peta panas dibuat monoton dalam luminansi. Jangan mengubahnya tanpa memeriksa ulang.

Lokasi umum menggantikan toilet

Keluhan awalnya hanya soal toilet. Yang dilaporkan sekarang adalah kondisi kebersihan *lokasi mana pun*: lantai gedung, halaman, gedung utuh, atau tempat di luar gedung seperti Lapang Voli. Daftar lokasi dipakai ulang dari area kebersihan (`cleaning_areas`), sehingga pelapor, petugas, dan SPV memilih dari satu daftar, dan satu stiker QR per lokasi melayani pelapor dan petugas sekaligus (sakelar peran). Migrasi 0011 menambah `reports.area_id` berdampingan dengan `toilet_id` (tepat salah satu terisi, dijaga `CHECK`) dan view `location_info` yang menyatukan keduanya, agar semua query membaca lokasi dengan cara sama. Putusan `not_toilet` menjadi `wrong_place` , dan prompt vision menanyakan “apakah ini lokasi yang dikeluhkan”, bukan “apakah ini toilet”. Biayanya: dua jalur data berdampingan (toilet lama dan area baru) dan tabel `reports` dibangun ulang; kesesuaian foto dengan lokasi yang beragam lebih sulit dinilai model daripada kloset.

Klaim bukan kunci, dan ada ambil alih

Mengunci keluhan bagi satu orang membuat keluhan yang ditinggalkan tertahan selamanya (petugas pulang, ganti shift, ponsel mati). Maka mengklaim hanya penanda “sedang dikerjakan oleh X”; siapa pun boleh mengambil alih sampai ada foto bukti yang lolos, dan itu tercatat di jejak audit (“X mengambil alih … dari Y”). Biayanya: dua petugas bisa mengerjakan hal yang sama tanpa tahu, dan pemegang klaim tidak diberi tahu. Klaim ulang oleh orang yang sama tidak ditulis dua kali agar log tidak penuh.

Keluhan yang sudah selesai tidak dapat dibuka kembali

Membuka kembali akan menghapus foto bukti yang menutupnya, dan itu menghapus satu-satunya jejak bahwa keluhan itu pernah ditangani. Permintaan membuka dijawab 409. Biayanya: keluhan yang kambuh harus dilaporkan sebagai keluhan baru.

Nama ditanyakan saat bertindak

Satu ponsel dipakai bergantian dan petugas jarang memakai aplikasi, jadi menanyakan “Siapa kamu?” sekali di halaman membuat tindakan tercatat atas nama orang yang salah. Kini setiap klaim, ambil alih, dan penyelesaian menanyakan siapa pelakunya, dengan nama terakhir di ponsel sebagai isian awal (dua ketukan). Hanya tab Tugas harian, yang milik satu orang sepanjang putaran, meminta nama di muka. Biayanya: satu langkah tambahan per tindakan. Petugas tetap hanya sebuah nama tanpa login; yang menjaga kejujuran catatan adalah bukti foto dan jejak audit atas nama yang dipilih.

Laporan kebersihan terpisah dari keluhan

Laporan kebersihan tidak punya prioritas, kategori, atau status. Menyimpannya di `reports` akan menggelembungkan jumlah keluhan, mengacaukan grafik dan ringkasan, muncul di papan keluhan, dan ikut di peringkat. Tabel terpisah (`cleaning_reports` , `cleaning_report_tasks`) menjaga statistik keluhan jujur. Biayanya: laporan kebersihan tidak menutup keluhan secara otomatis; petugas menutupnya sendiri dengan foto bukti.

Bukti tugas harian: waktu server, satu foto satu tugas, GPS menandai

Laporan lewat WhatsApp pernah memakai foto lama sebagai bukti hari ini. Maka waktu yang dihitung adalah *saat server menerima foto* (bukan jam ponsel), foto lebih dari 3 jam ditolak, satu foto hanya dapat membuktikan satu tugas (`UNIQUE` pada `photo_key`), dan tiap tugas dikirim sendiri-sendiri agar foto diunggah saat diambil. Posisi GPS di luar radius atau hilang **tidak menolak** laporan, hanya menandainya untuk SPV: GPS di dalam gedung sering meleset puluhan meter, dan laporan yang jujur tidak boleh hilang karena sinyal. Toleransinya memakai akurasi yang dilaporkan ponsel, dibatasi 200 m. Biayanya: foto tugas harian tidak diperiksa isinya oleh model, jadi foto yang baru tetapi tidak sesuai tugas masih lolos; pengawasan jatuh ke SPV lewat rekap dan foto.

Dropdown buatan sendiri yang selalu membuka ke bawah

`<select>` bawaan memilih arah sendiri: ke atas bila merasa sempit, di tengah layar pada ponsel, dan CSS tidak dapat mengubahnya, sehingga pilihan sering terpotong atau menutupi isian lain. `Select` buatan sendiri selalu membuka ke bawah dan, bila kurang ruang, menggulir atau memperpanjang halaman sementara. Biayanya: ratusan baris kode dan perilaku aksesibilitas (papan ketik, ARIA) yang kini ditanggung sendiri, bukan peramban.

Hapus akun hanya bila tanpa riwayat

Menghapus akun yang punya pekerjaan di belakangnya akan membuat riwayat yatim atau terhapus. Aturannya: akun dengan laporan kebersihan, laporan pekerjaan lama, atau keluhan sebagai pelapor ditolak (409) dan disarankan dinonaktifkan, yang menyembunyikan nama sama efektifnya. Akun sendiri dan SPV aktif terakhir tidak dapat dihapus. Biayanya: nama yang pernah bekerja tidak akan pernah hilang dari daftar akun (hanya nonaktif), dan keluhan yang hanya diklaim petugas tidak dihitung sebagai riwayat (bab 23).

Ketukan diproses saat pointer diangkat

Pada ikon Kato Bot yang dapat diseret, menunggu `click` setelah sentuhan yang bergeser sedikit tidak andal; bendera yang menunggu menelan satu `click` dapat menelan ketukan *berikutnya* (“harus ketuk dua kali”). Maka tekan tanpa menggeser melewati ambang diproses langsung saat jari diangkat (`pointerup`); `click` hanya diterima dari papan ketik. Biayanya: penanganan petunjuk (pointer) sendiri dan `touch-action: none` pada ikon.

Cache memori per-isolate, bukan Cache API

Cache API Cloudflare tidak bekerja di `*.workers.dev` . Cache di memori isolate sederhana dan dapat dibuang seketika pada penulisan di isolate yang sama. Biayanya: tiap isolate punya salinan sendiri; isolate lain tertinggal hingga TTL (15–60 detik, area 5 menit), dan cache hilang saat isolate didaur ulang. Untuk data dashboard hal itu dapat diterima; untuk data yang harus selalu mutakhir cache tidak dipakai (otorisasi tindakan selalu memeriksa basis data).

ETag lemah di samping cache

Dashboard memuat ulang tiap setengah menit; ETag membuat pemuatan yang tidak berubah berupa 304 tanpa isi. Biayanya: server tetap bekerja sebelum membandingkan (ETag menghemat byte, bukan kerja), dan foto dikecualikan karena menghitung *hash* gambar lebih mahal daripada manfaatnya.

Stiker QR dwibahasa

Stiker cetak tidak dapat mengikuti sakelar bahasa peramban, sedangkan pemindainya bisa siapa saja. Maka tiap stiker memuat judul dan petunjuk dalam Indonesia dan Inggris sekaligus (satu baris saja bila keduanya sama, mis. “Lapang Voli”). Biayanya: teks stiker lebih panjang.

Ringkasan harian satu per tanggal, hanya keluhan

Satu baris per tanggal (`date` kunci utama) dan *upsert* membuat “Buat ulang” sederhana dan aman. Biayanya: tidak ada riwayat versi ringkasan, dan ringkasan tidak mencakup laporan kebersihan.

23. Batasan yang diketahui dan pekerjaan lanjutan

Batasan	Dampak	Arah perbaikan
Petugas tanpa login: nama dapat dipakai orang lain	Siapa pun yang membuka halaman petugas dapat mengklaim, mengambil alih, menyelesaikan, atau mengirim laporan kebersihan atas nama petugas mana pun; yang menahannya hanya foto (untuk penutupan keluhan, yang harus lolos AI) dan jejak audit	PIN pendek per petugas, atau pembatasan per perangkat; tinjau jejak audit per nama (bab 21)
Pemegang klaim dikenali dari *nama*	`reports` hanya menyimpan `staff_name`, bukan `staff_id`. Penanda “Kamu yang mengerjakan ini” membandingkan nama; dua petugas bernama sama tampak sebagai satu orang, mengganti nama akun memutus kaitan dengan klaim lamanya, dan akun yang dihapus tidak terdeteksi punya klaim	Simpan `staff_id` pada keluhan di samping nama
Foto tugas harian tidak diperiksa isinya	Hanya usia (≤ 3 jam sejak diterima server), keunikan, dan GPS yang diperiksa; model vision tidak melihatnya. Foto yang baru tetapi bukan pekerjaan yang diklaim lolos	Periksa foto tugas dengan model vision, atau sampel acak untuk SPV
Asal foto dari kamera tidak dapat dibuktikan server	Kamera-saja ditegakkan di antarmuka; orang yang paham teknis dapat memanggil API langsung dengan foto apa pun. Tidak ada pemeriksaan EXIF atau perangkat	Tantangan acak di bingkai, atau pembandingan dengan foto keadaan
Cache per-isolate dapat tertinggal sampai TTL	Setelah penulisan di satu isolate, isolate lain menyajikan angka lama hingga 15–60 detik (area 5 menit); ditambah cache peramban. Angka dashboard antar-perangkat dapat berbeda sesaat	Pembatalan lintas-isolate (mis. Durable Object atau KV) bila perlu; saat ini dianggap cukup
Sesi tidak dapat dicabut	JWT diperiksa tanpa basis data: SPV yang dinonaktifkan, dihapus, atau diganti sandinya tetap punya akses hingga 12 jam. Peran di token juga tidak disegarkan	Periksa akun aktif pada `requireSupervisor`, atau daftar sesi yang dicabut
Kata sandi SPV bawaan bersifat publik	Nilainya tercatat di migrasi 0004 yang ter-commit; sebelum diganti, siapa pun yang membaca repositori dapat masuk ke dashboard. `backup.sql` di repositori juga memuat *hash* akun dan jejak audit lama	Ganti segera (bab 20); keluarkan `backup.sql` dari repositori dan tambahkan ke `.gitignore`;

		pertimbangkan kata sandi awal yang dibangkitkan saat penyiapan
Keluhan terbuka dan catatan petugas dapat dibaca tanpa login	`GET /api/reports/open` memuat teks asli dan foto keadaan pelapor; catatan bebas petugas tampil di papan publik	Batasi ke lokasi yang dipindai, atau token per stiker; saring catatan
Ringkasan harian hanya satu per hari dan hanya soal keluhan	Tidak mencakup laporan kebersihan; keluhan sesudah 17.00 tidak ikut sampai “Buat ulang” ditekan; menimpa ringkasan sebelumnya	Ringkasan mingguan, atau gabungkan rekap kebersihan
Cakupan waktu grafik tidak seragam	Pemilih 7/14/30/90 hari hanya memengaruhi deret harian, status, dan tren; kategori, prioritas, gedung, jam, dan waktu selesai menghitung sepanjang waktu, sehingga donat tidak saling cocok pada rentang pendek	Beri semua visual jendela yang sama, atau beri label “sepanjang waktu”
Maks. 200 baris per tampilan keluhan dan log pekerjaan (300 untuk jejak audit, 500 untuk laporan kebersihan)	Rentang panjang tidak tampil utuh; hitungan “*n* data ditampilkan” adalah baris yang dimuat, bukan seluruhnya	Paginasi atau hitungan agregat dari server
Filter jam tanpa indeks; jendela malam per hari kalender	Bagian jam dihitung per baris pada rentang tanggal; “22.00–06.00” tidak menyambung satu malam melintasi dua tanggal	Kolom turunan berindeks bila volume menuntut
Hapus akun tidak menghitung keluhan yang diklaim	Petugas yang hanya pernah mengklaim atau menutup keluhan dapat dihapus; namanya tetap tertulis di `reports.staff_name` dan jejak audit	Hitung `reports.staff_name` atau simpan `staff_id`
`resolved` dikirim ulang pada keluhan yang sudah selesai tidak ditolak	Menimpa `resolved_at` dan nama penyelesai (dan foto bukti bila disertakan; foto lama tidak dihapus dari R2). UI tidak melakukannya, tetapi API terbuka	Tolak 409 atau abaikan bila sudah selesai
Tab lama menjadi layar kosong setelah deploy	Nama *chunk* berganti dan aplikasi tidak menangkap kegagalan impor	Pasang `vite:preloadError` untuk memuat ulang otomatis, atau *error boundary* dengan tombol muat ulang
Tab Kebersihan memuat ulang meski tab peramban tersembunyi	`CleaningPanel` memakai `setInterval` sendiri, bukan `usePolling`	Pindah ke `usePolling`
Filter jenis aksi tidak memuat `user_changed`	Baris pengelolaan akun hanya terlihat tanpa filter (daftar pilihan di `ActivityPanel.tsx` melewatkannya), padahal API menerimanya	Tambahkan ke daftar `ACTIONS`
`photo_key` keluhan hanya dicek awalannya	`POST /api/reports` menerima kunci berawalan `report/` yang tidak ada di R2; keluhan tanpa foto nyata dapat tersimpan lewat API langsung	Periksa keberadaan objek seperti pada foto bukti

Tidak ada pembatasan laju atau CAPTCHA	`POST /api/reports` dan `POST /api/uploads` terbuka; spam dan foto yatim (diunggah tetapi tak dilampirkan) bisa menumpuk di R2; hanya deteksi kembar 2 menit yang menahan. SPV dapat menghapus (tercatat)	Pembatasan laju per IP lewat Cloudflare, Turnstile, dan pembersihan foto yatim terjadwal
Tidak ada kebijakan retensi	Foto, koordinat GPS, nama, dan jejak audit disimpan tanpa batas waktu	Tetapkan masa simpan dan pembersihan terjadwal
Tidak ada notifikasi ke petugas	Petugas melihat keluhan hanya saat membuka halaman petugas; pemegang klaim tidak diberi tahu bila klaimnya diambil alih	Web Push atau WhatsApp/Telegram untuk prioritas tinggi
Putusan model bersifat probabilistik	Mungkin salah tolak atau salah terima; belum ada ukuran akurasi terhadap penilaian manusia. Lokasi yang beragam lebih sulit dinilai daripada toilet	Uji dengan set foto berlabel; laporkan presisi dan *recall*
Ketergantungan pada penyedia model	Kuota harian (tingkat gratis), nama model yang berumur pendek (404 mendadak saat ditutup), data pada tingkat gratis dapat dipakai penyedia	Beberapa kunci terpisah koma (sudah didukung), model ringan, penagihan; pantau `verification_failed`
Tidak ada alur lupa sandi	SPV yang lupa harus minta SPV lain mengatur sandi baru; SPV terakhir yang lupa harus diperbaiki lewat SQL	Reset lewat e-mail atau OTP
Tidak ada uji otomatis	Repositori tidak punya berkas uji maupun skrip uji; hanya `npm run typecheck`. Verifikasi dilakukan manual (curl dan server model tiruan, bab 19)	Vitest dengan `unstable_dev` wrangler untuk rute dan `buildTimeFilter`, `changeStatus`, `locationStatus`
Data lokasi dan penugasan bergantung pada seed	Kampus berubah (lantai, gedung, penugasan) lewat penyuntingan `seed/cleaning-areas.sql` dan penjalanan ulang; seed membangun ulang seluruh penugasan. Pusat kampus pada `wrangler.jsonc` bertanda penanda sementara	Layar pengelolaan lokasi dan penugasan untuk SPV; koordinat yang dipastikan
Path API lama dan materi lama	URL halaman lama dialihkan hanya oleh router peramban; path API lama menjawab 404; PDF atau catatan lama yang menyebut tabel atau nama toilet tidak berlaku	Perbarui materi lama; pertahankan pengalihan selama stiker lama masih tertempel
Keluhan berbahasa selain Indonesia/Inggris	Belum diuji	—
Penyelesaian ulang dan pemundurkan status belum dijaga	`changeStatus` (`src/services/report-service.ts`) menolak membuka kembali laporan yang sudah selesai, tetapi tidak menolak `resolved` → `resolved`: nama petugas dan `resolved_at` tertimpa, dan bila foto bukti baru lolos, foto yang lama tertinggal di R2. Status `new` juga diterima untuk laporan yang sedang dikerjakan oleh siapa pun yang mengirim `staff_id` sah	Tolak transisi `resolved` → `resolved` dan setiap transisi ke `new`; hapus foto lama saat bukti diganti
Detail laporan publik membawa seluruh kolom	`GET /api/reports/:id` memilih `r.*` lalu `toDTO` hanya membuang `categories` dan kunci foto, sehingga respons tanpa login juga memuat `reporter_id`,	Proyeksikan hanya kolom yang dipakai halaman status

	`ai_error`, dan metadata analisis. Pelindungnya hanya UUID laporan yang sulit ditebak	
Pertanyaan Kato Bot yang dikelompokkan menurut waktu terpotong 20 baris	`countReports` dan `resolutionTime` (`src/repositories/analytics.ts`) memakai `LIMIT 20`; untuk pengelompokan `date`, `week`, `month`, dan `hour` urutannya menaik menurut label, jadi rentang lebih dari 20 hari hanya menampilkan 20 hari tertua dan pengelompokan jam kehilangan jam 20–23	Urutkan menurun untuk deret waktu, atau naikkan batas dan ringkas hasilnya
Unggah foto tanpa pembatas laju dan tanpa pembersihan	`POST /api/uploads` memeriksa tipe dan ukuran berkas, tetapi tidak membatasi jumlah unggahan per alamat; foto yang tidak pernah dipakai sebuah laporan tetap tersimpan di R2 karena satu-satunya pekerjaan terjadwal hanya menyusun ringkasan harian	Batas laju per alamat dan pembersihan terjadwal untuk foto yatim
Kegagalan ringkasan harian tidak meninggalkan jejak audit	Bila pekerjaan cron gagal, hanya `console.error` di log Worker yang tercatat (`src/index.ts`): tidak ada baris `activity_log` dan tidak ada percobaan ulang, sehingga ringkasan hari itu kosong sampai SPV menekan “Buat ulang”	Catat kegagalan ke jejak audit dan coba ulang beberapa kali
Kinerja produksi belum diukur	Lihat bab 17	Ukur waktu respons dan tingkat kena cache di produksi

24. Lampiran

A. Contoh respons

Id dipendekkan dengan …; waktu `created_at` dan sejenisnya adalah UTC.

`POST /api/reports` → 201


```
{"id":"8f0c…","location":"Gedung Dewi Sartika (Perkuliahan) – Lantai 2","duplicate":false}
```


`GET /api/reports/:id` setelah analisis (DTO lengkap)


```
{"id":"8f0c…","area_id":"C-2","toilet_id":null,
 "description":"Lantai 2 bau banget, lantainya becek, sama sabunnya habis.",
 "status":"in_progress","staff_name":"Nama Petugas","resolved_at":null,
 "ai_status":"ok","priority":"high",
 "summary":"Area berbau menyengat, lantai tergenang air, dan sabun habis.",
 "recommendation":"Keringkan lantai lebih dulu karena berisiko membuat pengguna terpeleset.
Setelah itu bersihkan sumber bau dan isi ulang sabun.",
 "ai_error":null,"ai_model":"deepseek-chat","ai_ms":1130,
 "proof_verdict":null,"proof_reason":null,"proof_model":null,"proof_ms":null,
 "reporter_id":null,"created_at":"2026-10-29 03:05:12","updated_at":"2026-10-29 03:20:41",
 "location_name":"Gedung Dewi Sartika (Perkuliahan) – Lantai 2",
 "building_code":"C","building_name":"Dewi Sartika","floor":2,
 "categories":["odor","flooding","supplies"],
 "photo_url":"/api/uploads/report/2026-10-29/….jpg","proof_photo_url":null}
```


`created_at` 03.05 UTC tampil di dashboard sebagai “Masuk: Kam, 29 Okt 2026 · 10.05 WIB”. Untuk keluhan lama, `area_id` null dan `toilet_id` terisi. `floor` 0 = halaman, null = gedung utuh atau tempat di luar gedung. `GET /api/reports/open` mengembalikan bentuk yang sama tanpa `reporter_id, ai_error, ai_model, ai_ms, proof_model, proof_ms`.

`GET /api/reports/public`


```
{"data":[{"id":"8f0c…","area_id":"C-2","status":"resolved","priority":"high","categories":
["odor","flooding","supplies"],
  "summary":"Area berbau menyengat, …","ai_status":"ok","created_at":"2026-10-29
03:05:12","resolved_at":"2026-10-29 04:12:40",
  "location_name":"Gedung Dewi Sartika (Perkuliahan) – Lantai 2","building_code":"C","floor":2,
  "proof_photo_url":"/api/uploads/proof/2026-10-29/….jpg"}],
 "counts":{"total":42,"resolved":37}}
```

`PATCH /api/reports/:id`

```
// klaim: {"status":"in_progress","staff_id":"3b9e…"}
{"ok":true,"status":"in_progress","verification":null}

// selesai, foto diterima: {"status":"resolved","staff_id":"3b9e…","proof_photo_key":"proof/2026-
10-29/….jpg"}
{"ok":true,"status":"resolved",
 "verification":{"verdict":"clean","reason":"Lantai kering dan bersih, tidak ada
genangan.","model":"gemini-3.6-flash","ms":4210}}

// 422: foto ditolak
{"error":"Lokasi pada foto masih terlihat kotor. Bersihkan lagi, lalu foto ulang.",
 "verdict":"dirty","reason":"Lantai masih basah dan ada tisu bekas."}

// 422: bukan lokasi yang dilaporkan  (verdict: "wrong_place")
{"error":"Foto tidak menunjukkan lokasi yang dilaporkan. Ambil foto lokasi itu setelah
dibersihkan.",
 "verdict":"wrong_place","reason":"Foto memperlihatkan koridor, bukan lantai yang dikeluhkan."}

// 409: keluhan sudah selesai
{"error":"Laporan ini sudah selesai."}
```

`POST /api/cleaning/reports`

```
// permintaan
{"staff_id":"3b9e…","area_id":"C-2","tasks":[{"task":"toilet","photo_key":"cleaning/2026-10-
29/….jpg"}],
 "note":"Air sempat mati","position":{"latitude":-7.3352,"longitude":108.2219,"accuracy":18}}
// 201
{"id":"c41d…","area":"Gedung Dewi Sartika (Perkuliahan) - Lantai
2","duplicate":false,"location_status":"ok","distance_m":25}
// 422: foto terlalu lama (sejak diterima server)
{"error":"Foto membersihkan toilet sudah terlalu lama. Ambil foto baru.","task":"toilet"}
```

`GET /api/cleaning/recap?date=2026-10-29` **(dipotong)**

```
{"date":"2026-10-29",
 "staff":[{"staff_id":"3b9e…","staff_name":"Nama
Petugas","expected":12,"done":9,"reports":9,"alerts":1,
   "areas":[{"area_id":"C-2","area_name":"Gedung Dewi Sartika (Perkuliahan) - Lantai
2","assigned":true,
     "tasks":{"sweep_mop":{"done_at":"2026-10-29 00:41:10","times":1,"flagged":false},
              "glass":{"done_at":null,"times":0,"flagged":false}}}]}],
 "totals":{"expected":12,"done":9,"reports":9,"alerts":1}}
```

`DELETE /api/users/:id` → **409**

```
{"error":"Akun ini punya 12 catatan (laporan atau pekerjaan). Nonaktifkan saja agar riwayatnya
tetap utuh.","records":12}
```

`GET /api/summary/stats`

```
{"date":"2026-10-29","today":
{"total":9,"high":2,"medium":4,"low":3,"ai_failed":0},"unresolved":14,
 "top_locations":[{"location":"Gedung Dewi Sartika (Perkuliahan) - Lantai 2","count":3}]}
```

`GET /api/summary/charts?days=14` **(dipotong)**


```
{"daily":[{"date":"2026-10-16","total":5,"resolved":4}, …],
 "dailyByPriority":[{"date":"2026-10-16","high":1,"medium":3,"low":1}, …],
 "statuses":[{"status":"new","count":6},{"status":"in_progress","count":3},
{"status":"resolved","count":38}],
 "aging":[2,3,4,3,2],
 "hourByDay":[{"day":1,"hour":9,"count":7}, …],
 "matrix":[{"building_code":"C","category":"odor","count":12}, …],
 "resolutionByPriority":[{"priority":"high","count":9,"minutes":71.4}, …],
 "categories":[{"category":"cleanliness","count":31}, …], "priorities":
[{"priority":"high","count":14}, …],
 "buildings":[{"building_code":"C","building_name":"Dewi Sartika","count":27}, …],
 "resolution":{"count":38,"minutes":104.2},
 "trend":{"days":14,"reports":47,"previous_reports":40,"change":18,"resolved":38,"high":9}}
```


`GET /api/activity` **(dua entri)**


```
{"data":[
 {"id":412,"created_at":"2026-10-29
09:21:09","action":"status_changed","report_id":"8f0c…","actor":"Petugas B",
  "summary":"Petugas B mengambil alih laporan di Gedung Dewi Sartika (Perkuliahan) – Lantai 2 dari
Petugas A",
  "details":{"from":"in_progress","to":"in_progress","handed_over_from":"Petugas
A","proof_photo":null,"verification":null}},
 {"id":405,"created_at":"2026-10-29
09:14:02","action":"report_deleted","report_id":"77aa…","actor":"Administrator",
  "summary":"Menghapus laporan di Lapang Voli",
  "details":{"location_name":"Lapang Voli","description":"tes
tes","status":"new","priority":"low",
             "categories":"[\"other\"]","summary":"Teks bukan keluhan.","created_at":"2026-10-29
09:10:44","had_photo":true}}]}
```


Keluaran mentah model yang diharapkan


```
// analisis keluhan
{"categories":["odor","flooding","supplies"],"priority":"high","summary":"…","recommendation":"…"}
// ringkasan harian
{"summary":"Hari ini masuk 9 laporan, terbanyak di …","highlights":["Tangani genangan di C-
2","…"]}
// pemeriksaan foto bukti (verdict diturunkan server: is_location=false -> wrong_place;
clean=false -> dirty; selain itu clean)
{"is_location":true,"clean":false,"confidence":0.9,"reason":"Lantai masih basah dan ada tisu
bekas."}
```


B. Daftar migrasi

Berkas di `migrations/`, diterapkan berurutan oleh `npm run db:local` / `db:remote`; masing-masing atomik.

Berkas	Isi
`0001_init.sql`	Skema awal (toilet, keluhan, ringkasan harian) dengan nama kolom Indonesia
`0002_gedung_upi.sql`	Nama gedung UPI Tasikmalaya dalam tabel sendiri; satu QR per lantai
`0003_bukti_dan_aktivitas.sql`	Foto bukti penyelesaian; tabel jejak audit (tanpa foreign key)

`0004_pengguna.sql`	Akun berperan menggantikan sandi bersama; akun admin bawaan; `pelapor_id` pada keluhan
`0005_verifikasi_bukti.sql`	Putusan model vision atas foto bukti (hasil, alasan, model, ms)
`0006_laporan_pekerjaan.sql`	Laporan pekerjaan petugas ("sudah saya bersihkan"), terpisah dari keluhan
`0007_peran_spv.sql`	Tiga pihak; petugas tanpa username/sandi; tabel pengguna dibangun ulang
`0008_english_schema.sql`	Seluruh skema, enum, dan data diterjemahkan ke bahasa Inggris; tabel dibangun ulang
`0009_cleaning_reports.sql`	Area kebersihan, penugasan, laporan kebersihan, dan tugas per laporan (foto `UNIQUE`)
`0010_daily_tasks.sql`	Enam tugas harian yang sama di setiap area; `sweep` / `mop` tetap sah untuk data lama
`0011_report_locations.sql`	`reports.area_id` berdampingan dengan `toilet_id`; view `location_info`; `not_toilet` menjadi `wrong_place`

C. Perintah yang berguna

Perintah	Untuk
`npx wrangler d1 migrations list kato --remote`	Migrasi yang belum diterapkan di produksi
`npx wrangler d1 export kato --remote --output <berkas>`	Cadangan basis data produksi
`npx wrangler d1 execute kato --remote --command "…"`	Menjalankan query (baca) pada produksi; tambahkan `--local` untuk lokal
`npx wrangler secret put|list <NAMA>`	Memasang / melihat nama rahasia
`npx wrangler deployments list`	Riwayat versi yang dideploy
`npx wrangler rollback`	Kembali ke versi kode sebelumnya (tidak membatalkan migrasi)
`npx wrangler tail`	Log Worker langsung
`npx wrangler dev --test-scheduled`	Mengaktifkan `/__scheduled` untuk memicu cron lokal
`curl https://ai-complaint.yvrtz.workers.dev/api/health`	Pemeriksaan hidup

D. Nilai tetap (enum)

Nama	Nilai	Di mana
Peran (`users.role`)	`supervisor, staff, reporter`	`domain/types.ts`
Status keluhan	`new, in_progress, resolved`	`reports.status`
Prioritas	`low, medium, high`	`reports.priority`
Kategori	`cleanliness, supplies, damage, odor, flooding, other`	`reports.categories (JSON)`

Status analisis	`pending, ok, failed`	`reports.ai_status`
Putusan foto bukti	`clean, dirty, wrong_place`	`reports.proof_verdict`
Status lokasi laporan kebersihan	`ok, far, unknown`	`cleaning_reports.location_status`
Tugas harian	`sweep_mop, glass, toilet, trash, gallon, tidy`	`cleaning_report_tasks.task (data lama juga: sweep, mop)`
Jenis foto / awalan kunci	`report, proof, cleaning, work (lama: laporan, bukti, kerja)`	`adapters/storage.ts`
Jenis aksi jejak audit	Empat belas nilai, bab 11	

E. Penyimpanan di peramban

Kunci	Jenis	Isi dan pemakai
`session`	cookie HttpOnly	JWT sesi SPV atau pelapor, 12 jam; diatur server
`language`	localStorage	`id` atau `en`
`theme`	localStorage	Hanya bernilai `dark` / `light`; selain `light` berarti gelap
`selected-staff`	localStorage	Id petugas terakhir yang dipilih di ponsel itu (isian awal nama; bukan identitas)
`chat-anchor`	localStorage	Posisi ikon Kato Bot `{side?, x, y}`

F. Glosarium istilah ↔ nama di kode

Istilah (Indonesia)	Nama di kode / basis data
Pelapor	`reporter` (peran); `reports.reporter_id`; pelaku anonim `reporter`; pintu “Saya pelapor” → `/student`
Petugas kebersihan	`staff` (peran); `staff_id`, `staff_name`; `/staff`, `/api/staff`
SPV (supervisor)	`supervisor` (peran, dulu `admin` / `spv`); `/supervisor`; `requireSupervisor`
Pengguna, akun	`users`; `/api/users`
Lokasi, area kebersihan	`cleaning_areas` (`id`, `name`, `building_code`, `floor`, `tasks`); view `location_info`; `/api/cleaning/areas`
Gedung; lantai; halaman; gedung utuh	`buildings` (`code`, `name`); `floor` bilangan; `floor = 0`; `floor` NULL
Penugasan	`cleaning_assignments` (`area_id`, `staff_id`)
Keluhan (laporan dari pelapor)	`reports`; teks = `description`; `/api/reports`; `/report/:locationId`, `/reports`
Status baru / diproses / selesai	`new` / `in_progress` / `resolved`; waktu selesai = `resolved_at`
Klaim, ambil alih	`PATCH` dengan `status: in_progress`; `details.handed_over_from`

Prioritas rendah / sedang / tinggi	`low` / `medium` / `high`
Kategori kebersihan, perlengkapan, kerusakan, bau, genangan, lainnya	`cleanliness, supplies, damage, odor, flooding, other`
Ringkasan, rekomendasi, sorotan	`summary`, `recommendation`, `highlights`
Analisis gagal	`ai_status = 'failed'`; `analysis_failed`; `/reanalyze`
Foto keadaan	`photo_key` / `photo_url`; awalan `report/`
Foto bukti	`proof_photo_key` / `proof_photo_url`; awalan `proof/`
Putusan bersih / kotor / bukan lokasinya	`proof_verdict`: `clean` / `dirty` / `wrong_place`; alasan = `proof_reason`
Laporan kebersihan, tugas harian	`cleaning_reports`, `cleaning_report_tasks`; `/api/cleaning/reports`; awalan `cleaning/`; aksi `cleaning_reported`
Di luar kampus / tanpa GPS	`location_status` `far` / `unknown`; aksi `cleaning_out_of_range`
Rekap harian per petugas	`GET /api/cleaning/recap`; `dailyRecap()`
Laporan pekerjaan (versi lama)	`work_logs`; `/api/work-logs`; awalan `work/` (lama `kerja/`); aksi `work_logged`. Tab “Log pekerjaan”
Jejak audit, log aktivitas	`activity_log` (`action`, `actor`, `summary`, `details`); `/api/activity`
Ringkasan harian	`daily_summaries` (`date`, `report_count`); `/api/summary`
Grafik, statistik	`/api/summary/charts`, `/api/summary/stats`
Kato Bot, tanya data	`FloatingChat`, `AskPanel`; `/api/ask`; `ask-service.ts`; aksi `question`
Papan peringkat	`/leaderboard`, `/api/leaderboard`
Papan publik	`/reports`, `/api/reports/public`, `/api/cleaning/reports/public`
Filter waktu (tanggal dan jam)	`from, to, hour_from, hour_to`; `time-filter.ts`, `TimeFilter.tsx`, `ClockPicker.tsx`
Stiker QR	`QrCodesPanel.tsx`; `scripts/generate-qr.mjs`; URL `/report/<id area>`

G. Istilah teknis

Istilah	Arti dalam dokumen ini
LLM	*Large language model*; model bahasa terlatih yang dipakai apa adanya lewat API
Model vision	LLM yang menerima gambar sebagai masukan (*multimodal*)

JSON mode / `response_format`	Permintaan agar model menjawab dengan JSON valid saja
Tool calling	Model memilih fungsi yang didefinisikan server dan argumennya; server menjalankannya dan mengembalikan hasilnya
Model “berpikir”	Model yang menghasilkan token penalaran internal sebelum jawaban; ikut menghabiskan `max_tokens`
Worker	Fungsi tanpa server di jaringan tepi Cloudflare yang menjalankan seluruh backend dan menyajikan frontend
Isolate	Instans ringan tempat Worker berjalan; memori cache aplikasi hidup di dalamnya dan tidak dibagi antar-isolate
D1 / R2	Basis data SQLite terkelola / penyimpanan objek Cloudflare
Binding	Cara Worker mengakses D1/R2 tanpa kredensial jaringan
Migrasi	Berkas SQL bernomor yang mengubah skema secara berurutan; tiap berkas diterapkan atomik
Seed	Data awal (lokasi, nama petugas, penugasan) yang aman dijalankan ulang
JWT	Token sesi bertanda tangan; disimpan di cookie HttpOnly `session`
PBKDF2	Fungsi turunan kunci untuk menyimpan sandi secara aman
ETag	Sidik isi respons; peramban mengirimnya kembali (`If-None-Match`) dan server menjawab 304 tanpa isi bila tidak berubah
Code splitting, *chunk*	Memecah JavaScript menjadi berkas-berkas yang diunduh sesuai kebutuhan
`getUserMedia`	API peramban untuk kamera langsung; hanya tersedia di HTTPS dan localhost
`waitUntil`	Mekanisme Workers untuk melanjutkan pekerjaan setelah respons dikirim
Cron	Pemicu terjadwal Cloudflare; di sini 17.00 WIB setiap hari
WIB	Waktu Indonesia Barat, UTC+7; semua “hari” dan “jam” dalam aplikasi mengikuti WIB, sedangkan basis data menyimpan UTC
Haversine	Rumus jarak lingkaran besar antara dua koordinat, dipakai untuk jarak ke pusat kampus

Sumber dokumen ini: `docs/dokumentasi-teknis.html` (render ulang dengan `npm run docs`).